

RESTAURANT DIGITAL PLATFORM

MASTER PRODUCT REQUIREMENTS DOCUMENT (PRD)

Volume 1

Vision, Product Strategy, Brand Identity & Product Philosophy

DOCUMENT INFORMATION

Project Name

Restaurant Digital Platform

Version

1.0

Prepared For

Restaurant Client

Prepared By

Senior Product Strategist

Senior Product Designer

Senior UI/UX Architect

Creative Director

Frontend Architect

Software Architect

Brand Strategist

AI Systems Designer

PROJECT VISION

We are not creating another restaurant website.

We are creating the complete digital identity of a modern restaurant.

The website should become the primary digital touchpoint between the restaurant and its customers while also serving as the operational platform used by the restaurant staff.

The platform must communicate quality before the customer even tastes the food.

Every interaction should reinforce the feeling that this restaurant is clean, modern, trustworthy, premium and memorable.

The experience should be elegant enough to impress first-time visitors while remaining intuitive enough for customers of all ages.

The goal is not merely to display information.

The goal is to increase footfall, online orders, customer retention and overall brand value.

LONG TERM VISION

Create a restaurant platform that feels comparable to what customers expect from premium restaurant brands.

The customer should immediately feel that the restaurant values

Quality

Freshness

Authenticity

Cleanliness

Speed

Professionalism

Technology

Hospitality

The restaurant should appear significantly more premium than its competitors despite operating in the same market.

PRIMARY GOALS

The platform must accomplish five major objectives.

Goal 1

Increase Online Orders

The website should make ordering effortless.

Customers should be able to browse the menu quickly.

Customize food.

Place orders.

Pay securely.

Receive updates.

Track status.

Goal 2

Increase Walk-in Customers

The website should encourage people nearby to visit the restaurant.

It should showcase

Food

Ambience

Cleanliness

Chef

Kitchen

Customer reviews

Location

Directions

Parking

Working hours

Goal 3

Increase Brand Trust

Every page should communicate

Professionalism

Consistency

Transparency

Fresh Ingredients

High Quality

Authentic Taste

Fast Service

Goal 4

Improve Restaurant Operations

The same platform should help restaurant staff manage

Orders

Kitchen

Billing

Inventory

Customer data

Analytics

without switching between multiple applications.

Goal 5

Future Ready

The architecture should support future additions like

Mobile App

Loyalty Program

Delivery Fleet

AI Features

WhatsApp Automation

without major redesign.

PRODUCT PHILOSOPHY

Technology should disappear.

The customer should never think about the software.

They should only think

"This restaurant looks amazing."

Ordering should feel effortless.

Navigation should feel natural.

Animations should feel meaningful.

Nothing should feel decorative for the sake of decoration.

Every visual decision must support usability.

DESIGN PHILOSOPHY

The interface should combine

Apple's simplicity

Linear's precision

Framer's motion

Stripe's professionalism

Awwwards-level visual polish

Luxury restaurant storytelling

Editorial food photography

Japanese minimalism

Modern hospitality branding

The result should feel timeless rather than trendy.

Avoid flashy effects that will feel outdated after one year.

BRAND PERSONALITY

Imagine the restaurant as a person.

The restaurant is

Warm

Welcoming

Confident

Honest

Modern

Professional

Premium

Energetic

Friendly

Passionate about food

Fast without feeling rushed

Luxury without feeling expensive

Technology-driven without feeling robotic

BRAND VALUES

Fresh Ingredients

Authentic Recipes

Consistency

Customer Satisfaction

Cleanliness

Transparency

Innovation

Community

Respect

Quality

Every design decision should reinforce these values.

TARGET AUDIENCE

Primary Audience

Age

18–40

College students

Young professionals

Families

Food lovers

Office workers

People ordering through smartphones

People discovering restaurants on Google

Instagram users

People searching

"Best Momos Near Me"

Secondary Audience

Families

Office Teams

Corporate Lunch Groups

Tourists

Weekend Diners

Late Night Customers

Takeaway Customers

Customer Expectations

Customers expect

Fast loading

Beautiful images

Easy navigation

Clear pricing

Simple ordering

Secure payments

Trustworthy reviews

Live support

Mobile-friendly experience

USER PERSONAS

Persona 1

Rahul

Age 23

College Student

Uses Instagram frequently.

Searches restaurants through Google.

Orders twice a week.

Wants affordable food.

Needs quick checkout.

Persona 2

Neha

Age 29

Working Professional

Orders dinner after work.

Values speed.

Reads reviews.

Prefers online payment.

Likes reorder functionality.

Persona 3

Amit

Age 37

Family Customer

Visits restaurants on weekends.

Looks for hygiene.

Checks gallery.

Needs parking information.

Books tables online.

Persona 4

Pooja

Age 25

Food Blogger

Loves aesthetics.

Shares food on Instagram.

Takes photos.

Values presentation.

Can influence many customers.

CUSTOMER JOURNEY

Discovery

Customer finds restaurant through

Google Search

Instagram

WhatsApp

Friends

Maps

QR Code

Advertisements

Exploration

Customer visits website.

Immediately notices

Beautiful hero section

Premium branding

Mouth-watering photography

Trust indicators

Ratings

Popular dishes

Offers

Consideration

Customer browses

Menu

Gallery

Reviews

Location

Pricing

Availability

Decision

Customer chooses

Visit Restaurant

Book Table

Order Online

Call Restaurant

Message on WhatsApp

Experience

Restaurant fulfills expectations.

Food matches images.

Fast delivery.

Good service.

Easy payment.

Retention

Customer returns.

Reorders.

Shares on Instagram.

Leaves review.

Recommends to friends.

PRODUCT PRINCIPLES

Every feature must satisfy at least one principle.

Principle 1

Beautiful

Every page should look professionally designed.

Principle 2

Fast

Every action should be completed in the shortest possible time.

Principle 3

Simple

Never overwhelm users.

Reduce unnecessary choices.

Principle 4

Accessible

Usable by everyone.

Principle 5

Scalable

Easy to extend.

Principle 6

Consistent

Same spacing.

Same typography.

Same animations.

Same colors.

Same interaction patterns.

Principle 7

Trustworthy

Professional photography.

Clear information.

No misleading content.

Transparent pricing.

BUSINESS OBJECTIVES

Increase online orders by providing a frictionless digital experience.

Reduce phone-based ordering errors through online ordering and structured menus.

Improve customer retention through repeat ordering, WhatsApp communication, and personalized experiences.

Strengthen the restaurant's brand identity with a premium online presence.

Support day-to-day restaurant operations through integrated management tools such as order tracking, billing, inventory, and analytics.

Create a platform that can grow with the restaurant without requiring a complete redesign.

SUCCESS METRICS

The project will be considered successful when it achieves measurable improvements in customer experience and restaurant operations.

Customer Metrics

- Increased online orders.
- Higher average order value.
- Reduced cart abandonment.
- More repeat customers.
- Increased reservations.
- Longer average session duration.
- Lower bounce rate.

Operational Metrics

- Faster order processing.
 - Reduced billing errors.
 - Improved kitchen efficiency.
 - Better inventory visibility.
 - Faster customer support response times.
-

COMPETITIVE POSITIONING

The website should not resemble a generic restaurant template.

Instead, it should visually compete with premium hospitality brands while remaining practical for a single restaurant.

The emphasis should be on exceptional user experience, elegant storytelling, high-quality food presentation, and seamless ordering rather than excessive visual effects.

PROJECT SCOPE

This platform consists of two connected experiences:

Customer Experience

- Premium public website.
- Interactive digital menu.
- Online ordering.
- Reservations.
- Contact & location.
- Gallery.
- Reviews.
- WhatsApp integration.

Restaurant Management Experience

- Admin dashboard.
- Order management.
- POS interface.
- Kitchen Display System (KDS).
- GST billing and invoice generation.
- Thermal printer (KOT) support.
- Inventory management.
- Customer database.
- Sales reports and analytics.

Both experiences must share a single design language, consistent branding, and a unified codebase so the platform feels like one cohesive product.

FINAL PRODUCT STATEMENT

By the end of this project, the restaurant should have a premium digital platform that elevates its brand, improves customer experience, streamlines daily operations, and provides a solid foundation for future enhancements—all while maintaining a polished, agency-quality design that feels modern, elegant, and purpose-built rather than template-based.

RESTAURANT DIGITAL PLATFORM

MASTER PRODUCT REQUIREMENTS DOCUMENT (PRD)

VOLUME 2

DESIGN SYSTEM & DESIGN LANGUAGE

1. INTRODUCTION

The Design System is the single source of truth for the entire Restaurant Digital Platform.

Every page, component, animation, interaction, spacing rule, typography decision, icon, button, and layout must derive from this system.

This is not merely a collection of UI components.

It is the visual language of the restaurant.

Every designer and developer should be able to build new pages without inventing new styles.

The customer should subconsciously recognize consistency across the entire experience.

2. DESIGN PHILOSOPHY

The interface should communicate

Luxury

Warmth

Authenticity

Craftsmanship

Technology

Minimalism

Trust

The visual experience should combine

Apple's simplicity

Linear's precision

Stripe's professionalism

Framer's interactions

Awwwards storytelling

Luxury hospitality branding

Japanese editorial minimalism

Modern restaurant photography

Avoid visual noise.

Avoid excessive gradients.

Avoid over-decoration.

Avoid unnecessary icons.

Avoid clutter.

Every element should earn its place.

3. VISUAL PRINCIPLES

Every screen must follow these principles.

Principle 1

Whitespace is a feature.

Never fill empty space unnecessarily.

Principle 2

Content should breathe.

Spacing creates luxury.

Principle 3

Images tell stories.

Large beautiful photography should always outperform decorative graphics.

Principle 4

Motion supports usability.

Never animate simply because something can move.

Principle 5

Typography creates hierarchy.

The customer should understand the page without consciously reading every word.

4. BRAND COLOR SYSTEM

Primary Background

#0B0B0B

Main website background.

Premium dark charcoal.

Secondary Background

#111111

Alternate sections.

Surface

#181818

Cards

Modals

Navigation

Dropdowns

Elevated Surface

#202020

Hover cards

Floating panels

Dialogs

Primary Accent

#C78A2C

Luxury gold.

Used sparingly.

Buttons

Highlights

Links

Badges

Accent Hover

#E6B84A

Success

#22C55E

Warning

#F59E0B

Error

#EF4444

Primary Text

#FFFFFF

Secondary Text

#C8C8C8

Muted Text

#9E9E9E

Borders

rgba(255,255,255,0.08)

Never use dark borders.

5. TYPOGRAPHY

Typography should create an editorial feel.

Headings

Primary

Playfair Display

Fallback

Cormorant Garamond

Body

Inter

UI Labels

Sora

Numbers

Manrope

Font Scale

Hero

96px

Desktop

H1

72px

H2

56px

H3

42px

H4

32px

H5

24px

Body Large

20px

Body

18px

Small

16px

Caption

14px

Micro

12px

Line Height

Large Headings

110%

Body

165%

Buttons

140%

6. SPACING SYSTEM

Use an 8pt Grid.

Every measurement should be divisible by 4 or 8.

Section

140px

Large Gap

96px

Medium Gap

64px

Standard

40px

Card Padding

32px

Small Padding

24px

Tiny

16px

Icon Gap

12px

Text Gap

8px

7. GRID SYSTEM

Desktop

12 Columns

1440px max width

Laptop

12 Columns

1280px

Tablet

8 Columns

Mobile

4 Columns

Container

max-width:1440px;
padding-inline:48px;

8. BORDER RADIUS

Cards

28px

Buttons

18px

Inputs

16px

Images

32px

Floating Cards

36px

9. SHADOW SYSTEM

Never use harsh shadows.

Luxury websites use soft depth.

Small

0 6px 18px rgba(0,0,0,.18)

Medium

0 18px 50px rgba(0,0,0,.28)

Large

0 40px 90px rgba(0,0,0,.38)

10. GLASS EFFECT

Glass should be subtle.

Never overuse.

`backdrop-filter: blur(18px);`

Opacity

12%

Border

1px

11. BUTTON SYSTEM

Primary

Gold

Filled

Rounded

Hover Lift

Glow

Secondary

Transparent

Border

Hover Fill

Ghost

No background

Underline animation

Floating Action

Circular

Shadow

Blur

Button Height

56px

Radius

18px

12. ICONOGRAPHY

Use

Lucide Icons

Outlined

2px Stroke

Never use emojis.

Never mix icon styles.

13. CARD SYSTEM

Cards should appear premium.

Rounded

Large image

Beautiful spacing

Soft shadow

Hover elevation

Image zoom

Content reveal

Every card should feel collectible.

14. IMAGE STYLE

All images must follow one photography language.

Luxury food photography.

Professional DSLR.

Dark background.

Steam visible.

Golden lighting.

Macro lens.

Ultra realistic.

Restaurant quality.

No AI artifacts.

No unrealistic food.

No random stock photos.

15. ANIMATION SYSTEM

Motion Library

Framer Motion

GSAP

Lenis

Animation Principles

Fast

Smooth

Elegant

Natural

Physics based

Durations

150ms

Hover

300ms

Cards

600ms

Page Entry

1000ms

Hero

Hover

Scale

1.03

Cards

Lift

Blur Shadow

Images

Zoom

105%

Buttons

Glow

Shadow

Scale

Sections

Fade Up

Mask Reveal

Hero

Parallax

Floating food

Text reveal

Gallery

Masonry reveal

Testimonials

Slide

Fade

Counters

Count Up

Progress Bars

Animated Fill

16. MICRO INTERACTIONS

Cursor

Magnetic Buttons

Image Tilt

Ripple

Button Press

Hover Glow

Glass Reflection

Smooth Scroll

Lazy Reveal

Animated Loading

Skeleton UI

Toast Animation

Form Validation

17. FORMS

Rounded

Clean

Large Inputs

Floating Labels

Validation

Instant Feedback

Success States

Loading States

Disabled States

18. TABLES

Admin Dashboard

Premium

Striped

Hover

Sticky Header

Search

Pagination

Filters

Sorting

Export

Responsive

19. CHARTS

Recharts

Minimal

No unnecessary colors

Gold

White

Muted Gray

Dark Background

20. LOADING STATES

Skeleton

Blur

Pulse

Never show blank pages.

21. EMPTY STATES

Illustration

Helpful Message

Call to Action

Never leave empty white space.

22. RESPONSIVE RULES

Desktop First

1440

1280

1024

768

480

360

Touch friendly.

Minimum tap area

48px

23. ACCESSIBILITY

WCAG AA

Keyboard Navigation

ARIA Labels

Visible Focus States

Proper Contrast

Semantic HTML

Reduced Motion Support

24. PERFORMANCE

Target

Lighthouse

Performance

95+

Accessibility

100

SEO

95+

Best Practices

100

CLS

Below 0.1

LCP

Below 2 seconds

25. TAILWIND DESIGN TOKENS

Create a centralized design token system for colors, typography, spacing, radii, shadows, and motion so every component references shared values instead of hard-coded styles.

26. COMPONENT LIBRARY

Every component must be reusable.

Components include:

- Navbar

- Sidebar
- Footer
- Hero Banner
- Buttons (Primary, Secondary, Ghost, Icon)
- Cards (Food, Feature, Testimonial, Analytics)
- Dialogs & Drawers
- Forms & Inputs
- Selects & Dropdowns
- Accordions
- Tabs
- Tables
- Charts
- Badges
- Toast Notifications
- Breadcrumbs
- Pagination
- Search Bar
- Filters
- Carousels
- Gallery Grid
- QR Code Display
- Order Status Timeline
- Statistic Cards
- KPI Widgets
- Empty States
- Loading Skeletons
- Error States

Each component should have defined variants, sizes, hover states, focus states, disabled states, loading states, and dark-mode support.

27. IMAGE GENERATION GUIDELINES

All AI-generated images for the website must follow a consistent creative direction.

Photography Style

- Editorial luxury food photography
- Professional restaurant branding
- Cinematic composition
- Dark premium backgrounds

- Warm golden lighting
- Natural steam and texture
- Shallow depth of field
- DSLR quality
- 4K realism
- Consistent color grading

Required Image Categories

- Hero banner
- Restaurant exterior
- Restaurant interior
- Chef preparing momos
- Steam Momos
- Fried Momos
- Tandoori Momos
- Peri Peri Momos
- Jhol Momos
- Schezwan Momos
- Hot Garlic Momos
- Melted Cheese Momos
- Veg, Paneer, and Chicken Momos
- Signature Chinese dishes
- Fresh ingredients
- Sauces and spices
- Packaging and takeaway
- Delivery lifestyle
- Customer dining experience
- Kitchen preparation
- Team and staff
- Social media promotional shots

Every image should reinforce the restaurant's premium identity and maintain a unified visual style across the entire platform.

28. FINAL DESIGN PRINCIPLE

When a customer opens the website, they should immediately feel:

"This restaurant is premium, trustworthy, modern, and serves exceptional food."

The design system should ensure that whether the user is browsing the public website, placing an online order, viewing the menu, or the restaurant staff is using the admin dashboard or POS, every screen feels like part of one cohesive product—not a collection of unrelated interfaces.

RESTAURANT DIGITAL PLATFORM

MASTER PRODUCT REQUIREMENTS DOCUMENT (PRD)

VOLUME 3

CUSTOMER WEBSITE EXPERIENCE & INFORMATION ARCHITECTURE

1. INTRODUCTION

The public website is the digital face of the restaurant.

It is not simply a place where users read information.

It is the restaurant's

- Brand Ambassador
- Salesperson
- Waiter
- Receptionist
- Marketing Team
- Customer Support
- Menu Card

- Booking Desk

Every section should encourage the visitor to take an action.

The website must create an emotional connection before asking the user to place an order.

2. WEBSITE OBJECTIVES

The website has six primary objectives.

Objective 1

Build Trust

The customer should immediately believe

"This restaurant is premium."

Objective 2

Increase Online Orders

Reduce friction.

One-click ordering.

Fast menu browsing.

Objective 3

Increase Walk-in Customers

Show atmosphere.

Show ambience.

Show quality.

Show hygiene.

Objective 4

Increase Reservations

Easy booking.

Minimal fields.

WhatsApp confirmation.

Objective 5

Increase Customer Retention

Favorites

Offers

WhatsApp

Repeat Orders

Objective 6

Create a Memorable Brand

The customer should remember the restaurant even after leaving the website.

3. WEBSITE INFORMATION ARCHITECTURE

Home

- Hero
- Signature Dishes
- Why Choose Us
- Story
- Featured Menu
- Gallery
- Customer Reviews
- Reservation CTA
- Contact
- Footer

Menu

- Steam Momos
- Fried Momos
- Kurkure Momos
- Tandoori Momos
- Chinese
- Beverages
- Combos

Order Online

Reservation

Gallery

About

Contact

Everything should be reachable within three clicks.

4. NAVIGATION DESIGN

The navigation must remain visible throughout the browsing experience.

Desktop

Restaurant Logo

Home

Menu

Gallery

About

Reservation

Contact

Order Now Button

Sticky Navigation

Glass Background

Blur Effect

Shrink on Scroll

Mobile

Hamburger Menu

Full Screen Drawer

Large Touch Targets

Animated Opening

Quick Access

Call

WhatsApp

Order

5. HOME PAGE EXPERIENCE

The home page is the most important page.

It should follow a storytelling approach.

SECTION ORDER

1 Hero



2 Trust Indicators



3 Signature Dishes



4 About Restaurant



5 Why Choose Us



6 Menu Preview



7 Gallery



8 Customer Reviews



9 Reservation



10 Contact



11 Footer

6. HERO SECTION

This section should immediately create desire.

Layout

Full viewport

Split composition

Large food photography

Minimal text

Left Side

Headline

Subheading

Description

CTA

Order Now

View Menu

Right Side

Large cinematic image

Steam animation

Floating ingredients

Soft parallax

Hero Headline Examples

Handcrafted Momos.

Authentic Flavours.

Unforgettable Taste.

Freshly Steamed.

Perfectly Fried.

Served with Passion.

Where Every Bite

Feels Like Home.

Background

Dark

Luxury

Blurred Restaurant

Warm Lighting

Scroll Indicator

Animated

Minimal

Elegant

7. TRUST BAR

Immediately after Hero.

Display



1000+ Happy Customers

Fresh Ingredients

Fast Delivery

Hygienic Kitchen

Google Rating

Years of Experience

8. SIGNATURE DISHES

Purpose

Immediately showcase best-selling food.

Layout

Three large premium cards

Desktop

Carousel

Mobile

Each Card

Large Image

Dish Name

Short Description

Price

Order Button

Chef Special Badge

Hover Animation

9. ABOUT RESTAURANT

Purpose

Tell the story.

Not company history.

Create emotional connection.

Include

Founder Story

Restaurant Journey

Fresh Ingredients

Handmade Preparation

Traditional Recipes

Modern Kitchen

Layout

Alternating Image + Text

10. WHY CHOOSE US

Grid

3 x 2

Cards

Fresh Ingredients

Premium Quality

Fast Service

Experienced Chefs

Affordable Prices

Hygienic Kitchen

Icons

Minimal

Lucide

Hover

Lift

Glow

11. MENU PREVIEW

Purpose

Encourage customers to explore menu.

Categories

Steam Momos

Fried Momos

Peri Peri

Tandoori

Jhol

Chinese

Combos

Beverages

Each Category

Cover Image

Items

Starting Price

View Menu

12. COMPLETE MENU PAGE

This page should become the centerpiece of the website.

Top Features

Search

Category Filter

Veg Filter

Paneer Filter

Chicken Filter

Price Filter

Spice Filter

Sort

Popular

Newest

Price

Rating

Each Food Card

Large Image

Name

Description

Price

Badges

Veg

Popular

Chef Special

Spice Meter

Preparation Time

Add to Cart

Quick View

Hover

Image Zoom

Card Lift

Shadow

13. PRODUCT DETAIL PAGE

Every menu item deserves its own experience.

Image Gallery

Ingredients

Description

Nutritional Info

Spice Level

Customization

Extra Cheese

Extra Sauce

Extra Mayo

Cooking Instructions

Related Items

Customer Reviews

Add to Cart

14. GALLERY PAGE

Purpose

Build Trust

Sections

Restaurant

Kitchen

Food

Chef

Customers

Events

Interior

Exterior

Layout

Pinterest Masonry

Hover

Zoom

Blur

Overlay

Lightbox

15. TESTIMONIALS

Purpose

Increase credibility.

Each Card

Customer Photo

Name

Rating

Review

Visited Date

Verified Customer Badge

Animation

Carousel

Auto Scroll

Pause on Hover

16. RESERVATION PAGE

Simple

Fast

Minimal

Fields

Name

Phone

Guests

Date

Time

Special Requests

Reserve Button

Confirmation

WhatsApp

SMS

Email

17. CONTACT PAGE

Google Map

Address

Phone

WhatsApp

Email

Opening Hours

Parking

Directions

Social Media

Floating Call Button

Floating WhatsApp Button

18. FOOTER

Minimal

Elegant

Restaurant Logo

Quick Links

Menu

Working Hours

Social Media

Contact

Newsletter

Copyright

Privacy

19. ONLINE ORDERING EXPERIENCE

The ordering process should require as few steps as possible.

Step 1

Browse Menu

Step 2

Customize Item

Step 3

Add to Cart

Step 4

Checkout

Step 5

Payment

Step 6

Confirmation

Step 7

Track Order

Every step should show progress and keep the user informed.

20. SEARCH EXPERIENCE

Global Search

Instant Results

Food

Categories

Offers

Suggestions

Recent Searches

Popular Searches

No page reloads.

21. WHATSAPP INTEGRATION

Provide a floating WhatsApp button on every page.

Customers should be able to:

- Ask questions
 - Share location
 - Request today's specials
 - Place an order
 - Confirm reservations
 - Receive order updates
 - Receive digital invoices
-

22. CUSTOMER DASHBOARD (OPTIONAL LOGIN)

Customers who create an account should have access to:

- Profile
- Saved Addresses
- Order History
- Favorite Dishes
- Saved Payment Methods
- Loyalty Points (future-ready)
- Active Orders
- Previous Invoices
- Reservation History
- Notifications

The dashboard should follow the same premium design language as the public website.

23. SEO & DISCOVERABILITY

Every public page should be optimized for search engines.

Include:

- Semantic HTML
 - Structured Restaurant Schema
 - Open Graph Tags
 - Twitter Cards
 - Local SEO metadata
 - Optimized image alt text
 - Fast loading assets
 - Human-readable URLs
 - XML Sitemap
 - Robots.txt
-

24. PERFORMANCE REQUIREMENTS

- Initial page load under 2 seconds on a good mobile connection.
- Lazy-load non-critical images.
- Code-split routes and heavy components.
- Use responsive image sizes.

- Avoid layout shifts.
 - Ensure smooth scrolling and animations without affecting responsiveness.
-

25. IMAGE DIRECTION

All photography should feel cohesive.

Hero Images

- Cinematic close-up of signature momos.
- Warm directional lighting.
- Visible steam.
- Dark stone or wooden serving surfaces.

Restaurant Interior

- Modern, clean, inviting.
- Warm ambient lighting.
- Customers enjoying meals naturally.

Kitchen

- Professional chefs preparing fresh food.
- Clean workstations.
- Fresh ingredients visible.

Food Photography

- Consistent camera angle.
 - Consistent editing style.
 - Macro details.
 - Minimal props.
 - Premium presentation.
-

26. USER EXPERIENCE PRINCIPLES

Every interaction should answer one question:

"Does this make it easier or more enjoyable for the customer?"

If the answer is no, redesign it.

The public website should never feel like a generic restaurant template. It should tell a story, build trust, showcase exceptional food, and guide visitors naturally toward ordering, reserving a table, or contacting the restaurant.

27. ACCEPTANCE CRITERIA

The customer-facing website will be considered complete when it:

- Delivers a premium, consistent visual identity.
- Provides intuitive navigation across all devices.
- Encourages online ordering and reservations.
- Showcases the menu with high-quality imagery and rich information.
- Loads quickly and scores highly in Lighthouse.
- Supports future integration with the restaurant's POS, inventory, analytics, and WhatsApp workflows without requiring a redesign.

RESTAURANT DIGITAL PLATFORM

MASTER PRODUCT REQUIREMENTS DOCUMENT (PRD)

VOLUME 4

ONLINE ORDERING SYSTEM & CUSTOMER ORDER JOURNEY

1. INTRODUCTION

The online ordering system is the heart of the restaurant's digital platform.

It should provide an experience that is:

- Fast
- Intuitive
- Mobile-first
- Reliable
- Frictionless
- Premium

The goal is to let a first-time customer complete an order within **90 seconds** without confusion.

Every interaction should reduce cognitive load.

The system should feel comparable to the ordering experience of modern food platforms while remaining tailored to this restaurant's brand.

2. ORDERING PHILOSOPHY

The ordering experience should be designed around three principles:

Speed

Customers should reach checkout with the fewest possible clicks.

Clarity

Prices, customizations, and totals should always be visible.

Confidence

The customer should know exactly what they are ordering and what will happen next.

3. ORDER TYPES

Support the following order modes:

Delivery

- Customer enters delivery address.
 - Delivery charges calculated automatically.
 - Estimated delivery time displayed.
 - Live order tracking.
-

Takeaway

- Customer selects pickup time.
 - No delivery charge.
 - Pickup confirmation via WhatsApp/SMS.
-

Dine-In (Future Ready)

- Customer scans QR code.
 - Selects table number automatically.
 - Places order from phone.
 - Kitchen receives order instantly.
-

4. MENU DISCOVERY

The ordering page should help customers discover food effortlessly.

Navigation

Sticky category navigation.

Examples:

- Steam Momos
- Fried Momos
- Kurkure Momos
- Peri Peri Momos
- Tandoori Momos
- Jhol Momos
- Chinese
- Rice
- Noodles
- Starters
- Beverages
- Desserts
- Combos

Category switching should animate smoothly without reloading the page.

5. SEARCH EXPERIENCE

Provide a global search bar with:

- Instant search
- Auto-complete
- Typo tolerance
- Recent searches
- Popular searches
- Suggested dishes

Customers should be able to search:

- Food name
 - Category
 - Ingredients
 - Popular dishes
-

6. FILTERS

Allow filtering by:

- Veg
- Paneer
- Chicken
- Best Seller
- New Arrival
- Chef Special
- Spicy
- Mild
- Medium
- High Protein
- Budget Friendly
- Premium
- Combo Meals

Sorting options:

- Popular
 - Price: Low to High
 - Price: High to Low
 - Customer Rating
 - Preparation Time
-

7. MENU ITEM CARD

Each food item should include:

Visual

- High-quality food image
- Hover zoom
- Subtle shadow animation

Information

- Name
- Short description
- Price
- Veg/Non-Veg indicator
- Spice level

- Preparation time
- Popular badge
- Chef's Recommendation
- Rating

Actions

- Add to Cart
 - Quick View
 - Favorite
 - Share
-

8. PRODUCT DETAIL MODAL

Clicking a menu item opens a premium product modal.

Contents:

- Multiple images
 - Full description
 - Ingredients
 - Allergen information
 - Spice level
 - Nutritional highlights (optional)
 - Preparation time
 - Customer reviews
 - Related dishes
-

9. FOOD CUSTOMIZATION

Customers should be able to personalize items.

Examples:

Portion

- Half
- Full

Filling

- Veg
- Paneer
- Chicken

Spice Level

- Mild
- Medium
- Hot
- Extra Hot

Extras

- Extra Cheese
- Extra Mayo
- Extra Sauce
- Extra Chilli Oil
- Extra Dip

Notes

Free-text cooking instructions:

- Less spicy
 - No onion
 - Extra crispy
 - Pack separately
-

10. SHOPPING CART

The cart should remain accessible from every page.

Cart Contents

- Product image
- Product name
- Quantity
- Variant
- Add-ons

- Individual price
- Total item price

Cart Summary

- Subtotal
- Taxes
- Delivery charges
- Discounts
- Packaging charges (if applicable)
- Grand total

Actions

- Increase quantity
 - Decrease quantity
 - Remove item
 - Save for later
 - Clear cart
-

11. PROMO CODES

Support:

- Percentage discounts
- Flat discounts
- First-order offers
- Free delivery
- Festival offers
- Buy One Get One
- Combo offers

Validation should happen instantly.

12. CHECKOUT EXPERIENCE

Checkout should be divided into logical steps.

Step 1

Customer Information

- Name
 - Phone
 - Email (optional)
-

Step 2

Delivery Details

For delivery:

- Saved addresses
- Add new address
- Google Maps integration
- Landmark
- Delivery notes

For takeaway:

- Pickup time
-

Step 3

Order Review

Display:

- Ordered items
- Variants
- Add-ons
- Total
- Taxes
- Delivery fee
- Estimated time

Allow edits without leaving checkout.

Step 4

Payment

Supported methods:

- UPI
 - Credit Card
 - Debit Card
 - Net Banking
 - Cash on Delivery (optional)
 - Wallets
-

Step 5

Confirmation

Display:

- Order ID
 - Estimated delivery/pickup time
 - Payment status
 - Restaurant contact
 - WhatsApp support button
-

13. PAYMENT EXPERIENCE

The payment interface should feel secure and professional.

Requirements:

- Loading indicators
 - Retry failed payments
 - Payment timeout handling
 - Duplicate payment protection
 - Secure confirmation screen
-

14. ORDER TRACKING

After placing an order, customers should see a live timeline.

Stages:

1. Order Received
2. Confirmed
3. Preparing
4. Cooking
5. Ready
6. Out for Delivery / Ready for Pickup
7. Delivered / Completed

Each stage should include:

- Icon
 - Timestamp
 - Status message
-

15. ORDER HISTORY

Customers should have access to previous orders.

Features:

- Search past orders
 - Repeat order
 - Download invoice
 - Leave review
 - Report issue
-

16. FAVORITES

Allow customers to:

- Save favorite dishes
- Save favorite combinations

- Quickly reorder favorites
-

17. NOTIFICATIONS

Customers should receive updates via:

- In-app notifications
- WhatsApp
- SMS (optional)
- Email (optional)

Examples:

- Order confirmed
 - Food being prepared
 - Out for delivery
 - Ready for pickup
 - Delivered
 - Promotional offers
-

18. QR ORDERING (FUTURE READY)

Support QR code ordering for dine-in.

Workflow:

1. Customer scans QR.
2. Table number detected automatically.
3. Menu opens instantly.
4. Customer places order.
5. Kitchen receives order.
6. Customer requests bill digitally.

No app installation required.

19. LOYALTY & REWARDS (OPTIONAL)

Future-ready support for:

- Reward points
 - Birthday offers
 - Referral rewards
 - Membership levels
 - Cashback campaigns
-

20. ERROR HANDLING

The system should gracefully handle:

- Out-of-stock items
- Network failures
- Payment failures
- Restaurant closed
- Delivery unavailable
- Invalid promo codes
- Session expiration

Each error should explain:

- What happened
 - Why it happened (when appropriate)
 - What the customer can do next
-

21. MOBILE EXPERIENCE

Because most customers will order from phones:

- Bottom navigation for ordering flow
- Sticky cart summary
- Large touch targets
- Thumb-friendly interactions
- Fast scrolling

- Persistent "View Cart" button
 - Optimized keyboard behavior for forms
-

22. PERFORMANCE REQUIREMENTS

- Instant search (<200 ms response)
 - Cart updates without page reload
 - Optimistic UI for quantity changes
 - Image lazy loading
 - Smooth transitions between steps
 - Fast checkout completion
-

23. SECURITY

Protect customer data by:

- Secure authentication (if logged in)
 - HTTPS
 - Payment gateway tokenization
 - Input validation
 - Rate limiting
 - CSRF protection
 - Order tampering prevention
-

24. ANALYTICS EVENTS

Track user interactions to improve conversions:

- Menu viewed
- Search performed
- Filter applied
- Product viewed
- Item added to cart
- Cart abandoned

- Checkout started
 - Payment completed
 - Order completed
 - Repeat order initiated
-

25. FUTURE ENHANCEMENTS

Design the ordering module to support:

- Scheduled orders
 - Group ordering
 - Subscription meal plans
 - Gift vouchers
 - AI dish recommendations
 - Dynamic pricing
 - Voice ordering
 - Multi-language support
-

26. ACCEPTANCE CRITERIA

The ordering system is considered complete when:

- A new customer can place an order without assistance.
- The interface is intuitive on mobile and desktop.
- Customizations are easy to understand.
- Checkout is fast and reliable.
- Order status is transparent.
- Customers can easily reorder and contact the restaurant.
- The system is ready to integrate with the restaurant's POS, Kitchen Display System (KDS), inventory, billing, and analytics modules.

RESTAURANT DIGITAL PLATFORM

MASTER PRODUCT REQUIREMENTS DOCUMENT (PRD)

VOLUME 5

RESTAURANT ADMIN DASHBOARD & MANAGEMENT SYSTEM

1. INTRODUCTION

The Admin Dashboard is the central command center of the restaurant.

It should allow owners, managers, cashiers, and authorized staff to manage every aspect of the restaurant from one place.

Unlike the customer-facing website, the dashboard prioritizes:

- Speed
- Efficiency
- Readability
- Productivity
- Real-time information
- Minimal clicks
- Operational clarity

The dashboard should feel like a modern SaaS platform inspired by:

- Stripe Dashboard
- Linear
- Vercel
- Notion
- Shopify Admin

- Toast POS
- Square Dashboard

Avoid clutter.

Avoid overwhelming charts.

Avoid unnecessary animations.

Every interaction should help staff complete tasks faster.

2. DESIGN PRINCIPLES

The dashboard must follow these principles:

Speed First

Staff should complete common tasks in seconds.

Information Hierarchy

Important information should always be visible first.

Minimal Cognitive Load

Never overload the screen.

Consistency

Every page follows identical layout rules.

Real-Time Updates

Dashboard updates without refreshing.

Mobile Friendly

Managers should access reports from phones.

3. USER ROLES

Support Role-Based Access Control (RBAC).

Owner

Full access.

Can manage everything.

Restaurant Manager

Orders

Inventory

Employees

Reports

Customers

Cashier

Billing

Orders

Payments

Receipts

Kitchen Staff

Kitchen Display

Order Status

Preparation

Delivery Staff

Assigned Orders

Status

Directions

Marketing Staff

Coupons

Offers

Customers

Reviews

Each role should only see the modules they are allowed to access.

4. LOGIN EXPERIENCE

Secure login screen.

Modern branding.

Restaurant logo.

Dark theme.

Fields:

Email

Password

Remember Me

Forgot Password

Sign In

Optional:

OTP Login

Two-Factor Authentication

5. DASHBOARD LAYOUT

Top Navbar

↓

Left Sidebar

↓

Main Content

↓

Right Notification Panel (optional)

Sidebar

Dashboard

Orders

POS

Kitchen

Reservations

Menu

Inventory

Customers

Employees

Analytics

Offers

Settings

Logout

Icons + Labels

Collapsible

Top Navbar

Restaurant Name

Global Search

Notifications

Quick Add

Theme Toggle (future)

Profile

6. DASHBOARD HOME

Purpose:

Give the owner a complete overview in one screen.

KPI Cards

Today's Revenue

Today's Orders

Pending Orders

Completed Orders

Average Order Value

Reservations

Repeat Customers

Low Stock Alerts

Each KPI card includes:

Icon

Value

Percentage Change

Mini Trend Graph

Sales Chart

Daily Revenue

Weekly Revenue

Monthly Revenue

Yearly Revenue

Interactive

Hover Details

Export

Order Status Widget

Pending

Preparing

Ready

Completed

Cancelled

Live updating.

Popular Dishes

Top 10 items today.

Image

Name

Orders

Revenue

Recent Activity

Order placed

Order cancelled

Reservation booked

Payment received

Inventory updated

Staff login

7. ORDER MANAGEMENT

Purpose:

Handle every order efficiently.

Order List

Table Columns:

Order ID

Customer

Type

Status

Payment

Amount

Time

Actions

Filters

Today

Yesterday

Pending

Preparing

Completed

Cancelled

Delivery

Takeaway

Dine-In

Order Detail Drawer

Customer

Items

Variants

Add-ons

Notes

Payment

Timeline

Invoice

Print

Assign Kitchen

Update Status

Status Flow

Received
Confirmed
Preparing
Ready
Delivered
Completed
Cancelled

8. MENU MANAGEMENT

Admin should manage the menu without coding.

Features:

Add Category
Edit Category
Delete Category
Reorder Categories

Food Item

Name

Description

Images

Price

Category

Variants

Availability

Preparation Time

Ingredients

Spice Level

Tags

Chef Special

Best Seller

Discount

Bulk Actions

Import CSV

Export CSV

Bulk Price Update

Bulk Availability

9. RESERVATION MANAGEMENT

View today's bookings.

Approve.

Reject.

Reschedule.

Assign tables.

Mark arrived.

Mark completed.

Customer history.

Special requests.

10. CUSTOMER MANAGEMENT

Customer database.

Profile

Name

Phone

Email

Address

Orders

Visits

Total Spending

Favorite Items

Reviews

Loyalty

Notes

Actions

Call

WhatsApp

Email

Export

11. REVIEW MANAGEMENT

View customer reviews.

Approve testimonials.

Reply.

Flag inappropriate reviews.

Feature selected reviews on website.

12. PROMOTIONS

Create:

Coupons

Discounts

Festival Offers

Combo Deals

Free Delivery

BOGO Offers

Loyalty Campaigns

Expiry Dates

Usage Limits

Target Customers

13. NOTIFICATION CENTER

Live notifications.

Examples:

New order

Reservation

Payment received

Low stock

Printer offline

Kitchen delay

Customer complaint

14. REPORTS

Generate reports.

Daily

Weekly

Monthly

Custom Date Range

Reports:

Sales

Orders

Revenue

Taxes

Discounts

Inventory

Customers

Reservations

Best Sellers

Staff Performance

Export

PDF

Excel

CSV

15. ANALYTICS

Interactive dashboard.

Charts:

Revenue Trend

Orders Per Hour

Peak Time

Customer Growth

Returning Customers

Food Category Sales

Payment Methods

Cancellation Rate

Delivery vs Takeaway

Heatmaps

AI Insights (Future Ready)

"Steam Momos are selling 28% more this week."

"Fridays generate the highest revenue."

"Inventory for Paneer will run out tomorrow."

16. EMPLOYEE MANAGEMENT

Staff Profiles

Role

Phone

Email

Joining Date

Status

Permissions

Attendance (future)

Shift (future)

17. SETTINGS

Restaurant Information

Logo

Cover Images

Address

GST Number

Phone

Email

Working Hours

Social Media

Delivery Charges

Taxes

Payment Methods

Business Hours

Printer Settings

18. QUICK ACTIONS

Floating Quick Action Button.

Examples:

New Order

Add Menu Item

Print Report

Add Customer

Add Coupon

[View Reservations](#)

19. GLOBAL SEARCH

Search everything.

[Orders](#)

[Customers](#)

[Menu](#)

[Reservations](#)

[Employees](#)

[Coupons](#)

[Settings](#)

Instant results.

Keyboard shortcut:

Ctrl + K

20. AUDIT LOGS

Track all administrative activity.

[User](#)

[Action](#)

[Time](#)

[IP Address](#)

[Device](#)

21. DASHBOARD COMPONENTS

KPI Cards

Analytics Charts

Data Tables

Timeline

Activity Feed

Calendar

Mini Charts

Notifications

Progress Bars

Statistics Cards

Empty States

Skeleton Loaders

Dialogs

Drawers

Confirmation Modals

22. RESPONSIVE BEHAVIOR

Desktop

Full Sidebar

Tablet

Collapsed Sidebar

Mobile

Bottom Navigation + Drawer

Critical actions remain accessible.

23. SECURITY

RBAC

JWT Authentication

Session Timeout

Password Policies

CSRF Protection

Rate Limiting

Encrypted Data

Audit Trails

Secure APIs

24. PERFORMANCE

Dashboard loads in under 2 seconds.

Real-time updates.

Pagination for large datasets.

Virtualized tables.

Optimistic UI.

Lazy-loaded modules.

25. FUTURE MODULES

The architecture should support future additions without redesign:

- Multi-branch management
 - Franchise dashboard
 - AI business assistant
 - Advanced inventory forecasting
 - Staff payroll
 - Attendance
 - Vendor management
 - Accounting integration
 - Mobile admin app
 - Customer loyalty engine
-

26. ACCEPTANCE CRITERIA

The Admin Dashboard is considered complete when:

- Staff can manage orders, reservations, menu items, customers, promotions, and settings from one interface.
- Owners have access to meaningful real-time analytics.
- The interface remains clean, fast, and consistent with the public website.
- Role-based permissions protect sensitive operations.
- The dashboard is scalable enough to accommodate future restaurant growth without major architectural changes.

RESTAURANT DIGITAL PLATFORM

MASTER PRODUCT REQUIREMENTS DOCUMENT (PRD)

VOLUME 6

RESTAURANT POS, BILLING, KITCHEN DISPLAY SYSTEM (KDS), GST & THERMAL PRINTING

1. INTRODUCTION

The Point of Sale (POS) system is the operational heart of the restaurant.

Every customer order—whether placed online, in-store, via QR code, or by phone—should ultimately flow through the POS.

The system must prioritize:

- Speed
- Accuracy
- Reliability
- Real-time synchronization
- Ease of use
- Minimal training
- Offline resilience (where possible)

The interface should feel like a premium modern POS inspired by:

- Square POS
- Toast POS

- Lightspeed Restaurant
 - Shopify POS
 - Stripe Dashboard
 - Apple Human Interface Guidelines
-

2. POS DESIGN PHILOSOPHY

The POS is a productivity tool, not a marketing website.

The interface should:

- Reduce the number of clicks.
 - Make frequent actions immediately accessible.
 - Minimize typing.
 - Support keyboard shortcuts.
 - Support touch-first interaction.
 - Use large buttons and clear typography.
 - Present only the information required at that moment.
-

3. POS DASHBOARD LAYOUT

Top Toolbar

Categories | Menu | Order Panel | Billing Panel

Bottom Actions

Payment
Hold Order
Print
Discount
Customer
Settings

The layout should remain responsive for tablets and touch-screen terminals.

4. ORDER SOURCES

The POS should clearly identify the origin of each order.

Examples:

- Walk-In
- Dine-In
- QR Order
- Website Order
- WhatsApp Order
- Phone Order
- Takeaway
- Delivery

Each order source should have a distinct badge or color for quick recognition.

5. QUICK ORDER INTERFACE

The cashier should be able to create a new order within seconds.

Workflow:

1. Select order type.
2. Search or browse menu.
3. Add items.
4. Customize items.
5. Review bill.
6. Accept payment.
7. Print receipt.
8. Send kitchen ticket.

Target time:

- Simple order: under 20 seconds.
- Large order: under 60 seconds.

6. MENU BROWSER

The POS menu should include:

Categories

- Steam Momos
- Fried Momos
- Kurkure Momos
- Tandoori Momos
- Chinese
- Rice
- Noodles
- Beverages
- Desserts
- Combos

Each category should show:

- Thumbnail
- Item count
- Availability

Menu search should support:

- Item name
 - SKU (optional)
 - Barcode (future)
 - Recent items
-

7. ORDER PANEL

The active order panel should display:

- Item image
- Item name
- Variant
- Quantity

- Add-ons
- Special instructions
- Individual price
- Line total

Actions:

- Increase quantity
 - Decrease quantity
 - Edit customization
 - Remove item
 - Duplicate item
-

8. ITEM CUSTOMIZATION

Support:

Variants

- Veg
- Paneer
- Chicken

Portion

- Half
- Full

Spice Level

- Mild
- Medium
- Hot
- Extra Hot

Add-ons

- Cheese
- Extra Mayo
- Extra Dip
- Extra Sauce

- Extra Chilli Oil

Kitchen Notes

- Less spicy
- No onion
- Extra crispy
- Separate packing

All changes should update pricing instantly.

9. CUSTOMER INFORMATION

Optional customer association.

Capture:

- Name
- Phone
- Email
- Membership ID (future)

Benefits:

- Repeat orders
 - Loyalty
 - Invoices
 - WhatsApp receipts
-

10. TABLE MANAGEMENT (DINE-IN)

Interactive floor layout.

Display:

- Table number
- Capacity
- Occupancy
- Status

Statuses:

- Available
- Occupied
- Reserved
- Cleaning
- Billing

Actions:

- Assign table
 - Move table
 - Merge tables
 - Split table
 - Close table
-

11. BILLING ENGINE

The billing module must support:

Charges

- Item total
- Taxes
- Delivery fee
- Packaging fee
- Service charge
- Tips (optional)

Discounts

- Percentage
- Fixed amount
- Coupon
- Manual

Total Calculation

The total should update in real time.

12. GST MANAGEMENT

Support Indian GST compliance.

Fields:

- CGST
- SGST
- IGST

Automatically determine the applicable tax based on configuration.

Invoices should clearly display:

- Taxable amount
- GST breakup
- Total GST
- Grand total

Restaurant GST details should be configurable in Settings.

13. PAYMENT PROCESSING

Supported payment methods:

- Cash
- UPI
- Credit Card
- Debit Card
- Net Banking
- Wallets
- Split Payment

Split payment example:

- ₹500 Cash
- ₹250 UPI
- Remaining Card

Payment confirmation should be instant.

14. RECEIPTS & INVOICES

Generate professional invoices containing:

- Restaurant logo
- Address
- GST Number
- Invoice Number
- Date & Time
- Cashier Name
- Customer Details (if available)
- Ordered Items
- Taxes
- Discounts
- Payment Method
- QR Code (optional)
- Thank-you message

Support:

- Print
 - PDF download
 - Email
 - WhatsApp sharing
-

15. THERMAL PRINTER SUPPORT

Support standard ESC/POS-compatible printers.

Printer types:

- 58mm
- 80mm

Connection methods:

- USB
- LAN
- Wi-Fi
- Bluetooth

Printer destinations:

Customer Receipt

Kitchen Order Ticket (KOT)

Reprint Receipt

Support automatic printer detection and retry on failure.

16. KITCHEN ORDER TICKET (KOT)

When an order is confirmed:

- Generate KOT automatically.
- Send to kitchen printer or Kitchen Display System.

Include:

- Order number
- Table number (if applicable)
- Order type
- Items
- Quantities
- Customizations
- Kitchen notes
- Timestamp

Do not include prices on KOT.

17. KITCHEN DISPLAY SYSTEM (KDS)

The KDS replaces paper tickets where supported.

Layout:

Large cards for each active order.

Each card shows:

- Order number
- Order source
- Time elapsed
- Items
- Customizations
- Priority
- Status

Statuses:

- New
- Accepted
- Preparing
- Ready
- Completed

Cards should change color based on waiting time.

18. ORDER PRIORITY

Automatically prioritize orders based on:

- Waiting time
- Scheduled pickup
- VIP customer (future)
- Manual override

Visual indicators:

- Green: On time
 - Yellow: Approaching delay
 - Red: Delayed
-

19. ORDER LIFECYCLE

Every order should follow a defined workflow:

1. Created
2. Confirmed
3. Sent to Kitchen
4. Preparing
5. Ready
6. Served / Packed
7. Completed
8. Archived

Each transition should be logged.

20. REFUNDS & VOIDS

Support:

- Full refund
- Partial refund
- Void item
- Void order

Require:

- Manager authorization
 - Reason for action
 - Audit log entry
-

21. CASH MANAGEMENT

Support daily cash operations.

Features:

- Opening cash balance
- Cash in
- Cash out
- Shift closing
- Expected cash
- Actual cash

- Variance report

Generate an end-of-day summary automatically.

22. OFFLINE MODE (FUTURE READY)

If internet connectivity is lost:

- Continue accepting orders locally.
- Queue transactions.
- Sync automatically when connectivity returns.

Prevent duplicate submissions during synchronization.

23. AUDIT TRAIL

Log every significant POS event:

- Order creation
- Item edits
- Discount applied
- Payment completed
- Refund issued
- Receipt reprinted
- Cash drawer opened
- Shift closed

Each log should record:

- User
 - Timestamp
 - Action
 - Device
-

24. SECURITY

Role-based permissions.

Examples:

Cashier:

- Create orders
- Accept payments

Manager:

- Discounts
- Refunds
- Voids

Owner:

- All permissions

Sensitive actions should require confirmation and be logged.

25. PERFORMANCE TARGETS

- POS interface loads in under 2 seconds.
 - Menu search returns results instantly.
 - Billing updates in real time.
 - Kitchen receives orders within one second.
 - Printing starts within two seconds of confirmation.
-

26. FUTURE ENHANCEMENTS

Design the architecture to support:

- Self-service kiosks
- Customer-facing display
- Kitchen stations
- Barcode scanning
- Weight-based pricing
- Table-side ordering

- Voice-assisted cashier
 - AI-powered upselling
 - Multi-branch synchronization
 - Cloud reporting
-

27. USER EXPERIENCE REQUIREMENTS

The POS should be usable after minimal training.

Staff should be able to:

- Find menu items quickly.
- Process payments confidently.
- Correct mistakes safely.
- Track kitchen progress.
- Print receipts without confusion.

The interface should remain consistent with the design system established in earlier volumes while emphasizing operational efficiency over visual effects.

28. ACCEPTANCE CRITERIA

The POS and KDS modules are considered complete when:

- Orders can be created, customized, billed, and completed efficiently.
- Kitchen staff receive accurate tickets immediately.
- GST-compliant invoices are generated correctly.
- Thermal printing is reliable.
- Managers can oversee order progress in real time.
- The workflow supports walk-in, takeaway, delivery, QR, and online orders from a single unified system.
- All operational actions are secure, auditable, and role-based.

RESTAURANT DIGITAL PLATFORM

MASTER PRODUCT REQUIREMENTS DOCUMENT (PRD)

VOLUME 7

INVENTORY MANAGEMENT, SUPPLIER MANAGEMENT, CRM, LOYALTY & BUSINESS ANALYTICS

1. INTRODUCTION

The purpose of this module is to provide complete visibility into the restaurant's operational health.

The restaurant owner should always know:

- What is selling
- What is not selling
- Which ingredients are running low
- Which suppliers need to be contacted
- Which customers return frequently
- Which marketing campaigns perform best
- Which dishes generate the highest profit
- Where money is being lost

This module should transform raw operational data into actionable insights.

2. DESIGN PHILOSOPHY

The management modules should focus on:

- Clarity over complexity
- Automation over manual work
- Insights over raw data
- Prevention over reaction

The interface should remain consistent with the design system while presenting large amounts of information in a clean and digestible way.

3. INVENTORY DASHBOARD

The inventory dashboard provides a live overview of restaurant stock.

Display:

- Total Inventory Value
- Total Ingredients
- Low Stock Items
- Out-of-Stock Items
- Expiring Items
- Today's Consumption
- Today's Purchases

Visual widgets should use KPI cards and trend indicators.

4. INVENTORY CATEGORIES

Organize inventory into logical groups.

Examples:

Vegetables

Meat

Paneer

Cheese

Flour

Sauces

Spices

Packaging

Beverages

Frozen Items

Cleaning Supplies

Kitchen Consumables

5. INGREDIENT MANAGEMENT

Every inventory item should contain:

Item Name

Category

SKU

Unit

Purchase Price

Selling Impact

Current Stock

Minimum Stock

Maximum Stock

Supplier

Expiry Date (if applicable)

Storage Location

Notes

6. STOCK MOVEMENTS

Every inventory transaction should be recorded.

Movement Types:

Purchase

Consumption

Adjustment

Waste

Damage

Return

Transfer (future)

Each record stores:

Date

User

Quantity

Reason

Reference

Supplier

7. LOW STOCK ALERTS

Automatic notifications when stock falls below threshold.

Alert channels:

Dashboard

Email (optional)

WhatsApp (future)

Priority Levels:

Normal

Warning

Critical

8. EXPIRY MANAGEMENT

Track ingredients with expiry dates.

Dashboard widgets:

Expiring Today

Expiring This Week

Expired Items

Notify managers before expiration.

9. RECIPE MAPPING

Each menu item should map to ingredients.

Example:

Steam Chicken Momos

↓

Flour

Chicken

Onion

Garlic

Ginger

Oil

Spices

Automatically deduct ingredient quantities after order completion.

10. FOOD COSTING

Calculate real-time cost of every menu item.

Display:

Ingredient Cost

Packaging Cost

Preparation Cost (optional)

Selling Price

Profit

Profit Margin

Managers should instantly know which dishes are most profitable.

11. PURCHASE MANAGEMENT

Create purchase orders.

Fields:

Supplier

Items

Quantity

Price

Expected Delivery

Status

Invoice Number

Notes

Statuses:

Draft

Ordered

Received

Cancelled

Completed

12. SUPPLIER MANAGEMENT

Maintain supplier database.

Each supplier profile includes:

Business Name

Contact Person

Phone

Email

GST Number

Address

Supplied Categories

Payment Terms

Rating

Purchase History

Outstanding Balance

Notes

13. WASTE MANAGEMENT

Track food waste.

Reasons:

Expired

Burnt

Overproduction

Customer Return

Damaged

Incorrect Preparation

Generate reports to identify recurring issues.

14. CUSTOMER RELATIONSHIP MANAGEMENT (CRM)

Maintain a centralized customer database.

Each customer profile should display:

Profile

Contact Information

Order History

Total Spend

Visit Frequency

Average Order Value

Favorite Items

Preferred Order Type

Last Visit

Last Order

Reviews

Notes

15. CUSTOMER SEGMENTS

Automatically group customers into:

New Customers

Returning Customers

VIP Customers

High Spenders

Inactive Customers

Birthday This Month

Frequent Visitors

Online Customers

Walk-In Customers

16. LOYALTY PROGRAM

Future-ready loyalty engine.

Support:

Points

Membership Levels

Rewards

Referral Bonuses

Birthday Rewards

Anniversary Offers

Exclusive Discounts

Track:

Current Points

Lifetime Points

Redeemed Rewards

Available Rewards

17. COUPONS & PROMOTIONS

Create marketing campaigns.

Campaign Types:

Percentage Discount

Flat Discount

Free Item

Free Delivery

Buy One Get One

Combo Deals

Happy Hour

Festival Offers

Loyalty Offers

Define:

Validity

Usage Limit

Customer Segment

Minimum Order Value

Maximum Discount

18. WHATSAPP MARKETING (FUTURE READY)

Send:

Order Updates

Offers

Festival Greetings

Birthday Wishes

Coupons

Menu Updates

New Dish Announcements

Abandoned Cart Reminders

Allow segmentation to avoid spamming customers.

19. EMAIL CAMPAIGNS (OPTIONAL)

Support:

Newsletter

Promotions

Feedback Requests

Reservation Reminders

Event Invitations

20. BUSINESS ANALYTICS DASHBOARD

Present restaurant performance through interactive charts.

Metrics:

Revenue

Orders

Customers

Inventory Value

Profit Margin

Average Order Value

Peak Hours

Order Sources

Repeat Customers

Refunds

Discount Usage

21. SALES ANALYTICS

Track:

Hourly Sales

Daily Sales

Weekly Sales

Monthly Sales

Yearly Sales

Compare periods.

Highlight growth trends.

22. MENU ANALYTICS

Identify:

Top Selling Items

Least Selling Items

Highest Revenue Items

Highest Profit Items

Frequently Purchased Together

Average Preparation Time

Customer Ratings

Use these insights to improve the menu.

23. CUSTOMER ANALYTICS

Display:

New vs Returning Customers

Customer Lifetime Value

Retention Rate

Average Spend

Order Frequency

Top Customers

Acquisition Sources

24. INVENTORY ANALYTICS

Monitor:

Consumption Trends

Waste Percentage

Stock Turnover

Purchase Frequency

Supplier Performance

Ingredient Cost Trends

Forecast Low Stock

25. RESERVATION ANALYTICS

Track:

Reservations by Day

Peak Hours

Cancellation Rate

No-Show Rate

Average Party Size

26. MARKETING ANALYTICS

Evaluate:

Coupon Usage

Campaign Performance

Referral Conversions

WhatsApp Click Rate

Repeat Order Rate

27. AI INSIGHTS (FUTURE READY)

The dashboard should generate intelligent recommendations such as:

"Chicken Momos generated 18% higher revenue this week."

"Paneer inventory will run out within two days."

"Fridays between 7–9 PM have the highest order volume."

"Customers who order Steam Momos frequently add Cold Drinks."

"Offer a combo discount on slow-moving menu items."

28. EXPORTS & REPORTS

Generate reports in:

PDF

Excel

CSV

Printable format

Supported reports:

Sales

Inventory

Customers

Taxes

Profit

Orders

Waste

Suppliers

Reservations

Campaign Performance

29. DASHBOARD WIDGETS

Use reusable widgets for:

KPI Cards

Trend Graphs

Pie Charts

Bar Charts

Heatmaps

Leaderboards

Activity Timeline

Calendar

Alerts

Recent Orders

Top Products

Low Stock List

Customer Growth

Revenue Breakdown

30. PERMISSIONS

Owners:

Full access.

Managers:

Inventory

Customers

Reports

Marketing

Cashiers:

Limited customer lookup.

Kitchen:

No CRM access.

Marketing Staff:

Campaigns

Coupons

Customer Segments

Reports

Every permission should be configurable.

31. AUTOMATION

Automate repetitive tasks where possible.

Examples:

Low stock alerts

Expiry reminders

Scheduled reports

Birthday coupons

Daily sales summaries

Inventory deductions after completed orders

Campaign scheduling

32. PERFORMANCE REQUIREMENTS

Analytics should load quickly using optimized queries and caching.

Large datasets should support:

Pagination

Filtering

Sorting

Export

Lazy loading

Interactive drill-downs

33. FUTURE ROADMAP

The architecture should support future additions without restructuring.

Possible enhancements:

- AI demand forecasting
- Supplier purchase recommendations
- Automated purchase orders
- Predictive inventory management

- Advanced customer segmentation
 - Personalized marketing campaigns
 - Multi-branch inventory synchronization
 - Financial accounting integration
 - Payroll and HR modules
-

34. ACCEPTANCE CRITERIA

This module is considered complete when:

- Inventory updates automatically based on completed orders.
- Managers receive proactive alerts for low stock and expiring ingredients.
- Customer data is centralized and supports repeat business.
- Marketing campaigns can be created and evaluated.
- Analytics provide actionable insights instead of just raw numbers.
- Reports are exportable and suitable for business decision-making.
- All modules integrate seamlessly with the POS, billing, online ordering, and admin dashboard.

RESTAURANT DIGITAL PLATFORM

MASTER PRODUCT REQUIREMENTS DOCUMENT (PRD)

VOLUME 8

AI, AUTOMATION, WHATSAPP, NOTIFICATIONS & SMART WORKFLOWS

1. INTRODUCTION

This module transforms the Restaurant Digital Platform from a traditional management system into an intelligent operational assistant.

The objective is to automate repetitive tasks, reduce manual errors, improve communication, and provide actionable insights.

AI should never feel intrusive.

It should quietly assist restaurant staff and enhance the customer experience.

Every AI recommendation must remain optional unless explicitly automated by the restaurant owner.

2. DESIGN PHILOSOPHY

Automation should follow these principles:

- Assist, don't interrupt.
- Automate repetitive work.
- Keep humans in control.
- Explain AI recommendations clearly.
- Prioritize reliability over novelty.

The interface should communicate confidence and transparency.

3. AI ASSISTANT

The platform should include a built-in Restaurant AI Assistant.

Capabilities:

- Answer operational questions.
- Search orders.
- Search customers.
- Search inventory.
- Generate reports.

- Explain analytics.
- Suggest improvements.
- Assist with menu management.
- Help managers navigate the system.

Example queries:

"Show today's revenue."

"Which menu item sold the most today?"

"Which ingredients are running low?"

"How many reservations do we have tonight?"

"Generate this week's sales report."

4. AI BUSINESS INSIGHTS

Continuously analyze restaurant data and generate useful observations.

Examples:

- "Steam Momos generated 22% more revenue this week."
- "Chicken Momos are frequently ordered with Soft Drinks."
- "Sales drop after 9 PM on weekdays."
- "Customers ordering combo meals spend 18% more."
- "Friday evenings generate the highest average order value."

Present insights in a dedicated dashboard widget.

5. AI MENU RECOMMENDATIONS

Suggest improvements based on sales and customer behavior.

Examples:

- Highlight best sellers.
- Recommend slow-moving dishes for promotional offers.

- Suggest combo meals.
 - Recommend seasonal menu changes.
 - Identify high-margin items that deserve better placement.
-

6. AI INVENTORY ASSISTANT

Monitor stock usage patterns.

Provide alerts such as:

- Ingredient usage is increasing faster than expected.
- Predicted stock shortage within two days.
- Unusual inventory consumption detected.
- Expiring ingredients should be prioritized.

Allow managers to acknowledge or dismiss suggestions.

7. AI SALES FORECASTING

Forecast future demand using historical trends.

Predictions include:

- Expected daily revenue.
- Peak order times.
- Weekend demand.
- Ingredient requirements.
- Staffing recommendations.

Forecasts should be presented as confidence ranges rather than absolute values.

8. AI CUSTOMER INSIGHTS

Analyze customer behavior.

Identify:

- Frequent customers.
- Customers at risk of not returning.
- Most valuable customers.
- Preferred ordering times.
- Favorite dishes.
- Average spend.

Generate suggestions such as:

"Offer a loyalty reward to this customer."

"Send a birthday coupon."

9. AI REVIEW ANALYSIS

Collect customer reviews from supported sources.

Categorize feedback into:

- Food Quality
- Service
- Delivery
- Packaging
- Cleanliness
- Value
- Staff

Detect recurring themes.

Summarize customer sentiment instead of displaying hundreds of individual reviews.

10. AI REPORT GENERATOR

Managers should be able to ask for reports in plain language.

Examples:

"Generate today's sales report."

"Show last month's profit."

"Compare this weekend with last weekend."

"Export this week's inventory report."

The system generates structured reports with charts and summaries.

11. AI RECIPE KNOWLEDGE (OPTIONAL)

Store standardized recipes.

Provide:

- Ingredient quantities.
- Preparation steps.
- Estimated preparation time.
- Serving instructions.

Future enhancement:

AI compares actual ingredient usage against recipe standards.

12. AI ORDER ASSISTANT

During order creation, AI may suggest:

- Popular add-ons.
- Frequently paired dishes.
- Combo upgrades.
- Better value meals.

Suggestions should appear subtly and never interrupt checkout.

13. AI PHONE ORDERING (FUTURE READY)

Support an AI-powered phone assistant capable of:

- Greeting callers.
- Understanding supported languages.
- Answering menu questions.
- Taking delivery orders.
- Confirming customer information.
- Reading back the order.
- Creating the order in the POS.
- Sending confirmation via WhatsApp.
- Escalating to staff when needed.

Calls should always be recorded with customer consent where required by law.

14. WHATSAPP BUSINESS INTEGRATION

The platform should integrate with WhatsApp Business.

Supported workflows:

Order confirmation.

Order status updates.

Reservation confirmation.

Reservation reminders.

Invoice sharing.

Digital receipts.

Customer support.

Promotional campaigns.

Feedback requests.

Menu sharing.

Location sharing.

15. WHATSAPP ORDER FLOW

Customers should be able to:

1. Open WhatsApp.
2. Start a conversation.
3. Receive the digital menu.
4. Place an order.
5. Confirm address.
6. Choose payment method.
7. Receive live updates.
8. Receive invoice.

Complex requests should transfer seamlessly to restaurant staff.

16. SMART NOTIFICATIONS

Notification types:

- New order.
- Reservation received.
- Payment completed.
- Order delayed.
- Kitchen delay.
- Low inventory.
- Expiring ingredients.
- Printer offline.
- Customer complaint.
- Refund processed.
- Campaign completed.

Every notification should include:

- Title

- Description
 - Timestamp
 - Priority
 - Action button
-

17. AUTOMATED WORKFLOWS

Support configurable workflows.

Examples:

When:

New online order.

Then:

Create POS order.

↓

Print KOT.

↓

Notify kitchen.

↓

Send WhatsApp confirmation.

↓

Update dashboard.

↓

Track order.

When:

Order completed.

Then:

Deduct inventory.



Generate invoice.



Send receipt.



Ask for review.



Update analytics.

When:

Reservation confirmed.

Then:

Send WhatsApp confirmation.



Create calendar entry.



Notify manager.



Send reminder before reservation.

18. SCHEDULED AUTOMATIONS

Allow managers to schedule:

- Daily sales summary.
 - Weekly performance report.
 - Monthly inventory report.
 - Low-stock reminders.
 - Promotional campaigns.
 - Birthday greetings.
 - Festival messages.
-

19. INTERNAL NOTIFICATION CENTER

Provide an internal notification hub.

Categories:

Orders.

Inventory.

Customers.

Marketing.

System.

Finance.

Each notification supports:

Mark as read.

Archive.

Filter.

Search.

Priority.

20. SMART SEARCH

Global search should understand natural language.

Examples:

"Today's orders."

"Paneer inventory."

"Customer Rahul."

"Invoice 1052."

"Revenue last Friday."

Search results should group by module.

21. COMMAND PALETTE

Keyboard shortcut:

Ctrl + K

Allows quick actions:

Create order.

Add menu item.

Open inventory.

Search customer.

Generate report.

Print receipt.

Navigate to any module.

22. AUTOMATION LOGS

Every automation should maintain a log.

Store:

Workflow name.

Trigger.

Execution time.

Result.

Errors.

User.

Retry attempts.

Allow manual reruns.

23. SYSTEM HEALTH MONITOR

Monitor:

Database.

API.

Printer.

WhatsApp integration.

Payment gateway.

Internet connectivity.

Storage.

Backups.

Display health indicators.

24. BACKUPS

Support:

Automatic daily backups.

Manual backups.

Restore points.

Backup verification.

Cloud storage integration (future).

Notify administrators if backups fail.

25. AUDIT & COMPLIANCE

Track all critical system events.

Examples:

User login.

Permission changes.

Price updates.

Inventory adjustments.

Refunds.

Configuration changes.

Automation edits.

Report exports.

Each log includes:

Timestamp.

User.

Action.

Module.

Device.

IP Address.

26. FUTURE AI ROADMAP

Architecture should support future capabilities:

- Voice-controlled POS.
 - AI demand prediction.
 - Dynamic staffing suggestions.
 - Dynamic menu optimization.
 - Personalized customer recommendations.
 - AI-powered marketing campaigns.
 - Multi-language customer support.
 - AI-generated financial summaries.
 - AI fraud detection.
 - AI anomaly detection.
-

27. USER EXPERIENCE REQUIREMENTS

AI should always:

- Explain recommendations.
- Allow users to ignore suggestions.
- Never hide important information.
- Never block manual workflows.
- Be fast and responsive.
- Respect user permissions.

The restaurant should always remain in control.

28. PERFORMANCE REQUIREMENTS

- AI responses under 3 seconds where practical.
 - Notifications delivered in real time.
 - Workflow execution without noticeable delay.
 - WhatsApp messages sent reliably with delivery tracking.
 - Background jobs should not affect UI responsiveness.
-

29. SECURITY & PRIVACY

Protect customer and business data.

Requirements:

- Secure API authentication.
 - Encrypted communication.
 - Role-based access to AI features.
 - Secure storage of conversation history.
 - Configurable data retention.
 - Audit trails for AI-generated actions.
-

30. ACCEPTANCE CRITERIA

This module is considered complete when:

- AI provides useful operational insights without disrupting workflows.
 - WhatsApp integration supports customer communication from order to follow-up.
 - Automation reduces repetitive manual work.
 - Notifications are timely and actionable.
 - Managers can generate reports and insights using natural language.
 - All intelligent features integrate seamlessly with the POS, inventory, CRM, analytics, and customer website.
 - The platform remains intuitive even with advanced automation enabled.
-

FINAL VISION FOR VOLUME 8

By completing this volume, the Restaurant Digital Platform evolves beyond a standard management system. It becomes an intelligent assistant that helps the restaurant operate more efficiently, communicate better with customers, and make data-driven decisions—all while preserving a clean, premium user experience consistent with the rest of the platform.

RESTAURANT DIGITAL PLATFORM

MASTER PRODUCT REQUIREMENTS DOCUMENT (PRD)

VOLUME 9

TECHNICAL ARCHITECTURE, SYSTEM DESIGN, DATABASE, API STANDARDS & DEVELOPMENT GUIDELINES

1. INTRODUCTION

This document defines the technical architecture of the Restaurant Digital Platform.

The goal is to create a modern, scalable, maintainable, and secure application that can grow with the restaurant without requiring significant architectural changes.

The system should follow industry best practices while remaining simple enough for a single restaurant deployment.

The architecture should prioritize:

- Performance
 - Security
 - Maintainability
 - Scalability
 - Modularity
 - Developer Experience
-

2. ARCHITECTURAL PHILOSOPHY

The application should follow these principles:

- Feature-first architecture
 - Modular development
 - Component-driven UI
 - API-first backend
 - Single source of truth
 - Strong typing
 - Separation of concerns
 - Reusable business logic
 - Predictable state management
 - Future-ready architecture
-

3. HIGH-LEVEL SYSTEM ARCHITECTURE

Customer

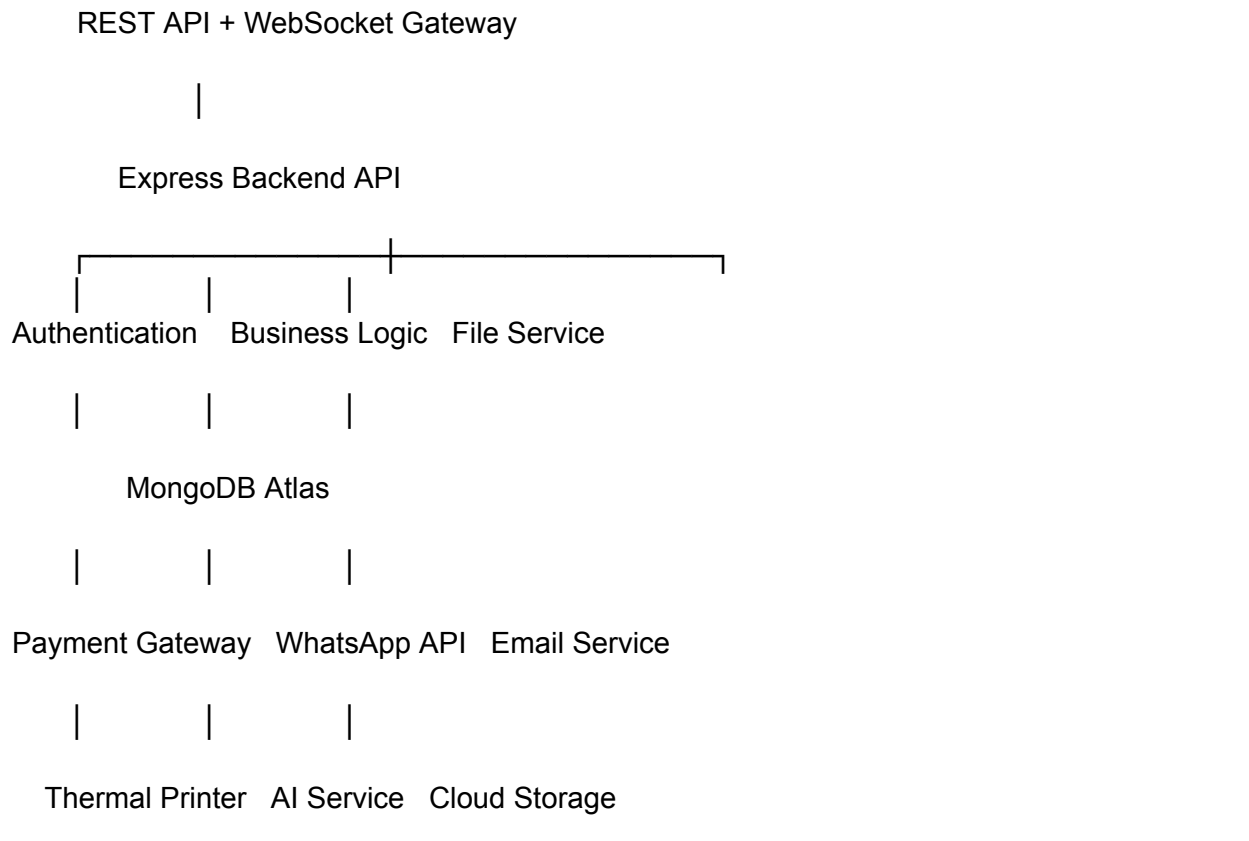
|

Public Website / Ordering Portal

|

React Frontend

|



4. RECOMMENDED TECH STACK

Frontend

React 19

TypeScript

Vite

Tailwind CSS v4

shadcn/ui

Framer Motion

GSAP (limited usage)

Lenis

React Router

TanStack Query

React Hook Form

Zod

Lucide React

Recharts

Embla Carousel

Sonner

React Helmet Async

React Dropzone

React Intersection Observer

Backend

Node.js LTS

Express.js

TypeScript

JWT

Passport (optional)

Socket.IO

Multer

Sharp

Helmet

Express Rate Limit

Compression

Morgan

CORS

Zod Validation

Database

MongoDB Atlas

Mongoose ODM

Redis (optional)

Storage

Cloudinary

Local storage for development

Future:

AWS S3

Payments

Razorpay

UPI

Future:

Stripe

PhonePe

Notifications

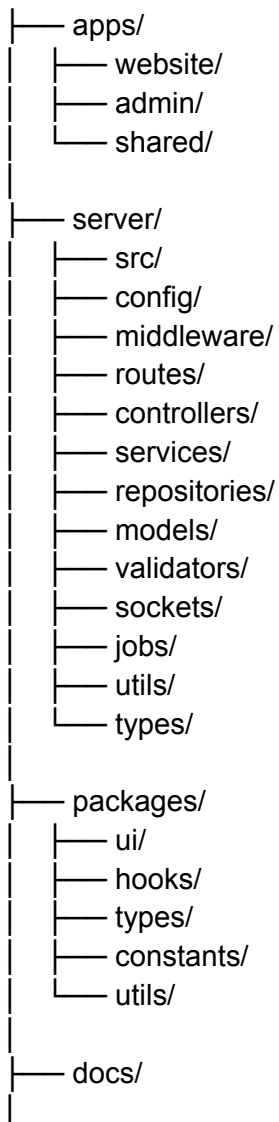
WhatsApp Business API

Resend Email

Firebase Push (future)

5. PROJECT STRUCTURE

restaurant-platform/



```
|— scripts/  
|  
|— tests/
```

6. FRONTEND ARCHITECTURE

Organize features rather than pages.

Example:

features/

auth/

orders/

menu/

checkout/

inventory/

analytics/

crm/

reservations/

settings/

dashboard/

reports/

notifications/

Each feature contains:

- Components
- Hooks
- Services
- Types

- Validation
 - API Layer
 - Constants
-

7. COMPONENT ARCHITECTURE

Every UI element should be reusable.

Hierarchy:

UI Components

↓

Business Components

↓

Feature Components

↓

Page Components

Example:

Button

↓

ProductCard

↓

MenuGrid

↓

MenuPage

8. STATE MANAGEMENT

Separate state into:

UI State

Server State

Form State

Session State

Local State

Recommended:

TanStack Query → Server State

React Context → Authentication

React Hook Form → Forms

Local State → UI interactions

Avoid unnecessary global state.

9. API DESIGN

RESTful architecture.

Examples:

GET

/menu

/menu/:id

/orders

/customers

/inventory

/reports

POST

/orders

/login

/register

/payments

/reservations

PUT

/orders/:id

/customers/:id

/menu/:id

DELETE

/menu/:id

/cart/:id

Response format:

```
{  
  success,  
  
  message,  
  
  data,  
  
  pagination,  
  
  errors,  
  
  timestamp  
}
```

10. DATABASE DESIGN

Collections:

Users

Roles

Customers

Employees

Orders

OrderItems

Reservations

MenuCategories

MenuItems

Ingredients

Inventory

Suppliers

PurchaseOrders

Payments

Invoices

Coupons

Campaigns

Reviews

Notifications

AuditLogs

Settings

Media

Reports

AnalyticsSnapshots

AIRecommendations

AutomationLogs

11. INDEXING STRATEGY

Create indexes for:

Phone

Email

Order Number

Invoice Number

Reservation Date

Customer Name

Menu Category

Inventory SKU

Payment ID

Created Date

Status

Composite indexes where appropriate.

12. AUTHENTICATION

JWT

Refresh Tokens

Remember Me

Secure Cookies (optional)

Role-based sessions

Session expiration

Device tracking (future)

13. AUTHORIZATION

Role-Based Access Control.

Roles:

Owner

Manager

Cashier

Kitchen

Marketing

Delivery

Permissions stored centrally.

Never hard-code permissions.

14. VALIDATION

Validate on both frontend and backend.

Use:

Zod

Examples:

Email

Phone

GST Number

Price

Quantity

Password

Reservation Time

Coupon Rules

15. ERROR HANDLING

Standardized error responses.

Example:

```
{  
  success:false,  
  
  errorCode,  
  
  message,  
  
  details,  
  
  timestamp  
}
```

User-friendly messages.

Developer logs.

16. FILE MANAGEMENT

Support:

Images

PDF Invoices

Reports

Restaurant Logo

Gallery Images

Menu Images

Store metadata separately from file URLs.

Optimize images before upload.

17. REAL-TIME COMMUNICATION

Use WebSockets for:

Live Orders

Kitchen Updates

Dashboard KPIs

Notifications

Order Tracking

Reservation Updates

Printer Status

Avoid polling whenever possible.

18. CACHING STRATEGY

Cache:

Menu

Restaurant Settings

Popular Items

Analytics Snapshots

Reports

Static Assets

Invalidate intelligently after updates.

19. LOGGING

Maintain logs for:

API Requests

Authentication

Errors

Payments

Inventory

POS

AI Actions

Automation

Audit

20. SECURITY

HTTPS only.

Helmet

Rate Limiting

Input Sanitization

CSRF Protection

XSS Prevention

SQL/NoSQL Injection Protection

Secure Headers

Password Hashing

Environment Variables

Secrets Management

File Upload Validation

21. BACKUP STRATEGY

Automatic daily backups.

Manual backups.

Encrypted storage.

Restore testing.

Backup health notifications.

22. DEPLOYMENT

Recommended:

Frontend → Vercel

Backend → Render / Railway

Database → MongoDB Atlas

Media → Cloudinary

Domain → Cloudflare DNS

SSL enabled by default.

23. ENVIRONMENT VARIABLES

Separate:

Development

Staging

Production

Never expose secrets to the client.

Use `.env.example` with documented variables.

24. PERFORMANCE OPTIMIZATION

Lazy loading

Dynamic imports

Image optimization

Tree shaking

Code splitting

Route prefetching

Virtualized tables

Database indexes

Response compression

CDN for assets

25. TESTING STRATEGY

Unit Tests

Integration Tests

API Tests

UI Tests

End-to-End Tests

Accessibility Tests

Performance Tests

Regression Tests

Critical user flows must be automated.

26. CI/CD

Pipeline stages:

Install

Lint

Type Check

Unit Tests

Build

Security Scan

Deploy Preview

Production Deployment

Rollback Support

27. CODING STANDARDS

Use:

ESLint

Prettier

Strict TypeScript

Meaningful naming

Small reusable functions

No duplicated business logic

No inline styles

No hardcoded values

Document complex logic.

28. DOCUMENTATION

Maintain:

README

Setup Guide

API Documentation

Architecture Diagram

Database Schema

Deployment Guide

Contribution Guide

Environment Variables

Feature Documentation

Changelog

29. MONITORING

Track:

Server uptime

API latency

Database health

Memory usage

CPU usage

Error rate

Failed payments

Printer status

WhatsApp delivery

Backup status

Use centralized dashboards for monitoring.

30. SCALABILITY

Design the platform to support future growth:

- Multiple branches
- Mobile applications
- Self-order kiosks
- Delivery fleet integration
- Additional payment providers
- AI enhancements
- Third-party integrations
- Multi-language support
- Advanced reporting

The architecture should make these additions possible without requiring major refactoring.

31. DEVELOPER EXPERIENCE

The codebase should be easy to understand.

Requirements:

- Consistent naming conventions.
 - Clear folder organization.
 - Reusable utilities.
 - Shared types.
 - Comprehensive comments for complex business rules.
 - Minimal onboarding time for new developers.
-

32. QUALITY GATES

No feature should be considered complete unless it:

- Passes TypeScript checks.
- Passes linting.

- Includes validation.
 - Handles errors gracefully.
 - Is responsive.
 - Meets accessibility standards.
 - Includes loading and empty states.
 - Integrates with logging and monitoring where appropriate.
-

33. ACCEPTANCE CRITERIA

The technical architecture is considered complete when:

- The frontend and backend are modular and maintainable.
 - APIs follow consistent standards.
 - The database is optimized and indexed.
 - Authentication and authorization are secure.
 - Real-time communication works reliably.
 - Deployments are repeatable.
 - Monitoring and logging are in place.
 - The system can scale without major architectural changes.
 - New developers can understand and extend the project with minimal friction.
-

FINAL VISION FOR VOLUME 9

The technical foundation should be as polished as the user experience. Every architectural decision should support long-term maintainability, reliability, and performance, allowing the Restaurant Digital Platform to evolve confidently as new features are introduced.

VOLUME 10

SYSTEM ARCHITECTURE, DOMAIN MODEL, DATA FLOW, BUSINESS RULES & APPLICATION LIFECYCLE

1. INTRODUCTION

This document defines how every module inside the Restaurant Digital Platform communicates, behaves, and exchanges information.

Rather than thinking about database tables, think about the restaurant itself.

When a customer orders food, what actually happens?

How does inventory change?

How is the kitchen notified?

When is GST calculated?

When is WhatsApp triggered?

How is analytics updated?

Everything begins here.

2. DOMAIN DRIVEN DESIGN

The application should be divided into independent business domains.

Each domain owns its own business rules.

Restaurant Platform

|

|— Authentication

- └─ Customer
- └─ Menu
- └─ Orders
- └─ Payments
- └─ Kitchen
- └─ Inventory
- └─ Reservations
- └─ Billing
- └─ Analytics
- └─ CRM
- └─ Notifications
- └─ AI
- └─ Settings
- └─ Reports

Each domain communicates only through defined services.

Never allow one module to directly manipulate another module's internal logic.

3. DOMAIN RESPONSIBILITIES

Authentication

Responsible for

Login

Logout

JWT

Sessions

Permissions

Passwords

Roles

Never stores restaurant business logic.

Menu Domain

Responsible for

Categories

Food

Pricing

Images

Availability

Customization

Variants

Discounts

Best Sellers

Nothing else.

Order Domain

Responsible for

Cart

Checkout

Order Creation

Order Status

Order Timeline

Order History

Order Cancellation

Order Notes

Kitchen Domain

Responsible for

Kitchen Queue

Preparation

Cooking Status

Ready Status

Kitchen Priority

Kitchen Printer

KDS

Inventory Domain

Responsible for

Ingredients

Recipes

Consumption

Stock

Expiry

Suppliers

Waste

Purchases

Billing Domain

Responsible for

GST

Taxes

Invoices

Payments

Refunds

Receipts

Cash Sessions

CRM Domain

Responsible for

Customers

Reviews

Loyalty

Coupons

Campaigns

Customer History

AI Domain

Responsible for

Predictions

Recommendations

Reports

Automation

Smart Insights

Chat Assistant

4. APPLICATION LIFECYCLE

The platform has one lifecycle.

Customer

↓

Website

↓

Order

↓

Kitchen

↓

Billing

↓

Inventory

↓

Analytics

↓

CRM

↓

Reports

↓

AI

Every action should automatically update the remaining modules.

5. COMPLETE ORDER FLOW

Customer Opens Website

↓

Views Menu

↓

Chooses Item

↓

Customization

↓

Cart

↓

Checkout



Payment



Create Order



POS



Kitchen Display



Kitchen Printer



Preparation



Ready



Delivery / Pickup



Completed



Generate Invoice



Deduct Inventory



Update Analytics



Update Customer History



Send WhatsApp



Request Review



AI Learns

No manual synchronization.

Everything happens automatically.

6. EVENT DRIVEN ARCHITECTURE

Every important action creates an event.

Example

Order Created



Kitchen Receives Order



Inventory Reserved

↓

Analytics Updated

↓

Notification Sent

↓

Customer Updated

Instead of one giant controller.

Everything reacts to events.

7. BUSINESS EVENTS

Examples

Customer Registered

Customer Logged In

Order Created

Order Confirmed

Order Cancelled

Payment Success

Payment Failed

Reservation Created

Reservation Confirmed

Reservation Cancelled

Kitchen Started

Kitchen Ready

Inventory Updated

Supplier Added

Coupon Applied

Invoice Generated

Report Exported

Printer Connected

AI Insight Generated

Every event should be logged.

8. STATE MACHINES

Every major entity has states.

Order States

Draft

Pending

Confirmed

Preparing

Ready

Out For Delivery

Completed

Cancelled

Refunded

Only valid transitions allowed.

Example

Preparing

↓

Ready

Allowed

Preparing

↓

Delivered

Not Allowed

Reservation States

Pending

Confirmed

Checked In

Completed

Cancelled

No Show

Payment States

Pending

Authorized

Paid

Failed

Refunded

Cancelled

Inventory States

Available

Low Stock

Out Of Stock

Expired

Reserved

9. BUSINESS RULES

Every rule should exist in only one place.

Examples

Restaurant Closed

↓

No New Orders

Inventory Empty

↓

Item Automatically Hidden

Reservation Full

↓

Booking Disabled

Printer Offline

↓

Display Warning

Payment Failed

↓

Do Not Create Invoice

Kitchen Completed

↓

Deduct Inventory

Order Cancelled

↓

Restore Reserved Stock

10. MODULE COMMUNICATION

Modules communicate through services.

Never access database directly from another module.

Bad

Kitchen

↓

Update Order Database

Good

Kitchen

↓

Order Service



Order Updated Event

11. SERVICE LAYER

Every business action should exist inside services.

Example

Order Service

Inventory Service

Kitchen Service

Billing Service

Customer Service

Menu Service

Notification Service

AI Service

Analytics Service

CRM Service

Report Service

Authentication Service

12. REPOSITORY LAYER

Services never query MongoDB directly.

Instead

Service



Repository



MongoDB

Benefits

Easy Testing

Clean Code

Reusable Queries

Centralized Database Logic

13. API LAYER

Routes should remain extremely thin.

Request



Validation



Service



Repository



Database

↓

Response

Never place business logic inside controllers.

14. VALIDATION FLOW

Every request follows

Client Validation

↓

API Validation

↓

Business Validation

↓

Database Validation

↓

Success

Example

Customer enters phone number.

Frontend checks format.

Backend checks format.

Backend checks duplicates.

Database stores.

15. ERROR FLOW

Every error has

Error Code

Friendly Message

Developer Message

Recovery Action

Example

OUT_OF_STOCK

This item is currently unavailable.

Suggested Action

Try another dish.

16. FILE FLOW

Image Upload

↓

Compression

↓

Optimization

↓

Cloudinary

↓

Store URL

↓

CDN

Never store images directly in MongoDB.

17. PAYMENT FLOW

Customer



Checkout



Payment Gateway



Webhook



Verify Signature



Create Payment



Generate Invoice



Update Order



Send WhatsApp



Analytics

18. PRINTER FLOW

Order Paid



Generate Invoice



Kitchen Ticket



Customer Receipt



Printer Queue



Print



Confirmation

Retry automatically if printing fails.

19. NOTIFICATION FLOW

Trigger



Notification Service



Email

WhatsApp

Dashboard

Push

SMS

↓

Delivery Status

↓

Logs

20. ANALYTICS FLOW

Every completed action updates analytics.

Order Completed

↓

Revenue

↓

Customer

↓

Inventory

↓

Reports

↓

Charts

↓

AI

Real-time updates.

21. AI FLOW

Restaurant Activity

↓

Collect Events

↓

AI Engine

↓

Generate Insights

↓

Dashboard Widget

↓

Manager Decision

AI never changes business data automatically unless explicitly configured.

22. SECURITY FLOW

User

↓

Authentication

↓

Authorization

↓

Permission Check

↓

Validation

↓

Business Logic

↓

Database

↓

Audit Log

↓

Response

Every request should pass through the same security pipeline.

23. AUDIT FLOW

Sensitive Action

↓

Audit Service

↓

Timestamp

↓

User



IP Address



Device



Old Value



New Value



Stored Forever

24. BACKUP FLOW

Daily Backup



Verification



Encryption



Cloud Storage



Health Check



Notification

25. LOGGING ARCHITECTURE

Separate logs.

Application

Security

Payments

Orders

Inventory

Notifications

Automation

AI

System

Error

Debug

Each log should include:

Timestamp

Correlation ID

User (if authenticated)

Module

Severity

26. CONFIGURATION MANAGEMENT

All configurable business rules should be stored in a centralized settings module.

Examples:

Restaurant Name

Working Hours

GST Percentage

Delivery Radius

Minimum Order Value

Printer Settings

WhatsApp Templates

Notification Preferences

Tax Rules

Payment Methods

Avoid hard-coded values.

27. FEATURE FLAGS

Support runtime feature toggles.

Examples:

QR Ordering

AI Assistant

WhatsApp Ordering

Loyalty Program

Online Payments

Reservation System

This allows features to be enabled or disabled without code changes.

28. DATA RETENTION POLICY

Define retention periods for:

Orders

Invoices

Audit Logs

Notifications

Reports

Temporary Files

Analytics Snapshots

Automation Logs

Retention policies should be configurable where practical.

29. FAILURE RECOVERY

Every critical workflow should include recovery logic.

Examples:

Payment webhook retries.

Printer retry queue.

WhatsApp message retry.

Inventory rollback on cancelled orders.

Database transaction compensation.

Background job retry with exponential backoff.

30. ACCEPTANCE CRITERIA

The platform architecture is considered complete when:

- Every business capability belongs to a well-defined domain.
- Modules communicate through services and events rather than direct coupling.
- Business rules are centralized and reusable.
- Critical workflows are resilient to failures.
- Logging, auditing, notifications, analytics, and AI are integrated into the application lifecycle.
- The architecture remains understandable, testable, and extensible for future enhancements.

RESTAURANT DIGITAL PLATFORM

MASTER PRODUCT REQUIREMENTS DOCUMENT (PRD)

VOLUME 11A

COMPLETE DATABASE ARCHITECTURE & MONGODB COLLECTION DESIGN

1. INTRODUCTION

The Restaurant Digital Platform requires a carefully designed MongoDB database that supports:

- High performance
- Scalability
- Flexibility
- Real-time updates
- Strong relationships
- Easy reporting
- Future expansion

The database should be optimized for a **single restaurant** while remaining flexible enough to support additional branches if required in the future.

2. DATABASE PHILOSOPHY

The database should follow these principles:

- One responsibility per collection
 - Minimize data duplication
 - Embed only when appropriate
 - Reference large datasets
 - Optimize read performance
 - Maintain data integrity at the application layer
 - Design for future growth
 - Use timestamps consistently
 - Support auditability
-

3. DATABASE OVERVIEW

MongoDB

|

|— users

|— roles

|— permissions

- └─ customers
- └─ employees
- └─ menu_categories
- └─ menu_items
- └─ menu_variants
- └─ ingredients
- └─ inventory
- └─ suppliers
- └─ purchase_orders
- └─ reservations
- └─ tables
- └─ orders
- └─ order_items
- └─ payments
- └─ invoices
- └─ reviews
- └─ coupons
- └─ campaigns
- └─ notifications
- └─ settings
- └─ audit_logs
- └─ analytics

└─ ai_logs

└─ reports

└─ media

4. USERS COLLECTION

Purpose

Stores all authenticated users.

Includes

Owner

Manager

Cashier

Kitchen Staff

Marketing Staff

Future

Delivery Staff

Fields

_id

fullName

email

phone

passwordHash

roleId

profileImage

status

lastLogin

createdAt

updatedAt

Indexes

email

phone

roleId

status

5. ROLES COLLECTION

Purpose

Role Based Access Control

Examples

Owner

Manager

Cashier

Kitchen

Marketing

Fields

_id

roleName

description

permissions

createdAt

6. PERMISSIONS COLLECTION

Purpose

Centralized permission management.

Examples

Manage Orders

Manage Inventory

Manage Customers

Manage Menu

Manage Coupons

View Reports

Manage Staff

Refund Payments

Change Settings

7. CUSTOMERS COLLECTION

Purpose

Every customer who places an order.

Fields

_id
name
phone
email
addresses
favoriteItems
totalOrders
totalSpent
lastOrderDate
loyaltyPoints
birthday
notes
createdAt
updatedAt
Indexes
phone
email
name

8. EMPLOYEES COLLECTION

Fields

_id

employeeCode

name

phone

email

designation

salary (future)

joiningDate

status

roleId

createdAt

9. MENU CATEGORIES

Examples

Steam Momos

Fried Momos

Kurkure Momos

Chinese

Rice

Noodles

Beverages

Desserts

Fields

_id

name

slug

description

image

displayOrder

status

10. MENU ITEMS

Every food item.

Fields

_id

categoryId

name

slug

description

price

preparationTime

calories

vegType

spiceLevel

images

badges

availability

rating

ingredients

variants

addons

createdAt

Indexes

category

name

availability

price

11. MENU VARIANTS

Purpose

Variants should not be stored separately inside menu items when they become complex.

Example

Steam Momos

↓

Veg

Paneer

Chicken

Half

Full

Fields

_id

menuItemId

variantName

price

sku

status

12. INGREDIENTS COLLECTION

Purpose

Recipe ingredients.

Fields

_id

name

unit

purchasePrice

supplierId

minimumStock

maximumStock

currentStock

expiryDate

storageLocation

13. INVENTORY COLLECTION

Purpose

Live stock.

Fields

_id

ingredientId

availableStock

reservedStock

consumedStock

wasteStock

lastUpdated

14. SUPPLIERS COLLECTION

Fields

_id

businessName

contactPerson

phone

email

GST

address

rating

paymentTerms

createdAt

15. PURCHASE ORDERS

Fields

_id

supplierId

items

status

invoiceNumber

total

orderedDate

receivedDate

16. RESTAURANT TABLES

Purpose

Table Management.

Fields

_id

tableNumber

capacity

floor

status

currentOrderId

Statuses

Available

Occupied

Reserved

Cleaning

Billing

17. RESERVATIONS

Fields

_id

customerId

guestCount

reservationDate

reservationTime

status

specialRequest

assignedTable

notes

18. ORDERS COLLECTION

The most important collection.

Fields

_id

orderNumber

customerId

orderType

status

paymentStatus

paymentMethod

tableId

subtotal

tax

discount

deliveryCharge

grandTotal

notes

estimatedTime

completedAt

createdAt

Indexes

orderNumber

customerId

status

createdAt

19. ORDER ITEMS

Instead of embedding everything inside orders.

Fields

_id

orderId

menuItemId

variantId

quantity

unitPrice

addons

notes

lineTotal

20. PAYMENTS

Fields

_id

orderId

transactionId

gateway

paymentMethod

status

amount

paidAt

21. INVOICES

Fields

_id

invoiceNumber

orderId

GSTNumber

CGST

SGST

IGST

subtotal

tax

grandTotal

pdfUrl

createdAt

22. REVIEWS

Fields

_id

customerId

orderId

rating

review

images

status

reply

createdAt

23. COUPONS

Fields

_id

code

discountType

discountValue

minimumOrder

maximumDiscount

expiryDate

usageLimit

active

24. MARKETING CAMPAIGNS

Fields

_id

campaignName

channel

audience

startDate

endDate

status

analytics

25. NOTIFICATIONS

Fields

_id

userId

title

message

priority

read

type

createdAt

26. SETTINGS

Purpose

Never hardcode restaurant values.

Examples

Restaurant Name

GST Number

Delivery Charges

Business Hours

Payment Methods

Printer Settings

WhatsApp Templates

Theme

Tax Percentage

Logo

Social Links

27. AUDIT LOGS

Every sensitive action.

Fields

_id

userId

action

module

oldValue

newValue

IP

device

createdAt

28. REPORTS

Generated reports.

Fields

_id

reportType

filters

generatedBy

downloadURL

createdAt

29. ANALYTICS SNAPSHOTS

Purpose

Avoid recalculating expensive analytics.

Fields

_id

type

period

metrics

generatedAt

30. AI LOGS

Store

Predictions

Recommendations

Insights

Automation History

Confidence Score

Model Version

Feedback

31. MEDIA COLLECTION

Purpose

Central media management.

Fields

_id

fileName

url

type

size

uploadedBy

createdAt

32. COMMON FIELDS

Every collection should contain

_id

createdAt

updatedAt

Soft Delete

deletedAt

deletedBy

Status

active

inactive

archived

33. INDEXING STRATEGY

Create indexes on

Order Number

Invoice Number

Customer Phone

Email

Reservation Date

Category

Supplier

Inventory SKU

Status

Created Date

Compound Indexes

Examples

status + createdAt

customerId + createdAt

reservationDate + status

category + availability

supplier + ingredient

34. RELATIONSHIP DESIGN

Customer

↓

Orders



Order Items



Menu Item



Ingredients



Inventory



Supplier

Reservations



Customer



Table

Payments



Order

Invoice



Order

Reviews



Customer



Order

35. DOCUMENT SIZE GUIDELINES

Embed:

Addresses

Small Settings

Tags

Add-ons

Badges

Reference:

Orders

Customers

Menu

Inventory

Reports

Analytics

Never create excessively large documents.

36. NAMING CONVENTIONS

Collections

snake_case

Example

menu_items

purchase_orders

audit_logs

Fields

camelCase

customerId

createdAt

updatedAt

paymentStatus

37. SCALABILITY GUIDELINES

Design collections so they can support:

- Multiple restaurant branches
- Franchise management
- Mobile applications
- Customer app
- Delivery app
- Kitchen tablets
- Analytics warehouse
- AI recommendation engine

without changing the existing schema significantly.

38. ACCEPTANCE CRITERIA

The database architecture is considered complete when:

- Every business domain has a dedicated collection.
 - Relationships are clear and efficient.
 - Frequently queried fields are indexed.
 - Data duplication is minimized.
 - Collections remain maintainable as the platform grows.
 - The schema supports all features defined in Volumes 1–10.
 - The design is ready for implementation using MongoDB Atlas and Mongoose.
-

FINAL VISION FOR VOLUME 11A

The database should be treated as the **foundation of the platform**, not just a place to store data. Every collection, relationship, and index should support the restaurant's operational workflows while remaining flexible enough for future enhancements.

RESTAURANT DIGITAL PLATFORM

MASTER PRODUCT REQUIREMENTS DOCUMENT (PRD)

VOLUME 11B

MONGOOSE SCHEMA DESIGN, VALIDATION, INDEXING & DATABASE ENGINEERING STANDARDS

1. INTRODUCTION

This document defines the implementation standards for MongoDB using Mongoose.

Every schema must follow identical conventions.

The objective is consistency, scalability, maintainability and predictable behavior.

Every document inside the database should look as if it was created by the same engineer.

2. GLOBAL SCHEMA STANDARDS

Every schema should include

`_id`

`createdAt`

`updatedAt`

`createdBy`

`updatedBy`

`deletedAt`

`deletedBy`

`isDeleted`

`status`

`version`

`Use`

`timestamps: true`

for every collection.

3. ID STRATEGY

Never expose MongoDB ObjectIds to customers.

Examples

Instead of

6859caab44bfa8f7

Generate

ORD-2026-000125

Examples

Customer

CUS-0001

Reservation

RES-0007

Invoice

INV-00455

Purchase Order

PO-00081

Menu

MENU-0045

4. FIELD VALIDATION STANDARDS

Every field should define

Required

Minimum Length

Maximum Length

Regex

Default Value

Trim

Lowercase

Uppercase

Index

Unique

Nullable

Transform

Example

Phone Number

required

trim

regex

index

Email

lowercase

trim

unique

index

validate

Price

min

required

5. ENUM STRATEGY

Never store magic strings.

Wrong

Preparing

Correct

```
enum OrderStatus {
```

```
PENDING
```

```
CONFIRMED
```

```
PREPARING
```

```
READY
```

```
COMPLETED
```

```
CANCELLED
```

```
REFUNDED
```

```
}
```

Enums required for

Order Status

Payment Status

Reservation Status

Inventory Status

Employee Status

Notification Type

Campaign Type

Role

Permission

Printer Status

Kitchen Status

Delivery Status

6. COMMON SUBDOCUMENTS

Create reusable schemas.

Example

Address

street

landmark

city

district

state

country

postalCode

coordinates

Money

currency

amount

Contact

phone

email

whatsapp

Audit

createdBy

updatedBy

createdAt

updatedAt

Reuse everywhere.

7. USER SCHEMA

Fields

fullName

email

phone

passwordHash

roleId

profileImage

status

permissions

lastLogin

loginHistory

devices

preferences

Virtuals

Full Name

Role Name

Active Sessions

Order Count

Indexes

email unique

phone unique

roleId

status

8. CUSTOMER SCHEMA

Embedded

Addresses

Favorite Items

Delivery Preferences

Notification Preferences

References

Orders

Reviews

Reservations

Validation

Phone unique

Email optional

Name required

9. MENU SCHEMA

Structure

Category

↓

Item

↓

Variant

↓

Addon

Menu Item

name

slug

description

category

images

price

discount

availability

preparationTime

nutrition

ingredients

badges

rating

reviewCount

tags

Virtual

Final Price

10. VARIANT SCHEMA

Example

Steam Momos

↓

Veg

↓

Half

↓

Extra Cheese

↓

Extra Mayo

Pricing calculated dynamically.

11. ORDER SCHEMA

Fields

orderNumber

customerId

tableId

items

subtotal

tax

discount

delivery

grandTotal

status

paymentStatus

paymentMethod

estimatedTime

timeline

notes

Embedded

Timeline

Status History

References

Customer

Payment

Invoice

Indexes

status

customer

createdAt

12. ORDER ITEM SCHEMA

Fields

menuItemid

variantId

quantity

unitPrice

addons

instructions

totalPrice

Never duplicate menu data.

Only snapshot important values.

13. PAYMENT SCHEMA

Fields

Gateway

Transaction ID

Status

Amount

Currency

Webhook Status

Refund Status

Failure Reason

Indexes

Transaction ID

Order

Status

14. INVENTORY SCHEMA

Fields

Ingredient

Stock

Reserved

Consumed

Waste

Supplier

Expiry

Batch

Location

Virtual

Available Stock

Current

-

Reserved

15. PURCHASE ORDER SCHEMA

Fields

Supplier

Items

Quantity

Received

Status

Invoice

Expected Date

Total Cost

Indexes

Supplier

Status

Created Date

16. SUPPLIER SCHEMA

Fields

GST

Contact

Rating

Payment Terms

Purchase History

Outstanding

Preferred Supplier

17. RESERVATION SCHEMA

Fields

Reservation Number

Customer

Guests

Table

Date

Time

Status

Special Request

Arrival

Completion

Indexes

Date

Table

Status

18. REVIEW SCHEMA

Fields

Customer

Order

Rating

Review

Images

Likes

Visibility

Admin Reply

Sentiment Score

AI Summary

19. NOTIFICATION SCHEMA

Fields

Title

Description

Module

Priority

Type

Read

Recipient

Action URL

Expiry

20. SETTINGS SCHEMA

Single document.

Contains

Restaurant Details

Business Hours

Taxes

Payments

Delivery

Printer

WhatsApp

Theme

Logo

Email

Social

Never create multiple settings documents.

21. AI SCHEMA

Store

Insight

Confidence

Generated By

Prompt

Context

Recommendation

Feedback

Accepted

Rejected

Executed

22. AUDIT LOG SCHEMA

Store

User

Module

Action

Old Value

New Value

Device

Browser

IP

Timestamp

Reason

Never modify audit logs.

Append only.

23. INDEXING STRATEGY

Single Field

Email

Phone

Invoice

Order Number

Compound

status

createdAt

Customer

customerId

createdAt

Reservation

date

status

Menu

category

availability

Inventory

ingredient

supplier

24. TEXT SEARCH

Enable Full Text Search

Menu

Customer

Supplier

Orders

Reviews

Reports

Weighted Search

Menu Name

Weight 10

Description

Weight 6

Tags

Weight 3

25. TTL INDEXES

Automatically remove

OTP

Verification Codes

Sessions

Temporary Files

Password Reset Tokens

Example

`expireAfterSeconds`

Never use TTL on business records.

26. SOFT DELETE STRATEGY

Never remove

Orders

Invoices

Payments

Audit Logs

Instead

`isDeleted`

`deletedAt`

`deletedBy`

Queries should automatically exclude deleted records.

27. VERSIONING

Enable optimistic concurrency.

Example

___v

Prevent simultaneous updates.

Useful for

Inventory

Orders

Payments

28. POPULATION STRATEGY

Populate only when necessary.

Example

Order

↓

Customer

↓

Menu

↓

Payment

↓

Invoice

Avoid deeply nested population.

Prefer aggregation.

29. AGGREGATION PIPELINES

Predefine reusable pipelines.

Revenue

Today's Sales

Weekly Sales

Top Customers

Best Sellers

Inventory Usage

Customer Growth

Peak Hours

Cancellation Rate

Payment Breakdown

Store pipeline builders.

30. TRANSACTION STRATEGY

Use MongoDB Transactions for

Payment

↓

Invoice

↓

Inventory

↓

Order

↓

Analytics

Either

Everything succeeds

OR

Everything rolls back.

Never leave partial updates.

31. MIDDLEWARE

Pre Save

Generate IDs

Slug

Audit

Validate

Post Save

Analytics

Notifications

Inventory

AI

Pre Delete

Soft Delete

Audit

32. VIRTUAL FIELDS

Examples

Customer

Full Address

Order Count

Lifetime Spend

Menu

Discounted Price

Inventory

Available Stock

Reservation

Remaining Time

33. REPOSITORY PATTERN

Never use

`Order.find()`

inside services.

Instead

`OrderRepository`

↓

`findByStatus()`

↓

findByCustomer()

↓

createOrder()

↓

cancelOrder()

↓

completeOrder()

34. BASE REPOSITORY

Every repository extends

BaseRepository

Contains

Create

Update

Delete

Find

Pagination

Filtering

Sorting

Transactions

35. QUERY OPTIMIZATION

Always

Limit fields

Project only needed values

Use lean()

Avoid N+1 queries

Use indexes

Cache expensive queries

36. DATABASE EVENTS

Every successful write generates events.

Examples

OrderCreated

OrderPaid

OrderCompleted

ReservationConfirmed

InventoryUpdated

ReviewAdded

CouponRedeemed

PaymentRefunded

These events trigger

Analytics

Notifications

AI

37. SCHEMA DOCUMENTATION

Every schema should include

Description

Relationships

Indexes

Validation Rules

Business Rules

Example Document

Migration Notes

Future Expansion

38. DATABASE TESTING

Every schema requires tests.

Validation Tests

Unique Constraint Tests

Index Tests

Transaction Tests

Concurrency Tests

Aggregation Tests

Performance Tests

39. ACCEPTANCE CRITERIA

The schema layer is considered complete when:

- Every collection has a clearly defined Mongoose schema.
 - Validation rules are enforced consistently.
 - Indexes support expected query patterns.
 - Transactions protect critical business operations.
 - Repositories encapsulate database access.
 - Middleware automates repetitive tasks.
 - Aggregation pipelines support analytics without excessive application logic.
 - Soft deletes, audit trails, and versioning are implemented where appropriate.
 - The data layer is optimized for maintainability, scalability, and future growth.
-

FINAL VISION FOR VOLUME 11B

The database layer should feel as professionally engineered as the user interface. Every schema, repository, index, validation rule, and transaction should exist to support reliable restaurant operations while remaining clean, extensible, and easy to maintain.

RESTAURANT DIGITAL PLATFORM

MASTER PRODUCT REQUIREMENTS DOCUMENT (PRD)

VOLUME 11C

REST API SPECIFICATION, WEBSOCKET ARCHITECTURE, BACKGROUND JOBS & BACKEND COMMUNICATION STANDARDS

1. INTRODUCTION

The Restaurant Digital Platform is built around an API-first architecture.

Every client—including:

- Public Website
- Admin Dashboard
- POS
- Kitchen Display System
- Future Mobile App
- AI Services
- WhatsApp Integration

must communicate through standardized APIs.

The API should be:

- RESTful
 - Predictable
 - Versioned
 - Secure
 - Documented
 - Consistent
 - Easy to extend
-

2. API DESIGN PRINCIPLES

Every endpoint should follow:

- Nouns over verbs.
- Consistent HTTP methods.
- Idempotent updates where applicable.
- Predictable status codes.
- Consistent pagination.
- Standard response format.
- Strong validation.

Never create inconsistent APIs.

3. API VERSIONING

Every endpoint begins with:

/api/v1

Examples:

GET /api/v1/menu

POST /api/v1/orders

GET /api/v1/customers

PUT /api/v1/settings

Future versions:

/api/v2

/api/v3

4. STANDARD RESPONSE FORMAT

Every response should follow:

```
{  
  "success": true,  
  "message": "Order created successfully.",  
}
```

```
"data": {},
"meta": {},
"timestamp": "2026-06-22T14:30:00Z"
}
```

Error responses:

```
{
  "success": false,
  "error": {
    "code": "OUT_OF_STOCK",
    "message": "Steam Chicken Momos are unavailable.",
    "details": {}
  },
  "timestamp": "2026-06-22T14:30:00Z"
}
```

Never return inconsistent JSON structures.

5. AUTHENTICATION API

Endpoints

POST

/auth/login

/auth/logout

/auth/refresh

/auth/forgot-password

/auth/reset-password

/auth/change-password

/auth/profile

/auth/verify

Future

OTP Login

Google Login

WhatsApp Login

6. USER MANAGEMENT API

GET

/users

/users/:id

POST

/users

PUT

/users/:id

DELETE

/users/:id

Admin only.

7. CUSTOMER API

GET

/customers

/customers/:id

/customers/:id/orders

/customers/:id/reviews

/customers/:id/reservations
POST

/customers

PUT

/customers/:id

8. MENU API

GET

/menu

/menu/:slug

/menu/categories

/menu/popular

/menu/recommended

/menu/search

Admin

POST

/menu

PUT

/menu/:id

DELETE

/menu/:id

9. ORDER API

Customer

POST

/orders

Retrieve

GET

/orders

/orders/:id

/orders/history

/orders/track

Update

PATCH

/orders/:id/status

/orders/:id/cancel

10. CART API

GET

/cart

POST

/cart/items

PUT

/cart/items/:id

DELETE

/cart/items/:id

DELETE

/cart

11. CHECKOUT API

POST

/checkout

Validate

Coupon

Inventory

Pricing

Taxes

Payment

Return

Order

Payment Intent

Estimated Time

12. PAYMENT API

POST

/payments/create

/payments/verify

/payments/refund

GET

/payments/:id

Webhooks

POST

/webhooks/payment

13. RESERVATION API

GET

/reservations

/reservations/:id

POST

/reservations

PATCH

/reservations/:id

DELETE

/reservations/:id

14. INVENTORY API

GET

/inventory

/inventory/low-stock

/inventory/history

POST

/inventory

PATCH

/inventory/:id

15. SUPPLIER API

GET

/suppliers
POST

/suppliers
PUT

/suppliers/:id
DELETE

/suppliers/:id

16. PURCHASE ORDER API

GET

/purchase-orders
POST

/purchase-orders
PATCH

/purchase-orders/:id

17. REVIEW API

GET

/reviews
POST

/reviews
PUT

/reviews/:id
DELETE

/reviews/:id

18. COUPON API

GET

/coupons
POST

/coupons
PATCH

/coupons/:id
DELETE

/coupons/:id

19. ANALYTICS API

GET

/analytics/dashboard

/analytics/revenue

/analytics/orders

/analytics/customers

/analytics/menu

/analytics/inventory

/analytics/marketing

Support

Date Range

Grouping

Comparison

Export

20. REPORT API

GET

/reports

Generate

POST

/reports/generate

Download

GET

/reports/:id/download

21. SETTINGS API

GET

/settings

PATCH

/settings

Includes

Restaurant

GST

Delivery

Printer

WhatsApp

Theme

Payment

22. NOTIFICATION API

GET

/notifications

PATCH

/notifications/read

DELETE

/notifications

23. AI API

POST

/ai/chat

POST

/ai/report

GET

/ai/recommendations

GET

/ai/insights

POST

/ai/menu-analysis

24. MEDIA API

POST

/media/upload

DELETE

/media/:id

GET

/media

Supports

Images

PDF

Reports

Restaurant Assets

25. DASHBOARD API

GET

/dashboard

/dashboard/kpis

/dashboard/recent-orders

/dashboard/activity

/dashboard/charts

26. SEARCH API

Global Search

GET

/search?q=

Should search

Customers

Orders

Reservations

Menu

Reports

Inventory

Suppliers

27. API PAGINATION

Every list endpoint

?page=

&limit=

&sort=

&search=

&filter=

Example

GET

/orders?page=2&limit=20

28. FILTERING

Support

Status

Category

Date

Customer

Payment

Price

Supplier

Role

Examples

GET

/orders?status=completed

GET

/menu?category=momos

29. SORTING

Support

Ascending

Descending

Date

Price

Popularity

Revenue

Alphabetical

30. FILE UPLOAD

Support

Images

PDF

CSV

Excel

Maximum size configurable.

Compression

Virus Scan (future)

31. WEBSOCKET EVENTS

Real-time communication.

Events

order.created

order.updated

order.cancelled

order.completed

payment.success

payment.failed

reservation.created

reservation.confirmed

inventory.low

inventory.updated

notification.created

dashboard.updated

printer.connected

printer.offline

32. EVENT PAYLOAD STANDARD

Every event

```
{  
  "event": "order.created",  
  "timestamp": "...",  
  "payload": {}  
}
```

33. BACKGROUND JOBS

Jobs

Generate Reports

Send WhatsApp

Email

Analytics

Inventory Sync

Daily Backup

Cleanup

Retry Payments

Campaign Scheduler

AI Insights

34. JOB QUEUE

Queue Types

Critical

Orders

Payments

Kitchen

Normal

Notifications

Reports

Analytics

Low Priority

Cleanup

AI

Campaigns

35. WEBHOOKS

Incoming

Payment Gateway

WhatsApp

Future

Google Reviews

Third-party Delivery

Webhook validation

Signature verification

Retry handling

Idempotency

36. RATE LIMITING

Public

100 requests/minute

Authenticated

300 requests/minute

Admin

Configurable

Protect

Login

Payments

Reports

Search

37. API SECURITY

JWT

HTTPS

Helmet

CORS

Rate Limiting

CSRF

Input Sanitization

Schema Validation

Request Size Limits

Secure Headers

Audit Logging

38. ERROR CODE CATALOG

Define reusable error codes.

Examples

AUTH_INVALID

AUTH_EXPIRED

ORDER_NOT_FOUND

OUT_OF_STOCK

PAYMENT_FAILED

COUPON_INVALID

TABLE_OCCUPIED

RESERVATION_FULL

PRINTER_OFFLINE

AI_TIMEOUT

VALIDATION_ERROR

SERVER_ERROR

39. OPENAPI / SWAGGER

Every endpoint should include:

- Description
- Request Schema
- Response Schema
- Status Codes
- Authentication
- Examples
- Error Cases
- Rate Limits

Automatically generate API documentation.

40. ACCEPTANCE CRITERIA

The API layer is considered complete when:

- All business modules expose consistent REST endpoints.
- Responses follow a unified format.
- Authentication and authorization are enforced.
- WebSocket events provide real-time updates.
- Background jobs handle asynchronous work reliably.

- Webhooks are secure and idempotent.
 - APIs are fully documented and versioned.
 - The backend can support the website, admin dashboard, POS, KDS, AI services, and future mobile applications through the same API surface.
-

FINAL VISION FOR VOLUME 11C

The API should become the **contract of the entire platform**. Every client—whether it's the public website, POS terminal, Kitchen Display System, admin dashboard, AI assistant, or a future mobile app—should interact with the same well-designed, secure, versioned, and documented backend.

RESTAURANT DIGITAL PLATFORM

MASTER PRODUCT REQUIREMENTS DOCUMENT (PRD)

VOLUME 11D

PRODUCTION ENGINEERING, EVENT ORCHESTRATION, QUEUES, CACHING, OBSERVABILITY & DEPLOYMENT ARCHITECTURE

1. INTRODUCTION

The Restaurant Digital Platform is more than a website.

It is a real-time operational system where multiple users interact simultaneously:

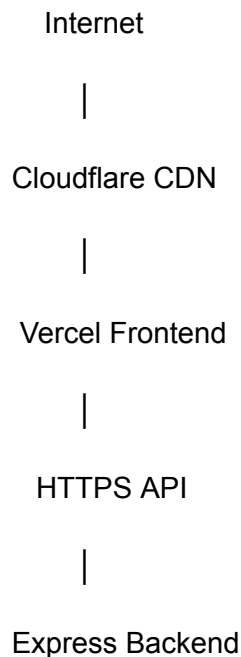
- Customers
- Cashiers
- Kitchen Staff
- Restaurant Manager
- Owner
- AI Assistant
- WhatsApp
- Payment Gateway
- Thermal Printer

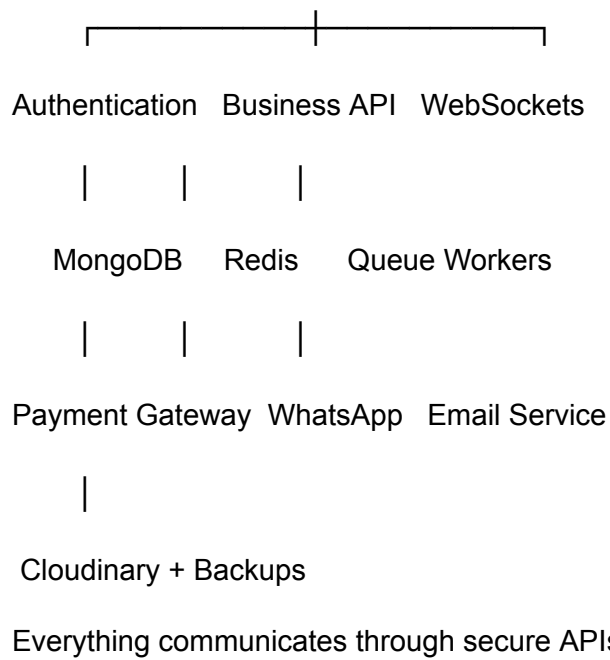
Every action must remain synchronized.

Every failure must be recoverable.

Every transaction must remain consistent.

2. PRODUCTION ARCHITECTURE





3. REQUEST LIFECYCLE

Every request follows exactly the same path.

Client



HTTPS



Express



Rate Limiter



Authentication

↓

Authorization

↓

Validation

↓

Business Service

↓

Repository

↓

MongoDB

↓

Response Formatter

↓

Response

Never bypass middleware.

4. EVENT-DRIVEN ARCHITECTURE

The application communicates internally through events.

Example

Order Created

↓

Order Service

↓

Emit Event

↓

Kitchen

Inventory

Analytics

Notifications

AI

CRM

Reports

Each module listens only to events relevant to it.

Avoid direct coupling.

5. EVENT BUS

Every business event should be published to a centralized event bus.

Examples:

- order.created
- order.confirmed
- order.cancelled
- payment.completed
- payment.failed
- reservation.created
- reservation.confirmed
- inventory.updated
- inventory.low_stock
- menu.updated

- customer.created
- review.created
- notification.sent
- ai.insight.generated

Every event includes:

Event Name

Event ID

Correlation ID

Timestamp

Version

Actor

Payload

Metadata

6. CORRELATION ID

Every request receives a Correlation ID.

Example

REQ-4F8B-9A21

Every log.

Every queue.

Every event.

Every webhook.

Every API call.

Every notification.

Every AI action.

Should reference this ID.

Debugging becomes simple.

7. BACKGROUND JOB ARCHITECTURE

Long-running tasks should never block API responses.

Examples:

Generate invoice PDF.

↓

Send WhatsApp.

↓

Email receipt.

↓

Generate analytics.

↓

Update CRM.

↓

Generate AI insight.

↓

Archive logs.

↓

Create backup.

API responds immediately.

Workers perform background tasks.

8. JOB QUEUE DESIGN

Create dedicated queues.

Critical Queue

- Orders
- Payments
- Refunds
- Kitchen Tickets

High Priority

- Inventory
- Reservations
- Printer Jobs

Medium Priority

- WhatsApp
- Emails
- Reports

Low Priority

- Analytics
- AI
- Cleanup
- Archive

Each queue should support:

- Retry
- Delay
- Priority
- Dead Letter Queue
- Metrics

9. RETRY STRATEGY

Transient failures should retry automatically.

Examples:

WhatsApp failed

↓

Retry after 30 seconds

↓

Retry after 2 minutes

↓

Retry after 10 minutes

↓

Mark failed

↓

Notify administrator

Use exponential backoff.

Never retry indefinitely.

10. DEAD LETTER QUEUE

If a job repeatedly fails:

Move it into a Dead Letter Queue.

Store:

Job

Reason

Attempts

Stack Trace

Payload

Timestamp

Administrator can:

Retry

Discard

Investigate

11. DATABASE TRANSACTIONS

Critical workflows must use MongoDB transactions.

Example

Payment

↓

Order

↓

Invoice

↓

Inventory

↓

Analytics

↓

Commit

If any step fails:

Rollback everything.

Never leave partial business state.

12. IDEMPOTENCY

Support idempotency for critical operations.

Examples:

Payment webhook.

Order creation.

Refund.

Invoice generation.

Duplicate requests should never create duplicate business records.

13. REDIS STRATEGY

Use Redis for:

Session cache.

Menu cache.

Popular dishes.

Rate limiting.

Temporary OTPs.

Queue management.

Live dashboard metrics.

Do not store permanent business records.

14. CACHING LAYERS

Level 1

Browser Cache

Level 2

CDN Cache

Level 3

Application Cache

Level 4

Redis Cache

Level 5

Database

Always invalidate caches after updates.

15. CACHE STRATEGY

Cache:

Restaurant settings.

Menu.

Categories.

Gallery.

Popular dishes.

Business hours.

Do NOT cache:

Orders.

Payments.

Inventory.

Customer balances.

Active kitchen status.

16. WEBSOCKET ARCHITECTURE

Real-time channels.

Customer

- Order tracking

Kitchen

- New orders

Cashier

- Payment updates

Manager

- Dashboard KPIs

Admin

- System alerts

Each client subscribes only to authorized channels.

17. PRINTER SERVICE

Printer communication should occur through a dedicated print service.

Workflow:

Generate receipt.

↓

Queue print job.

↓

Printer.

↓

Acknowledgement.

↓

Success.

↓

Audit log.

If printer unavailable:

Retry.

Alert staff.

Preserve print queue.

18. FILE STORAGE ARCHITECTURE

Uploads:

↓

Validation.

↓

Virus scan (future).

↓

Compression.

↓

Optimization.

↓

Clouinary.

↓

Metadata stored in MongoDB.

Never store large binary files in MongoDB.

19. LOGGING ARCHITECTURE

Separate logs by category.

Application.

Authentication.

Orders.

Payments.

Inventory.

Kitchen.

Printer.

WhatsApp.

AI.

Background Jobs.

Security.

Every log contains:

Timestamp.

Correlation ID.

Module.

Severity.

User.

Request.

Response time.

20. OBSERVABILITY

Monitor:

API latency.

Error rate.

CPU.

Memory.

Database response time.

Queue size.

WebSocket connections.

Printer availability.

Payment success rate.

WhatsApp delivery.

Dashboard health.

21. METRICS

Collect:

Orders per minute.

Revenue per hour.

API requests.

Database queries.

Cache hit ratio.

Queue throughput.

Average kitchen preparation time.

Payment latency.

Printer success rate.

22. HEALTH CHECK ENDPOINTS

Expose endpoints.

/health

/health/live

/health/ready

/metrics

Checks include:

Database.

Redis.

Queue.

Clouldinary.

WhatsApp.

Payment Gateway.

Printer.

Disk space.

Memory.

23. ALERTING

Alert administrators when:

Database unavailable.

Queue backlog exceeds threshold.

Payment gateway unavailable.

Printer offline.

Backup fails.

Storage nearing limit.

Excessive API errors.

Repeated authentication failures.

24. BACKUP STRATEGY

Automatic:

Daily incremental.

Weekly full.

Monthly archive.

Retention:

30 days.

90 days.

1 year.

Support encrypted restoration.

Verify backup integrity regularly.

25. DISASTER RECOVERY

Plan for:

Server failure.

Database corruption.

Cloud outage.

Printer failure.

Payment gateway outage.

Internet outage.

Recovery objectives:

Minimal downtime.

Minimal data loss.

Document restoration procedures.

26. DEPLOYMENT PIPELINE

CI Pipeline:

Install dependencies.



Lint.



Type check.



Unit tests.



Integration tests.



Build.



Security scan.



Deploy preview.



Production deployment.



Health verification.

Automatic rollback on failed deployment.

27. ENVIRONMENTS

Maintain separate environments.

Development.

Testing.

Staging.

Production.

Each environment has:

Separate database.

Separate secrets.

Separate storage.

Separate API keys.

Never share production credentials.

28. FEATURE FLAGS

Support runtime feature flags.

Examples:

AI Assistant.

QR Ordering.

Loyalty Program.

WhatsApp Ordering.

Voice Ordering.

New Checkout.

Dark Dashboard.

Flags should allow gradual rollout without redeployment.

29. SECURITY OPERATIONS

Operational security includes:

Encrypted secrets.

HTTPS everywhere.

JWT rotation.

Password hashing.

Input validation.

Rate limiting.

Dependency scanning.

Audit logging.

Least privilege access.

Security headers.

Regular dependency updates.

30. LOAD TESTING

Test scenarios:

100 concurrent users.

500 concurrent users.

1000 concurrent users.

Peak dinner rush.

Large online ordering spikes.

Simulate:

Order placement.

Checkout.

Kitchen updates.

Dashboard usage.

Ensure graceful degradation.

31. PERFORMANCE TARGETS

Public Website:

First Contentful Paint < 1.5 seconds.

Largest Contentful Paint < 2.5 seconds.

CLS < 0.1.

API:

Average response < 200 ms.

95th percentile < 500 ms.

WebSockets:

Update latency < 100 ms.

Dashboard:

Load < 2 seconds.

32. PRODUCTION CHECKLIST

Before every release verify:

- All tests passing.
 - Database migrations complete.
 - Environment variables validated.
 - API documentation updated.
 - Feature flags configured.
 - Backup successful.
 - Monitoring active.
 - Alerting configured.
 - Security scan passed.
 - Rollback plan verified.
-

33. MAINTENANCE STRATEGY

Schedule:

Database optimization.

Index review.

Log cleanup.

Media cleanup.

Dependency updates.

Security patches.

Performance review.

Backup verification.

Document every maintenance operation.

34. OBSERVABILITY DASHBOARD

Provide a unified operations dashboard showing:

System Health.

API Health.

Orders.

Kitchen Queue.

Payments.

Queue Status.

Printer Status.

Database Health.

Redis Health.

Background Jobs.

Recent Errors.

Active Users.

Critical Alerts.

Managers should understand system health at a glance.

35. ACCEPTANCE CRITERIA

The production architecture is considered complete when:

- Critical workflows are event-driven and resilient.
 - Background jobs execute asynchronously without blocking user requests.
 - Transactions maintain data consistency.
 - Queues support retries and dead-letter handling.
 - Redis is used appropriately for caching and temporary data.
 - WebSockets provide reliable real-time communication.
 - Monitoring, logging, metrics, and alerts provide full operational visibility.
 - Backups and disaster recovery procedures are documented and tested.
 - The deployment pipeline supports safe, repeatable releases with rollback capability.
-

FINAL VISION FOR VOLUME 11D

The Restaurant Digital Platform should not merely function—it should be **operationally excellent**. Every request, event, queue, cache, deployment, and recovery process should be designed with production reliability in mind, ensuring the restaurant can depend on the system during its busiest hours.

RESTAURANT DIGITAL PLATFORM

MASTER PRODUCT REQUIREMENTS DOCUMENT (PRD)

VOLUME 12

FRONTEND ARCHITECTURE, DESIGN SYSTEM IMPLEMENTATION & REACT ENGINEERING STANDARDS

1. INTRODUCTION

The frontend of the Restaurant Digital Platform is responsible for delivering a premium, responsive, performant, and accessible experience across every interface.

This includes:

- Public Website
- Online Ordering
- Customer Dashboard

- Admin Dashboard
- POS
- Kitchen Display System
- Reservation System
- Analytics Dashboard

Every screen should feel like part of the same product.

2. FRONTEND PHILOSOPHY

The frontend should follow these principles:

- Component First
- Feature First
- Atomic Design
- Composition over Inheritance
- Reusability over Duplication
- Accessibility by Default
- Performance First
- Mobile First
- Design System Driven

Every UI element should be reusable.

3. TECHNOLOGY STACK

Core

React 19

TypeScript

Vite

Tailwind CSS v4

UI

shadcn/ui

Radix UI

Lucide React

Animation

Framer Motion

GSAP

Lenis

Motion One (where lightweight)

Forms

React Hook Form

Zod

Data

TanStack Query

Axios

Charts

Recharts

Utilities

clsx

tailwind-merge

date-fns

react-intersection-observer

Embla Carousel

Sonner

React Helmet Async

4. PROJECT STRUCTURE

src/

- app/
- assets/
- components/
- ui/
- layout/
- forms/
- charts/
- common/
- feedback/
- navigation/
- overlays/
- typography/
- cards/
- tables/
- dialogs/
- animations/
- features/
- auth/
- menu/
- cart/
- checkout/
- orders/
- reservations/
- inventory/
- customers/

- analytics/
- reports/
- settings/
- ai/
- hooks/
- services/
- lib/
- utils/
- constants/
- types/
- routes/
- layouts/
- providers/
- store/
- styles/
- main.tsx

Never organize by pages alone.

Always organize by features.

5. ATOMIC DESIGN

Structure

Atoms

↓

Molecules

↓

Organisms

↓

Templates

↓

Pages

Example

Button



MenuCard



MenuGrid



MenuPage

Every component belongs to one level.

6. DESIGN TOKENS

Never hardcode values.

Create centralized tokens.

Colors

Typography

Spacing

Radius

Shadow

Blur

Animation

Opacity

Borders

Breakpoints

Every component consumes tokens.

7. COLOR TOKENS

background.primary

background.secondary

surface.default

surface.elevated

accent.gold

accent.hover

text.primary

text.secondary

border.default

success

warning

error

Never use raw hex values inside components.

8. TYPOGRAPHY TOKENS

Define reusable styles.

Display

Hero

H1

H2

H3

Body

Small

Caption

Label

Code

Each token includes:

Font

Weight

Line Height

Letter Spacing

9. SPACING TOKENS

Use an 8-point system.

4

8

12

16

24

32

40

48

64

80

96

120

140

Never use arbitrary spacing.

10. COMPONENT LIBRARY

Build reusable components.

Buttons

Inputs

Select

Textarea

Checkbox

Radio

Badge

Avatar

Navbar

Sidebar

Breadcrumb

Tabs

Accordion

Dialog

Drawer

Tooltip

Popover

Toast

Progress

Pagination

Calendar

Date Picker

Time Picker

Tables

Charts

Cards

Gallery

Carousels

Each component supports:

Variants

Sizes

Disabled

Loading

Hover

Focus

Dark Theme

Accessibility

11. LAYOUT SYSTEM

Layouts include:

Public Website

Admin Dashboard

POS

Kitchen Display

Authentication

Customer Dashboard

Each layout manages:

Navigation

Sidebar

Header

Footer

Content

Responsive behavior

12. ROUTING ARCHITECTURE

Use React Router.

Example

/

/menu

/cart

/checkout

/orders

/reservation

/dashboard

/admin

/inventory

/reports

/settings

/login

Use lazy loading for every major route.

13. FEATURE MODULES

Every feature contains

components

hooks

services

api

schemas

constants

types

utils

pages

No shared business logic between unrelated features.

14. API LAYER

Never call Axios directly inside components.

Instead

Component

↓

Hook

↓

Service

↓

API Client

Example

MenuService

↓

MenuAPI

↓

Axios

15. CUSTOM HOOKS

Reusable hooks.

Examples

useMenu

useOrders

useCustomers

useReservations

useInventory

useAnalytics

useInfiniteScroll

usePagination

useSearch

useTheme

useDebounce

useRealtime

16. STATE MANAGEMENT

Separate state.

UI State

↓

useState

Form State

↓

React Hook Form

Server State



TanStack Query

Authentication



Context

Persistent State



Local Storage

Never place server data in Context.

17. FORM STANDARDS

Every form includes

Validation

Loading

Error

Success

Accessibility

Auto Focus

Keyboard Navigation

Optimistic Updates where appropriate.

18. TABLE SYSTEM

Reusable data table.

Supports

Pagination

Search

Sorting

Filtering

Column Visibility

Export

Bulk Actions

Sticky Header

Responsive Layout

19. CHART SYSTEM

Reusable wrappers around Recharts.

Charts

Line

Area

Bar

Pie

Radar

Heatmap (future)

Each chart supports:

Loading

Empty

Error

Dark Theme

Export

20. ANIMATION ARCHITECTURE

Animation libraries:

Framer Motion

↓

Component animations

GSAP

↓

Hero

Landing

Storytelling

Lenis

↓

Smooth scrolling

Avoid using GSAP for basic UI interactions.

21. PAGE TRANSITIONS

Every page transition should include:

Fade

Slide

Blur

Scale

Duration

300–500 ms

Respect reduced-motion preferences.

22. MICRO INTERACTIONS

Hover Lift

Button Press

Image Zoom

Ripple

Loading Skeleton

Toast

Drawer

Accordion

Tabs

Progress Indicators

Input Focus

Magnetic CTA (landing pages only)

23. IMAGE OPTIMIZATION

Responsive images.

Lazy loading.

Blur placeholders.

WebP/AVIF.

Correct aspect ratios.

Avoid layout shifts.

24. RESPONSIVE STRATEGY

Breakpoints

360

480

640

768

1024

1280

1440

1600

Design mobile first.

25. ACCESSIBILITY

WCAG AA.

Keyboard navigation.

ARIA.

Focus management.

Semantic HTML.

Screen reader support.

Reduced motion.

Color contrast.

Skip navigation links.

26. ERROR BOUNDARIES

Every feature should have:

Error Boundary

Fallback UI

Retry Button

Logging

Recovery path

27. LOADING STATES

Never display blank pages.

Use:

Skeletons

Spinners (only for small actions)

Progress bars

Placeholder cards

Optimistic updates where possible

28. EMPTY STATES

Every empty state should include:

Illustration

Helpful text

Suggested action

Primary CTA

Never show an empty table without guidance.

29. PERFORMANCE

Goals:

Lighthouse Performance > 95

CLS < 0.1

LCP < 2.5 s

INP < 200 ms

Bundle optimization:

- Route-level code splitting
- Component lazy loading
- Dynamic imports
- Memoization where justified

- Virtualized lists for large datasets
 - Avoid unnecessary re-renders
-

30. FRONTEND SECURITY

Never expose secrets.

Sanitize user input.

Escape dynamic content.

Validate uploads.

Protect routes.

Handle expired tokens gracefully.

Avoid storing sensitive data in Local Storage unless necessary.

31. TESTING STRATEGY

Component Tests

Hook Tests

Integration Tests

Accessibility Tests

Visual Regression Tests

End-to-End Tests

Critical flows:

Login

Order

Checkout

Reservation

POS

Dashboard

32. STORYBOOK

Every reusable component should have:

Story

Variants

States

Documentation

Accessibility notes

Usage examples

Interactive controls

Storybook becomes the living UI documentation.

33. CODE QUALITY

Strict TypeScript.

ESLint.

Prettier.

No inline CSS.

No duplicated logic.

Small focused components.

Meaningful names.

JSDoc for complex utilities.

34. FRONTEND DOCUMENTATION

Maintain:

Component Library

Hooks Guide

Theme Tokens

Folder Structure

Animation Guide

Accessibility Guide

Performance Checklist

Contribution Guide

Coding Standards

35. FRONTEND ACCEPTANCE CRITERIA

The frontend is considered complete when:

- Every interface follows the design system.
- Components are reusable and documented.
- Feature modules are isolated and maintainable.
- Accessibility standards are met.
- Animations are purposeful and performant.
- API communication is abstracted through services.
- Performance targets are achieved.
- The codebase is easy to understand, test, and extend.

36. FINAL VISION

The frontend should feel as refined as the restaurant itself.

Whether a customer is browsing the menu, a cashier is using the POS, or the owner is reviewing analytics, every interaction should feel intentional, polished, and cohesive.

The frontend architecture should make it possible to evolve the product for years without sacrificing maintainability or design consistency.

RESTAURANT DIGITAL PLATFORM

UX SPECIFICATION DOCUMENT

VOLUME 13

COMPLETE UI/UX SCREEN SPECIFICATIONS & USER EXPERIENCE BLUEPRINT

1. INTRODUCTION

This document defines every screen within the Restaurant Digital Platform.

It specifies:

- Screen purpose
- User goals
- Layout hierarchy
- Navigation
- Components
- States
- Animations
- Responsive behavior
- Accessibility
- Empty states
- Error handling
- Loading behavior
- Micro interactions
- User flows

Every screen should have a clear objective.

No screen exists without purpose.

2. SCREEN DESIGN PHILOSOPHY

Every screen should answer three questions immediately.

Where am I?

Navigation should clearly indicate location.

What can I do?

Actions should be obvious.

What should I do next?

The interface should guide users naturally.

3. COMPLETE APPLICATION MAP

PUBLIC WEBSITE

Home

Menu

Menu Details

Gallery

About

Reservation

Contact

Cart

Checkout

Order Tracking

Login

Register

Customer Dashboard

ADMIN

Dashboard

Orders

Order Details

POS

Kitchen Display

Reservations

Menu Management

Category Management

Inventory

Ingredients

Suppliers

Customers

CRM

Reviews

Marketing

Coupons

Reports

Analytics

Notifications

Employees

Settings

AI Assistant

System Health

Audit Logs

4. GLOBAL NAVIGATION

Every screen follows consistent navigation.

Public

Navbar



Content



Footer

Dashboard

Sidebar



Top Bar



Content



Drawer

POS

Full Screen Workspace

Minimal Navigation

Maximum Focus

5. SCREEN SPECIFICATION TEMPLATE

Every screen should contain:

Purpose

Primary User

Goals

Entry Points

Exit Points

Layout

Components

Primary Actions

Secondary Actions

Loading States

Empty States

Error States

Responsive Rules

Accessibility

Analytics Events

Animations

Performance Targets

Acceptance Criteria

6. HOME PAGE

Purpose

Introduce the restaurant.

Increase online orders.

Increase reservations.

Build trust.

Layout

Hero



Trust Bar



Signature Dishes



Restaurant Story



Featured Menu



Gallery



Reviews



Reservation CTA



Contact



Footer

Hero

Height

100vh

Background

Video or Cinematic Image

CTA

Order Now

Reserve Table

Explore Menu

Animation

Parallax

Text Reveal

Steam Effect

Floating Ingredients

7. MENU PAGE

Purpose

Food discovery.

Layout

Category Sidebar

↓

Search

↓

Filters

↓

Food Grid



Floating Cart

Food Card

Image

Name

Description

Price

Rating

Preparation Time

Veg Badge

Popular Badge

Add Button

Quick View

Hover

Scale

Glow

8. FOOD DETAIL SCREEN

Purpose

Help customers decide confidently.

Sections

Large Gallery

Description

Ingredients

Customization

Nutrition

Reviews

Related Items

Add to Cart

Sticky CTA on mobile.

9. CART SCREEN

Layout

Cart Items

↓

Summary

↓

Coupon

↓

Checkout

Item Card

Image

Quantity

Variant

Notes

Price

Actions

Increase

Decrease

Remove

Save for Later

10. CHECKOUT SCREEN

Steps

Customer



Address



Review



Payment



Confirmation

Progress indicator visible.

No unnecessary distractions.

11. ORDER TRACKING

Timeline

Received



Confirmed



Preparing



Ready



Out for Delivery



Delivered

Live updates.

Estimated time.

Driver info (future).

12. CUSTOMER DASHBOARD

Modules

Overview

Orders

Reservations

Favorites

Addresses

Profile

Invoices

Notifications

Settings

Cards

Recent Orders

Repeat Order

Reward Points (future)

13. LOGIN

Minimal.

Logo.

Brand.

Illustration.

Email.

Password.

Remember Me.

Forgot Password.

Google Login (future).

OTP Login (future).

14. ADMIN DASHBOARD

Layout

Sidebar



Top Bar



Widgets



Charts



Recent Activity

Widgets

Revenue

Orders

Reservations

Inventory

Customers

AI Insights

Quick Actions

15. ORDER MANAGEMENT

Table

Search

Filters

Bulk Actions

Drawer

Timeline

Customer

Invoice

Payment

Kitchen

Print

16. POS SCREEN

Workspace

Categories

↓

Menu

↓

Current Order

↓

Billing

↓

Payment

Keyboard shortcuts.

Touch optimized.

Minimal scrolling.

17. KITCHEN DISPLAY

Large Cards

Order Number

Items

Elapsed Time

Priority

Status

Big typography.

Hands-free usability.

Dark theme.

18. INVENTORY SCREEN

KPI

↓

Filters

↓

Inventory Table

↓

Stock Alerts

↓

History

Actions

Add

Edit

Adjust

Export

19. SUPPLIER SCREEN

Cards

Contact

Purchases

Outstanding

Orders

Performance

Actions

Call

Email

Purchase Order

20. CRM SCREEN

Customer Cards

Segments

Timeline

Orders

Reviews

Coupons

Notes

21. ANALYTICS SCREEN

Charts

Revenue

Orders

Menu

Customers

Peak Hours

Inventory

Filters

Date

Week

Month

Year

Comparison

Export

22. REPORT SCREEN

Generate



Preview



Download



Email



Print

23. SETTINGS SCREEN

Sections

Restaurant

Business Hours

GST

Payments

Delivery

WhatsApp

Printer

Theme

Users

Security

Backups

Each section inside collapsible panels.

24. AI ASSISTANT

Floating assistant.

Chat interface.

Suggested prompts.

Quick actions.

Reports.

Insights.

Search.

Context-aware responses.

25. NOTIFICATION CENTER

Grouped by:

Orders

Inventory

Marketing

Payments

System

Priority colors.

Mark read.

Archive.

Filter.

26. SYSTEM HEALTH

Status cards.

Database.

API.

Redis.

Printer.

WhatsApp.

Payments.

Backups.

Green

Yellow

Red

Health indicators.

27. EMPTY STATES

Every screen has designed empty states.

Examples

No Orders

↓

Illustration



Create First Order

No Customers



Invite Customers

No Reports



Generate Report

Never display blank tables.

28. ERROR STATES

Examples

Network Error



Retry

Payment Failed



Retry Payment

Inventory Conflict



Refresh Data

Printer Offline

↓

Reconnect

Every error includes:

Explanation

Suggested action

Recovery path

29. LOADING STATES

Skeleton UI.

Animated placeholders.

Optimistic updates.

Progress indicators.

Avoid blocking interfaces.

30. RESPONSIVE BEHAVIOR

Desktop

Full experience.

Tablet

Collapsible sidebar.

Mobile

Drawer navigation.

Bottom navigation where appropriate.

Touch targets $\geq 48\text{px}$.

31. ACCESSIBILITY

Keyboard navigation.

Screen readers.

ARIA.

Focus management.

Reduced motion.

High contrast.

Semantic HTML.

32. MICRO INTERACTIONS

Button press.

Hover lift.

Image zoom.

Ripple.

Toast.

Drawer.

Dialog.

Accordion.

Progress.

Success animation.

Confetti only for special achievements (never during ordering).

33. ANALYTICS EVENTS

Track:

Page View.

Button Click.

Search.

Filter.

Checkout.

Reservation.

Order.

Coupon.

Review.

AI Interaction.

34. DESIGN QA CHECKLIST

Every screen must pass:

- Design System compliance.
- Responsive validation.
- Accessibility review.
- Animation review.
- Performance review.
- Empty state review.
- Error state review.
- Loading state review.
- Visual consistency review.

35. USER JOURNEYS

Document complete journeys for:

Customer

- Discover restaurant
- Browse menu
- Customize food
- Checkout
- Track order
- Leave review
- Reorder

Restaurant Staff

- Receive order
- Prepare food
- Update kitchen status
- Print receipt
- Complete order

Manager

- Monitor dashboard
- Review analytics
- Manage inventory
- Launch campaign
- Generate reports

Owner

- Review business performance
- Configure settings
- Manage staff
- Analyze AI insights
- Plan growth

36. DESIGN SYSTEM ENFORCEMENT

Every screen must use:

- Shared typography tokens
- Shared color tokens
- Shared spacing scale
- Shared component library
- Shared animation system
- Shared iconography
- Shared interaction patterns

No screen may introduce custom styling that conflicts with the design system.

37. VISUAL CONSISTENCY RULES

Across every screen:

- Card radius remains consistent.
 - Shadows follow elevation tokens.
 - Buttons use shared variants.
 - Tables share the same layout.
 - Forms behave identically.
 - Status colors are universal.
 - Success, warning, and error messaging are standardized.
-

38. ACCEPTANCE CRITERIA

The UI/UX specification is considered complete when:

- Every screen in the application has a defined purpose and layout.
 - Navigation is consistent across all experiences.
 - Components follow the design system.
 - Loading, empty, and error states are documented.
 - Accessibility and responsiveness are built into every screen.
 - User journeys are optimized for clarity and speed.
 - Developers can implement screens without making design assumptions.
-

FINAL VISION FOR VOLUME 13

The Restaurant Digital Platform should feel like a **single, unified product** regardless of whether someone is ordering food, managing inventory, operating the POS, or reviewing analytics. Every screen should reinforce the restaurant's premium identity while maximizing usability and operational efficiency.

RESTAURANT DIGITAL PLATFORM

MASTER IMPLEMENTATION ROADMAP

VOLUME 14

DEVELOPMENT EXECUTION, PROJECT MANAGEMENT, QA, DEVOPS & PRODUCTION LAUNCH PLAYBOOK

1. INTRODUCTION

This document defines the complete execution strategy for building the Restaurant Digital Platform.

Rather than focusing on features, this document defines:

- Development methodology
- Sprint planning
- Team responsibilities
- Coding standards

- Git workflow
- QA process
- Testing
- Release management
- Deployment
- Monitoring
- Long-term maintenance

This is the document that ensures the project is delivered successfully.

2. PROJECT VISION

The Restaurant Digital Platform should launch as a production-ready system that supports:

- Customer Website
- Online Ordering
- Reservations
- POS
- Kitchen Display System
- Inventory
- CRM
- Billing
- Reports
- AI Assistant
- WhatsApp Integration

without requiring a major rewrite.

3. DEVELOPMENT METHODOLOGY

Recommended methodology:

Agile Scrum

Sprint Length:

2 Weeks

Daily Standup:

15 Minutes

Sprint Planning:

Every Sprint

Sprint Review:

End of Sprint

Sprint Retrospective:

End of Sprint

Backlog Refinement:

Weekly

4. PROJECT PHASES

Phase 1

Foundation

Duration

2 Weeks

Deliverables

Architecture

Authentication

Database

Design System

Navigation

CI/CD

Phase 2

Public Website

Duration

2 Weeks

Features

Landing Page

Menu

Gallery

Reservations

Contact

SEO

Animations

Phase 3

Ordering System

Duration

3 Weeks

Features

Menu

Cart

Checkout

Payments

Order Tracking

Customer Dashboard

Phase 4

Admin Dashboard

Duration

3 Weeks

Features

Dashboard

Orders

Reservations

Customers

Reports

Notifications

Phase 5

POS & Kitchen

Duration

3 Weeks

Features

POS

KDS

Billing

Thermal Printing

GST

Tables

Phase 6

Inventory & CRM

Duration

2 Weeks

Inventory

Suppliers

Purchases

Reviews

Loyalty

Coupons

Phase 7

Analytics & AI

Duration

2 Weeks

Charts

Reports

AI

Automation

Insights

Phase 8

Testing

Optimization

Deployment

Documentation

Production Launch

5. MILESTONES

Milestone 1

Architecture Complete

Milestone 2

Frontend Complete

Milestone 3

Backend Complete

Milestone 4

Integration Complete

Milestone 5

Beta Testing

Milestone 6

Production Launch

6. SPRINT STRUCTURE

Every sprint contains:

Planning

↓

Implementation

↓

Code Review

↓

Testing

↓

Bug Fixes

↓

Documentation

↓

Demo

↓

Release

7. FEATURE DEVELOPMENT WORKFLOW

Backlog



Design Review



Architecture Review



Implementation



Unit Tests



Integration Tests



QA



Merge



Deployment

8. GIT STRATEGY

Main Branches

main

develop

Feature Branches

feature/menu

feature/cart

feature/checkout

feature/pos

feature/inventory

feature/analytics

Bug Fixes

bugfix/payment

bugfix/cart

Hotfix

hotfix/payment-timeout

Release

release/v1.0

Never develop directly on `main`.

9. COMMIT CONVENTIONS

Examples

feat(menu): add menu filtering

fix(cart): resolve quantity bug

refactor(api): optimize order service

docs(readme): update setup guide

style(ui): improve button spacing

test(payment): add refund tests

chore(deps): update dependencies

Commits should remain small and focused.

10. PULL REQUEST CHECKLIST

Every PR must include:

Description

Screenshots

Testing Notes

Linked Issue

Checklist

Reviewer

No PR should exceed ~500 lines unless justified.

11. CODE REVIEW STANDARDS

Review:

Architecture

Security

Performance

Accessibility

Naming

Reusability

Testing

Documentation

Never approve code with duplicated business logic.

12. DEFINITION OF READY

A task is ready when:

Requirements are clear.

Design approved.

API defined.

Acceptance criteria written.

Dependencies resolved.

Effort estimated.

13. DEFINITION OF DONE

A task is complete when:

Feature implemented.

Tests passing.

Documentation updated.

Responsive.

Accessible.

Reviewed.

Merged.

Deployed to staging.

QA approved.

14. TESTING STRATEGY

Levels

Unit Tests

↓

Component Tests

↓

Integration Tests

↓

API Tests

↓

End-to-End Tests

↓

Accessibility Tests

↓

Performance Tests

↓

Regression Tests

15. CRITICAL USER FLOWS

Must always be tested.

Customer:

Browse Menu

Add to Cart

Checkout

Payment

Reservation

Review

Restaurant:

Create Order

Print Receipt

Kitchen Update

Inventory Adjustment

Analytics

Reports

16. BUG SEVERITY

Critical

Payment failure

Data loss

Security issue

System unavailable

High

Checkout failure

Inventory mismatch

Printer failure

Medium

UI inconsistency

Broken filter

Validation issue

Low

Spacing

Typography

Animation

Minor visual defects

17. RELEASE CHECKLIST

Verify:

Environment variables.

Database migrations.

API documentation.

Feature flags.

Monitoring.

Backups.

Printer connectivity.

Payment gateway.

WhatsApp integration.

18. PERFORMANCE BUDGETS

Website

Lighthouse > 95

JavaScript bundle < 300 KB (initial target)

Images optimized.

Dashboard

Load < 2 seconds.

API

Average response < 200 ms.

19. SECURITY CHECKLIST

Validate:

JWT

RBAC

Input validation

Rate limiting

CORS

Helmet

HTTPS

Dependency audit

Secret management

Audit logging

20. ACCESSIBILITY CHECKLIST

Keyboard navigation.

ARIA labels.

Focus states.

Semantic HTML.

Contrast.

Screen reader support.

Reduced motion.

21. DOCUMENTATION REQUIREMENTS

Maintain:

README

Architecture Guide

API Documentation

Deployment Guide

Database Schema

Design System

Environment Variables

Developer Onboarding

22. STAGING ENVIRONMENT

Purpose:

Final validation before production.

Verify:

Payments.

WhatsApp.

Printers.

Reports.

Performance.

Cross-browser compatibility.

23. PRODUCTION DEPLOYMENT

Deployment sequence:

1. Backup database.
2. Deploy backend.
3. Run migrations.
4. Health checks.
5. Deploy frontend.
6. Smoke tests.
7. Enable monitoring.
8. Verify integrations.
9. Announce release.

Rollback plan must be documented before deployment.

24. POST-LAUNCH MONITORING

Monitor:

Orders.

Payments.

API errors.

Database health.

Printer status.

WhatsApp delivery.

User feedback.

Performance.

Track metrics continuously during the first 72 hours.

25. INCIDENT MANAGEMENT

For production incidents:

1. Detect.
 2. Acknowledge.
 3. Assign owner.
 4. Mitigate.
 5. Resolve.
 6. Verify.
 7. Conduct post-incident review.
 8. Document preventive actions.
-

26. PRODUCT BACKLOG (FUTURE)

Potential future features:

- Native Android app.
 - Native iOS app.
 - Customer loyalty engine.
 - AI voice ordering.
 - Multi-language support.
 - Multi-branch management.
 - Delivery partner integration.
 - Kitchen display analytics.
 - Digital signage.
 - Self-service kiosks.
-

27. TEAM ROLES

Product Owner

- Defines priorities.
- Accepts features.

UI/UX Designer

- Maintains design system.

Frontend Engineer

- Builds user interfaces.

Backend Engineer

- Implements APIs and business logic.

QA Engineer

- Validates functionality.

DevOps Engineer

- Maintains infrastructure and deployments.

Project Manager

- Coordinates timelines and communication.
-

28. RISK MANAGEMENT

Potential risks:

- Payment gateway downtime.
- Printer hardware failures.
- Third-party API limits.
- Scope expansion.
- Performance degradation.
- Data migration issues.

Mitigation plans should be prepared for each risk.

29. SUCCESS METRICS

Business:

- Increase online orders.
- Reduce order errors.
- Improve repeat customers.
- Reduce average checkout time.

Technical:

- 99.9% uptime.
- API latency under target.
- Zero critical security vulnerabilities.
- High Lighthouse scores.
- Low error rate.

Operational:

- Faster kitchen turnaround.
- Reduced inventory waste.
- Faster report generation.

- Lower manual effort.
-

30. LONG-TERM ROADMAP

Version 1.0

- Complete restaurant platform.

Version 1.5

- AI enhancements.
- Loyalty improvements.
- Better analytics.

Version 2.0

- Mobile applications.
- Multi-branch support.
- Franchise management.
- Advanced AI forecasting.

Version 3.0

- Voice ordering.
 - Predictive inventory.
 - Dynamic pricing.
 - AI-powered operational optimization.
-

31. PROJECT GOVERNANCE

Establish regular governance meetings:

Weekly:

- Sprint review.
- Blockers.
- Risks.

Monthly:

- Product roadmap review.
- Technical debt review.
- Performance review.

Quarterly:

- Architecture review.
 - Security audit.
 - Disaster recovery drill.
 - Business KPI review.
-

32. KNOWLEDGE MANAGEMENT

Maintain a centralized knowledge base containing:

- Product decisions.
- Architecture Decision Records (ADRs).
- API documentation.
- Database documentation.
- UI guidelines.
- Deployment runbooks.
- Troubleshooting guides.
- Frequently asked questions.

Every significant technical or product decision should be documented.

33. FINAL ACCEPTANCE CRITERIA

The Restaurant Digital Platform is considered production-ready when:

- All planned features are implemented and tested.
- Critical user journeys pass end-to-end validation.
- Performance, accessibility, and security targets are met.
- Documentation is complete and current.
- Monitoring, backups, and incident response procedures are operational.
- The restaurant staff can perform day-to-day operations without assistance.
- The system is maintainable, scalable, and ready for future enhancements.

FINAL VISION

The Restaurant Digital Platform should not be viewed as "a restaurant website."

It should function as a complete digital ecosystem that manages the restaurant's customer experience, operations, finances, analytics, and future growth from a single, cohesive platform.

Every decision—from the first pixel to the final deployment—should reinforce four principles:

- Premium user experience
- Operational efficiency
- Engineering excellence
- Long-term maintainability

RESTAURANT DIGITAL PLATFORM

UI ARCHITECTURE SPECIFICATION

VOLUME 15

COMPLETE SCREEN WIREFRAMES, COMPONENT INVENTORY, VISUAL HIERARCHY & DESIGN IMPLEMENTATION GUIDE

1. INTRODUCTION

This document defines the visual architecture of every screen.

It is not a collection of mockups.

It is the blueprint that designers and developers follow before implementation.

Every screen should have:

- Purpose
- Layout Grid
- Component Placement
- Visual Hierarchy
- Typography Hierarchy
- Animation Rules
- Responsive Rules
- Accessibility Rules
- Loading States
- Empty States
- Error States

Nothing should be left to interpretation.

2. DESIGN LANGUAGE

The platform should feel like a hybrid of

Apple

↓

Linear

↓

Stripe Dashboard

↓

Framer

↓

Awwwards Restaurant Website

↓

Toast POS

↓

Square POS

↓

Notion Simplicity

The feeling should be

Luxury

Minimal

Elegant

Fast

Professional

Trustworthy

Calm

Confident

3. DESIGN PRINCIPLES

Every screen follows

Large whitespace

↓

Clear hierarchy



Strong typography



Minimal colors



Purposeful animation



Content-first layout



Consistent spacing



Premium photography



Accessibility first

4. MASTER GRID SYSTEM

Desktop

1440px Container

12 Columns

80px Margin

24px Gutter

Tablet

8 Columns

24px Margin

Mobile

4 Columns

16px Margin

Never break the grid.

5. SPACING SCALE

4

8

12

16

20

24

32

40

48

56

64

80

96

120

160

Everything aligns to this scale.

6. ELEVATION SYSTEM

Level 0

Background



Level 1

Cards



Level 2

Dialogs



Level 3

Floating Action



Level 4

Modal



Level 5

Notifications

Consistent shadows.

Consistent blur.

7. TYPOGRAPHY SYSTEM

Display XXL

Hero

90px

Display XL

72px

Display L

60px

H1

48px

H2

36px

H3

30px

H4

24px

Body

18px

Small

16px

Caption

14px

Micro

12px

Buttons

16px SemiBold

8. ICONOGRAPHY

Only Lucide Icons.

Consistent stroke width.

Never mix icon packs.

No emojis.

Icons always support:

Hover

Active

Disabled

Loading

Dark Theme

9. BUTTON SYSTEM

Primary

Gold Fill

White Text

Secondary

Glass

Gold Border

Ghost

Transparent

Danger

Red

Success

Green

Sizes

XS

SM

MD

LG

XL

States

Default

Hover

Focus

Pressed

Disabled

Loading

Success

10. INPUT SYSTEM

Every input contains

Label

Placeholder

Helper Text

Validation

Icon

Error Message

Character Count (where applicable)

Support:

Text

Password

Phone

Email

Number

Date

Time

Textarea

Search

OTP

File Upload

11. CARD SYSTEM

Card Types

Menu Card

Analytics Card

Review Card

Inventory Card

Customer Card

Employee Card

Reservation Card

Report Card

Statistic Card

Every card includes

Padding

Radius

Elevation

Hover State

Loading Skeleton

Empty State

12. TABLE SYSTEM

Unified table component.

Supports

Pagination

Search

Sort

Filters

Bulk Selection

Export

Responsive Collapse

Sticky Header

Column Visibility

Density Modes

13. MODAL SYSTEM

Every modal includes

Header

Body

Footer

Primary Action

Secondary Action

Keyboard Shortcuts

Escape Close

Focus Trap

Loading State

14. DRAWER SYSTEM

Use drawers for

Order Details

Customer Details

Reservation

Inventory Item

Notifications

Avoid unnecessary full-page navigation.

15. SIDEBAR DESIGN

Desktop

Expanded



Collapsed



Floating

Sections

Dashboard

Orders

POS

Kitchen

Inventory

CRM

Analytics

Reports

Settings

Support

Bottom

Profile

Theme

Logout

16. NAVBAR

Restaurant Logo



Navigation



Search



Notifications



Profile

Sticky.

Glass effect.

Blur.

17. DASHBOARD WIDGETS

Each widget contains

Title

Value

Trend

Comparison

Mini Chart

Action

Tooltip

Widgets

Revenue

Orders

Customers

Reservations

Inventory

Reviews

AI Insights

18. MENU CARD

Layout

Image



Badges



Title



Description



Rating



Price



Add Button

Hover

Scale

Image Zoom

Glow

19. ORDER CARD

Order Number



Customer



Items



Status



Time



Amount



Actions

Status Color

Animated Progress

20. RESERVATION CARD

Guest



Time



Table



Status



Contact



Quick Actions

21. CUSTOMER PROFILE

Avatar



Name



Contact



Statistics



Orders



Reservations



Reviews



Notes



Activity Timeline

22. ANALYTICS DASHBOARD

Hero KPIs



Revenue Charts



Orders



Customers



Inventory



Peak Hours



Best Sellers



AI Insights



Export

23. INVENTORY SCREEN

Header



KPIs



Filters



Inventory Table



Low Stock



Recent Activity

24. POS SCREEN

Three-column layout

Left

Categories

Center

Menu Grid

Right

Current Order

Bottom

Payment Summary

Large touch targets.

Keyboard shortcuts.

Optimized for rapid interaction.

25. KDS SCREEN

Dark theme only.

Columns:

Pending

Preparing

Ready

Each order card shows:

Order Number

Items

Modifiers

Elapsed Time

Priority

Quick status actions.

Designed for visibility from a distance.

26. ORDER TRACKING SCREEN

Progress Timeline



Current Status



Estimated Time



Order Summary



Support CTA

Real-time updates.

27. GALLERY

Masonry Grid



Hover Zoom



Lightbox



Keyboard Navigation



Swipe on Mobile

28. LOADING EXPERIENCE

Use Skeleton Components.

Never use blank white screens.

Examples:

Menu Skeleton

Order Skeleton

Dashboard Skeleton

Table Skeleton

Gallery Skeleton

Chart Skeleton

29. EMPTY STATES

Every empty state contains:

Illustration

Headline

Description

Primary Action

Secondary Action (optional)

Examples:

No Orders
No Reservations
No Customers
No Inventory
No Reports
No Notifications

30. ERROR STATES

Every error includes:

Error icon
Title
Explanation
Retry button
Support link (if needed)

Examples:

Payment failed
Printer offline
Network unavailable
Inventory conflict
Session expired

31. RESPONSIVE BLUEPRINT

Desktop

1440+

Tablet

1024

Mobile

768

Small Mobile

480

Ultra Small

360

Every component must specify:

Visibility

Stacking

Spacing

Navigation behavior

Touch targets

32. ANIMATION SPECIFICATION

Landing Page

GSAP

Text reveal

Parallax

Scroll storytelling

Image masking

Admin

Framer Motion

Fade

Slide

Scale

Micro interactions

Duration:

200–400 ms

Easing:

Natural

Avoid excessive animation in operational screens.

33. DESIGN TOKENS

Define centralized tokens for:

Colors

Typography

Spacing

Radius

Elevation

Borders

Opacity

Blur

Animation

Breakpoints

Z-index

All components consume these tokens.

34. COMPONENT INVENTORY

Core Components:

Buttons

Inputs

Selects

Tables

Cards

Dialogs

Drawers

Badges

Tooltips

Dropdowns

Tabs

Accordions

Breadcrumbs

Pagination

Search Bars

Charts

Carousels

Forms

Date Pickers

Time Pickers

File Uploaders

QR Components

Status Indicators

KPI Cards

Notification Items

Timeline

Progress Bars

Skeletons

Every component must include:

- Variants
- Sizes
- States
- Accessibility behavior
- Responsive behavior
- Usage guidelines

35. SCREEN INVENTORY

Public Website:

- Home
- About
- Menu
- Menu Detail
- Gallery
- Reservation

- Contact
- Cart
- Checkout
- Order Tracking
- Login
- Register
- Customer Dashboard

Operations:

- Admin Dashboard
- Orders
- Order Detail
- POS
- Kitchen Display
- Inventory
- Suppliers
- Customers
- Reviews
- Coupons
- Reports
- Analytics
- Notifications
- AI Assistant
- Settings
- System Health
- Audit Logs

Each screen should have a dedicated specification document before implementation.

36. VISUAL QA CHECKLIST

Before approving any screen:

- Grid alignment verified.
- Spacing follows design tokens.
- Typography hierarchy is correct.
- Color usage matches the system.
- Components use shared variants.
- Loading, empty, and error states exist.
- Responsive layouts tested.

- Accessibility reviewed.
 - Animation performance verified.
-

37. FIGMA ORGANIZATION

Structure the design file as:

01 Foundations

02 Design Tokens

03 Components

04 Patterns

05 Public Website

06 Customer Dashboard

07 Admin Dashboard

08 POS

09 Kitchen Display

10 Analytics

11 CRM

12 Settings

13 Prototypes

14 Assets

15 Developer Handoff

Use Auto Layout, Variables, Components, Component Properties, and Design Tokens consistently.

38. DEVELOPER HANDOFF

Each screen should include:

- Component list
- Token references
- Responsive behavior
- Animation notes
- API dependencies
- States (loading/empty/error)
- Accessibility notes
- Interaction specifications

Developers should not need to infer UI behavior.

39. ACCEPTANCE CRITERIA

The UI Architecture is complete when:

- Every screen has a documented layout.
 - Every component exists in the shared library.
 - Design tokens are used consistently.
 - Responsive behavior is defined.
 - Accessibility is documented.
 - Animation behavior is specified.
 - Developer handoff artifacts are complete.
 - The interface feels cohesive across the public website, POS, KDS, and admin dashboard.
-

40. FINAL VISION

The Restaurant Digital Platform should feel like a product designed from a **single design language**, not a collection of unrelated screens.

Customers should experience a premium, memorable restaurant website.

Staff should experience an efficient, distraction-free operational interface.

Managers should experience clear, actionable business insights.

Every screen should reinforce the same visual identity, interaction patterns, and engineering standards established throughout the platform.

RESTAURANT DIGITAL PLATFORM

SOFTWARE ARCHITECTURE DOCUMENT (SAD)

VOLUME 16

ENTERPRISE ARCHITECTURE BLUEPRINT, UML, C4 MODEL, SEQUENCE DIAGRAMS & ENGINEERING SPECIFICATIONS

1. INTRODUCTION

This document formally defines the software architecture of the Restaurant Digital Platform.

It establishes:

- System boundaries
- Components
- Communication
- Data flow
- Object relationships

- Runtime behavior
- Deployment topology
- Architectural decisions

This document should allow any experienced engineering team to understand how the platform operates without reading source code.

2. ARCHITECTURAL PRINCIPLES

The platform must follow these principles:

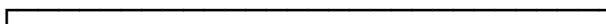
- Modular Architecture
 - Domain Driven Design (DDD)
 - Clean Architecture
 - SOLID Principles
 - API First
 - Event Driven Communication
 - Feature Based Frontend
 - Repository Pattern
 - Service Layer Pattern
 - Dependency Injection
 - Separation of Concerns
 - Single Responsibility
 - High Cohesion
 - Low Coupling
-

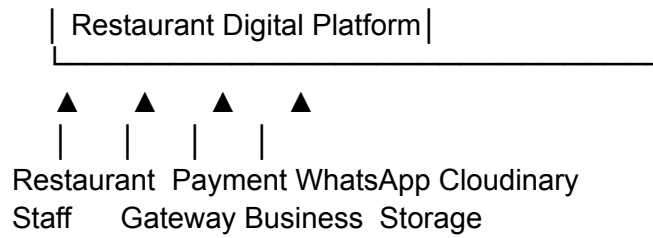
3. C4 MODEL

The architecture should be documented using the complete C4 methodology.

Level 1 — System Context

CUSTOMER





External systems:

- Payment Gateway
- WhatsApp Business API
- Email Provider
- Cloudinary
- Google Maps
- Google Analytics
- Restaurant Staff
- Customers

Level 2 — Container Diagram

Frontend (React)



API Server (Express)



Application Services



Repositories



MongoDB Atlas



Redis



Background Workers



Third Party APIs

Containers:

- Public Website
- Admin Dashboard
- POS
- Kitchen Display
- Backend API
- WebSocket Server
- Worker Service
- MongoDB
- Redis
- Object Storage

Level 3 — Component Diagram

Backend

Authentication



Menu



Orders



Kitchen



Inventory



Payments



CRM



Analytics



Reports



AI



Notifications



Settings

Each component owns:

Controller



Service



Repository



Model



Validator

↓

DTO

↓

Mapper

Level 4 — Code Level

Document every class.

Every interface.

Every dependency.

Every inheritance.

Every composition.

4. UML CLASS DIAGRAMS

Every domain should have class diagrams.

Example:

Customer

Customer

id

name

phone

email

addresses

orders

loyaltyPoints

reviews

reservations

createOrder()

reserveTable()

leaveReview()

Order

Order

orderNumber

customer

items

payment

invoice

status

timeline

confirm()

cancel()

complete()

refund()

Menu Item

MenuItem

name

price

variants

ingredients

category

availability

rating

calculatePrice()

applyDiscount()

checkAvailability()

5. ENTITY RELATIONSHIP DIAGRAM (ERD)

Customer

↓

Orders



OrderItems



MenuItem



Ingredient



Inventory



Supplier



PurchaseOrder

Also define relationships for:

Reservations

Payments

Invoices

Reviews

Employees

Roles

Notifications

Audit Logs

Settings

6. SEQUENCE DIAGRAMS

Every critical workflow should have a sequence diagram.

Customer Order

Customer



Website



API



Order Service



Inventory



Payment



Invoice



Kitchen



WhatsApp

↓

Analytics

Reservation

Customer

↓

Website

↓

Reservation Service

↓

Table Allocation

↓

Database

↓

Notification

↓

Confirmation

POS Order

Cashier

↓

POS

↓

API



Kitchen



Printer



Invoice



Customer

Inventory Update

Supplier



Inventory



Purchase Order



Stock Update



Analytics



Reports

7. ACTIVITY DIAGRAMS

Document activities for:

Customer Checkout

Reservation

Order Cancellation

Refund

Inventory Purchase

Menu Update

Staff Login

Report Generation

AI Recommendation

8. STATE MACHINES

Every entity has defined states.

Order

Draft

↓

Pending

↓

Confirmed

↓

Preparing

↓

Ready

↓

Completed

Reservation

Pending

↓

Confirmed

↓

Checked In

↓

Completed

Payment

Pending

↓

Authorized

↓

Paid

↓

Refunded

Inventory

Available

↓

Reserved



Consumed

9. COMPONENT DIAGRAMS

Each module should expose interfaces only.

Example

Order Module

Controller



Service



Repository



MongoDB

Never bypass layers.

10. PACKAGE DIAGRAM

restaurant/



auth/

menu/

orders/

inventory/

reports/

analytics/

crm/

notifications/

ai/

shared/

core/

Dependencies only move downward.

No circular dependencies.

11. DEPLOYMENT DIAGRAM

User



Cloudflare



Vercel



Express API



MongoDB Atlas



Redis



Queue Workers



Cloundinary

Future

Docker



Kubernetes



Auto Scaling

12. DATA FLOW DIAGRAM (DFD)

Customer



Website



API



Validation



Business Logic



Database



Analytics



AI



Dashboard

13. EVENT STORMING

Important events:

CustomerRegistered

OrderCreated

PaymentSucceeded

OrderCancelled

KitchenStarted

KitchenReady

InventoryUpdated

ReservationConfirmed

CouponRedeemed

ReviewSubmitted

AIInsightGenerated

Every event documents:

Producer

Consumers

Payload

Side Effects

14. BPMN PROCESS FLOWS

Business processes:

Online Ordering

Walk-in Ordering

Reservation

Inventory Purchasing

Refund Processing

Daily Closing

Shift Management

Supplier Management

Report Generation

Each process includes:

Actors

Activities

Gateways

Events

Timers

Exceptions

15. ARCHITECTURE DECISION RECORDS (ADR)

Maintain ADRs for significant decisions.

Examples:

ADR-001: Choose MongoDB over PostgreSQL.

ADR-002: Use React + Vite instead of Next.js.

ADR-003: Adopt Domain-Driven Design.

ADR-004: Event-driven internal architecture.

ADR-005: TanStack Query for server state.

ADR-006: Cloudinary for media.

Each ADR contains:

- Context
 - Decision
 - Alternatives
 - Consequences
 - Status
 - Date
-

16. INTERFACE CONTRACTS

Document every interface.

Example:

OrderService

Methods:

- createOrder()
- updateStatus()
- cancelOrder()
- completeOrder()
- refundOrder()

Each interface includes:

Inputs

Outputs

Exceptions

Side effects

17. DEPENDENCY GRAPH

Map dependencies between modules.

Allowed:

Website → API

API → Service

Service → Repository

Repository → Database

Not allowed:

Component → Database

Controller → Database

Frontend → Database

Repository → Controller

18. DOMAIN BOUNDARIES

Clearly separate domains.

Authentication

Menu

Orders

Kitchen

Inventory

Billing

CRM

Analytics

AI

Notifications

Settings

Domains communicate only through published interfaces and events.

19. NON-FUNCTIONAL REQUIREMENTS (NFR)

Availability

99.9%

Performance

<200 ms average API response

Scalability

10× projected restaurant growth

Security

OWASP Top 10 mitigations

Reliability

Graceful degradation

Maintainability

Modular architecture

Observability

Centralized logging, metrics, and tracing

20. CAPACITY PLANNING

Design for:

- Thousands of orders per day
- Hundreds of concurrent users
- Multiple POS terminals
- Multiple kitchen displays
- High-resolution media assets
- Years of historical analytics

Document expected storage, bandwidth, and compute growth.

21. FAILURE MODE ANALYSIS

For each critical subsystem:

Potential failure

Impact

Detection

Mitigation

Recovery

Examples:

Payment gateway unavailable

Printer offline

Database connection loss

Redis outage

WhatsApp API failure

Worker queue backlog

22. SECURITY ARCHITECTURE

Define:

Trust boundaries

Authentication flow

Authorization model

Encryption

Secret management

Key rotation

API security

Audit strategy

Threat model

Session management

23. OBSERVABILITY ARCHITECTURE

Logs

Metrics

Distributed tracing

Health checks

Dashboards

Alerts

Incident management

Correlation IDs

Service-level objectives (SLOs)

24. DOCUMENTATION STANDARDS

Every module must include:

Overview

Responsibilities

Dependencies

Public interfaces

Configuration

Examples

Testing notes

Known limitations

Future enhancements

25. ENGINEERING ACCEPTANCE CRITERIA

The architecture is considered complete when:

- C4 documentation exists for all levels.
 - UML diagrams cover every major domain.
 - Sequence diagrams exist for critical workflows.
 - ERDs accurately represent persistence.
 - Domain boundaries are clearly defined.
 - Interfaces and dependencies are documented.
 - Architectural decisions are recorded.
 - Non-functional requirements are measurable.
 - The architecture supports future evolution without major redesign.
-

FINAL VISION

The Restaurant Digital Platform should be engineered like a long-lived enterprise system rather than a one-off website.

Every architectural decision should improve clarity, scalability, resilience, and maintainability.

The documentation should allow a new engineer to understand the platform from the highest-level business context down to individual classes and interactions without relying on tribal knowledge.

RESTAURANT DIGITAL PLATFORM

ENTERPRISE INFRASTRUCTURE & OPERATIONS MANUAL

VOLUME 17

DEVOPS, CLOUD ARCHITECTURE, INFRASTRUCTURE AS CODE, CI/CD, SECURITY & PRODUCTION OPERATIONS

1. INTRODUCTION

The Restaurant Digital Platform must be deployed using modern cloud-native practices that prioritize:

- Reliability
- Security
- Performance
- Observability
- Disaster Recovery
- Scalability
- Maintainability
- Cost Efficiency

The infrastructure should support a single restaurant today while being architecturally prepared for multi-branch expansion in the future.

2. INFRASTRUCTURE PHILOSOPHY

Infrastructure should be:

Infrastructure as Code



Immutable



Automated



Observable



Secure



Recoverable

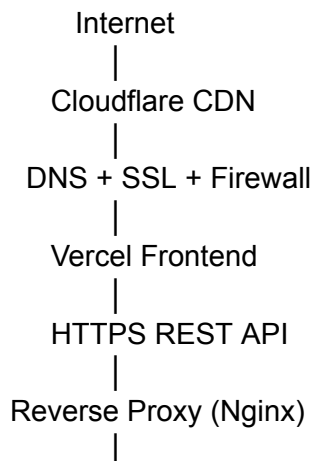


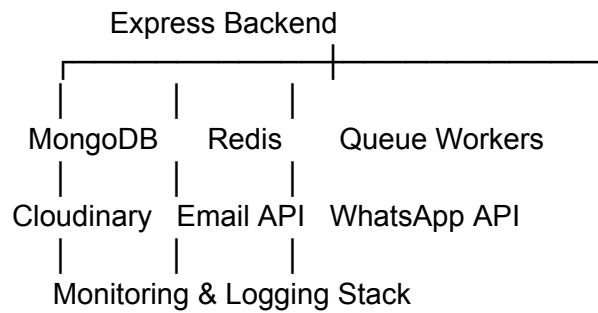
Documented

Manual production changes are prohibited.

Everything should be reproducible from version-controlled configuration.

3. PRODUCTION INFRASTRUCTURE





4. CLOUD PROVIDERS

Recommended Architecture

Frontend

Vercel

Backend

Railway / Render

Production Upgrade

AWS ECS / GCP Cloud Run / Azure Container Apps

Database

MongoDB Atlas

Cache

Redis Cloud

Media

Cloudinary

DNS

Cloudflare

Email

Resend

Monitoring

Grafana + Prometheus

Error Tracking

Sentry

5. ENVIRONMENT STRATEGY

Maintain isolated environments.

Development

Local development

↓

Testing

Automated QA

↓

Staging

Pre-production validation

↓

Production

Live restaurant operations

Each environment has:

Separate database

Separate secrets

Separate storage

Separate API keys

Separate domains

6. DOMAIN ARCHITECTURE

Examples

restaurant.com

www.restaurant.com

admin.restaurant.com

api.restaurant.com

status.restaurant.com

docs.restaurant.com

Future

branch1.restaurant.com

branch2.restaurant.com

7. DNS MANAGEMENT

Cloudflare should manage:

DNS

SSL

CDN

Caching

Firewall

DDoS Protection

Always enable HTTPS.

8. SSL STRATEGY

HTTPS everywhere.

TLS 1.3 preferred.

Automatic certificate renewal.

HSTS enabled.

Redirect HTTP → HTTPS.

9. REVERSE PROXY

Nginx responsibilities

HTTPS termination

Compression

Static asset caching

Security headers

API routing

Rate limiting

WebSocket forwarding

Health checks

10. DOCKER ARCHITECTURE

Every service should be containerized.

Containers

Frontend

Backend

Worker

Redis

Nginx

Future

AI Worker

Notification Worker

Analytics Worker

11. DOCKER COMPOSE

Development stack should include:

Frontend

Backend

MongoDB

Redis

Mail testing service

Nginx

Volume mounts

Environment variables

Health checks

12. INFRASTRUCTURE AS CODE

Use Terraform (preferred) or Pulumi.

Manage:

Cloud resources

DNS

Secrets references

Storage

Networking

Monitoring

Infrastructure should be reproducible.

13. SECRET MANAGEMENT

Never commit secrets.

Store:

Database URI

JWT Secret

API Keys

Payment Keys

WhatsApp Tokens

Cloudinary Keys

SMTP Keys

Use environment variables or a managed secret service.

Rotate secrets periodically.

14. CI/CD PIPELINE

Pipeline

Git Push

↓

Install

↓

Lint

↓

Type Check

↓

Unit Tests

↓

Integration Tests

↓

Security Scan

↓

Build

↓

Deploy Preview

↓

Approval



Production Deploy



Health Check



Release

15. BRANCH DEPLOYMENTS

main



Production

develop



Staging

feature/*



Preview Deployments

Each Pull Request generates an isolated preview.

16. DATABASE MIGRATIONS

Every schema change requires:

Migration script

Rollback script

Verification

Backup

Never edit production data manually.

17. DATABASE BACKUPS

Schedule:

Hourly (critical data)

Daily Incremental

Weekly Full

Monthly Archive

Retention:

7 days

30 days

90 days

1 year

Test restoration quarterly.

18. MEDIA STORAGE

Store:

Food Images

Restaurant Photos

Invoices

Reports

Customer Uploads

Media pipeline:

Upload

↓

Validation

↓

Compression

↓

Optimization

↓

Clouinary

↓

CDN

↓

Metadata in MongoDB

19. MONITORING STACK

Metrics

Prometheus

Visualization

Grafana

Error Tracking

Sentry

Uptime

UptimeRobot / Better Stack

Centralized Logging

OpenSearch / Loki

20. APPLICATION METRICS

Monitor:

Orders per hour

Revenue

API latency

Memory usage

CPU

Database response time

Queue size

Printer connectivity

Payment success rate

WhatsApp delivery rate

Cache hit ratio

21. DISTRIBUTED TRACING

Trace every request.

Customer



Frontend



API



Database



Queue



WhatsApp



Response

Use correlation IDs throughout.

22. ALERTING

Notify administrators when:

API unavailable

Database unavailable

Redis unavailable

Payment failures spike

Queue backlog grows

Printer offline

High error rate

Low disk space

Backup failure

SSL expiry approaching

Alerts should include severity and recommended actions.

23. LOG MANAGEMENT

Separate logs:

Application

Authentication

Orders

Payments

Inventory

AI

Notifications

Workers

Security

Infrastructure

Retention policies should be configurable.

24. PERFORMANCE OPTIMIZATION

Frontend

Code splitting

Lazy loading

Image optimization

CDN

Backend

Connection pooling

Compression

Caching

Efficient queries

Database

Indexes

Aggregation optimization

Read-heavy optimization

25. SECURITY OPERATIONS

Security checklist:

OWASP Top 10 mitigation

Dependency scanning

Container image scanning

Secret scanning

Rate limiting

Bot protection

File upload validation

Audit logging

Periodic penetration testing

26. FIREWALL & NETWORKING

Allow only required ports.

Restrict administrative access.

IP allow-list for sensitive endpoints where practical.

Protect admin interfaces.

Segment internal services from public traffic.

27. BUSINESS CONTINUITY

Prepare for:

Internet outage

Payment provider outage

Printer failure

Database failure

Cloud outage

Staff device failure

Document manual fallback procedures.

28. DISASTER RECOVERY

Recovery objectives:

RPO (Recovery Point Objective)

≤ 15 minutes

RTO (Recovery Time Objective)

≤ 1 hour

Document restoration steps for every critical service.

29. RELEASE MANAGEMENT

Each release includes:

Release notes

Migration plan

Rollback plan

Risk assessment

Smoke tests

Approval checklist

Production verification

Tag releases semantically (e.g., v1.2.0).

30. MAINTENANCE WINDOWS

Schedule planned maintenance.

Notify stakeholders.

Display maintenance banners if required.

Verify backups before maintenance.

Validate system health after completion.

31. CAPACITY PLANNING

Forecast growth for:

Orders

Storage

Media

Database size

Concurrent users

API throughput

Plan infrastructure upgrades before limits are reached.

32. COST OPTIMIZATION

Monitor:

Cloud compute costs

Database usage

Storage costs

CDN bandwidth

API usage

Email volume

WhatsApp messaging costs

Use dashboards to identify optimization opportunities.

33. COMPLIANCE

Maintain:

Audit logs

Invoice retention

Tax records

Access logs

Security documentation

Privacy policy

Terms of service

Ensure compliance with applicable regulations and local tax requirements.

34. OPERATIONS RUNBOOKS

Create runbooks for:

Deployment

Rollback

Printer troubleshooting

Payment gateway issues

Database restoration

Queue failures

Redis failures

SSL renewal

API outages

High CPU usage

Each runbook should include step-by-step recovery procedures.

35. INCIDENT RESPONSE

Incident lifecycle:

Detect



Classify



Assign



Mitigate



Resolve



Verify



Postmortem



Prevent recurrence

Document every significant incident.

36. DEVOPS DASHBOARD

Unified dashboard should display:

Infrastructure health

API status

Database health

Redis status

Queue status

Worker health

Deployment history

Recent incidents

Error trends

System capacity

37. AUTOMATED HEALTH CHECKS

Endpoints:

/health

/health/live

/health/ready

/health/dependencies

/metrics

Checks:

Database

Redis

Storage

Queue

Third-party APIs

Disk

Memory

CPU

38. ACCEPTANCE CRITERIA

The infrastructure is considered production-ready when:

- All services are containerized.
 - Infrastructure is reproducible through Infrastructure as Code.
 - CI/CD supports automated testing and safe deployments.
 - Monitoring, logging, and alerting are operational.
 - Secrets are securely managed.
 - Backup and disaster recovery procedures are tested.
 - Performance and security standards are met.
 - Operations runbooks are documented and accessible.
 - The platform can recover gracefully from common failure scenarios.
-

FINAL VISION

The Restaurant Digital Platform should operate with the reliability of a professional SaaS product, not just a restaurant website.

Infrastructure should be treated as a first-class part of the product. Every deployment, backup, alert, and recovery procedure should be automated wherever possible, ensuring that restaurant operations remain uninterrupted even under adverse conditions.

RESTAURANT DIGITAL PLATFORM

ENTERPRISE QUALITY ENGINEERING MANUAL

VOLUME 18

QUALITY ASSURANCE, TESTING STRATEGY, VALIDATION FRAMEWORK & RELEASE CERTIFICATION

1. INTRODUCTION

The Restaurant Digital Platform must maintain enterprise-grade quality standards.

Quality is not achieved through testing alone.

Quality begins during:

Requirements

↓

Architecture

↓

Development



Code Review



Testing



Deployment



Production Monitoring

Every phase contributes to software quality.

2. QUALITY PHILOSOPHY

Quality Principles

Build Quality In



Test Early



Automate Everything



Measure Everything



Prevent Bugs



Continuous Improvement

Testing should detect issues.

Engineering should prevent them.

3. QUALITY OBJECTIVES

The platform should achieve:

Reliability

Security

Performance

Accessibility

Usability

Maintainability

Scalability

Recoverability

Consistency

4. TEST PYRAMID

Manual Exploratory



End-to-End Tests



Integration Tests



Unit Tests

Goal

70%

Unit

20%

Integration

10%

End-to-End

5. TESTING LAYERS

Level 1

Unit Testing



Level 2

Component Testing



Level 3

API Testing



Level 4

Integration Testing



Level 5

End-to-End Testing



Level 6

Performance Testing



Level 7

Security Testing



Level 8

Accessibility Testing



Level 9

User Acceptance Testing

6. UNIT TESTING

Every:

Utility

Helper

Service

Hook

Validation

Business Rule

Calculation

should have unit tests.

Coverage Target

90%+

Examples

GST Calculation

Order Total

Coupon Validation

Inventory Deduction

Price Calculation

AI Recommendation Logic

7. COMPONENT TESTING

Every reusable component must be tested.

Examples

Buttons

Cards

Tables

Dialogs

Forms

Dropdowns

Charts

Sidebar

Navbar

Menu Cards

Test

Rendering

Props

Interactions

States

Accessibility

8. API TESTING

Validate every endpoint.

Check

Authentication

Authorization

Validation

Response

Pagination

Filtering

Sorting

Error Responses

Headers

Rate Limiting

9. INTEGRATION TESTING

Validate interaction between modules.

Examples

Order



Inventory



Invoice



Analytics

Customer



Reservation



Notification

Payment



Invoice



CRM

No isolated testing only.

Test workflows.

10. END-TO-END TESTING

Critical journeys.

Customer

Home



Menu



Cart



Checkout



Payment



Order Tracking



Review

Restaurant

POS



Kitchen



Printer



Invoice



Analytics

Manager

Dashboard



Inventory



Reports



Settings

11. CROSS-BROWSER TESTING

Support

Chrome

Edge

Firefox

Safari

Mobile Browsers

Test

Layouts

Animations

Payments

Forms

12. RESPONSIVE TESTING

Devices

360

390

414

768

1024

1280

1440

1600

Verify

Navigation

Spacing

Tables

Forms

Images

Dialogs

13. ACCESSIBILITY TESTING

WCAG AA

Keyboard

Screen Reader

Focus

ARIA

Contrast

Reduced Motion

Semantic HTML

Every release passes accessibility review.

14. PERFORMANCE TESTING

Measure

LCP

CLS

INP

TTFB

Bundle Size

API Latency

Database Queries

Queue Latency

Rendering Time

Targets

Lighthouse

95+

15. LOAD TESTING

Simulate

100 users

250 users

500 users

1000 users

Peak Dinner Rush

Verify

Response Time

Error Rate

CPU

Memory

Database

Queues

16. STRESS TESTING

Continue increasing traffic until failure.

Identify

Breaking Point

Recovery

Bottlenecks

Memory Leaks

Thread Issues

17. SOAK TESTING

Run system continuously.

24 Hours

48 Hours

72 Hours

Detect

Memory leaks

Performance degradation

Queue growth

Resource exhaustion

18. SECURITY TESTING

Validate

Authentication

Authorization

SQL/NoSQL Injection

XSS

CSRF

SSRF

Rate Limiting

JWT

Secrets

File Upload

OWASP Top 10

19. PENETRATION TESTING

Perform manual security review.

Attempt

Privilege escalation

Broken authentication

Sensitive data exposure

Business logic abuse

IDOR

Session fixation

API abuse

20. DATABASE TESTING

Validate

Indexes

Transactions

Rollback

Aggregation

Concurrency

Backups

Restoration

Migration

21. PAYMENT TESTING

Test

Success

Failure

Duplicate payment

Webhook retry

Refund

Timeout

Cancelled payment

Network interruption

Every payment scenario documented.

22. PRINTER TESTING

Validate

Receipt

Kitchen Ticket

Invoice

Retry

Offline

Reconnect

Multiple printers

Queue ordering

23. WHATSAPP TESTING

Verify

Order Confirmation

Reservation Confirmation

Invoice Delivery

Campaign

Failure Retry

Invalid Number

API Timeout

24. AI TESTING

Validate

Prompt handling

Recommendations

Reports

Response time

Fallback behavior

Hallucination handling

Permission enforcement

AI should never modify data without authorization.

25. USABILITY TESTING

Observe real users.

Tasks

Order food

Reserve table

Update inventory

Generate report

Measure

Completion time

Errors

Confusion

Satisfaction

26. TEST DATA MANAGEMENT

Create datasets for

Customers

Orders

Payments

Inventory

Reservations

Employees

Coupons

Reviews

Separate:

Development

Testing

Production

Never use production data in test environments.

27. REGRESSION TESTING

Before every release.

Verify

Ordering

Payments

Inventory

Dashboard

Reservations

POS

Reports

Authentication

Printing

28. BUG MANAGEMENT

Workflow

Reported

↓

Verified

↓

Prioritized

↓

Assigned

↓

Fixed

↓

Retested

↓

Closed

Every bug linked to:

Version

Module

Severity

Root Cause

29. DEFECT SEVERITY

Critical

Data loss

Payment failure

Security

System crash

High

Checkout broken

Inventory corruption

POS unavailable

Medium

UI issue

Validation

Reports

Low

Typography

Spacing

Animation

Minor alignment

30. ROOT CAUSE ANALYSIS

Every critical issue requires:

Problem

Impact

Timeline

Root Cause

Resolution

Prevention

Owner

Due Date

31. TEST AUTOMATION

Automate

Unit Tests

API Tests

Component Tests

Regression

Smoke Tests

Accessibility

Performance Checks

CI/CD Validation

Manual testing only for exploratory scenarios.

32. RELEASE VALIDATION

Before production

Smoke Tests



Regression



Performance



Security



Accessibility



Approval



Deployment



Monitoring

33. USER ACCEPTANCE TESTING (UAT)

Restaurant staff should validate:

Order flow

POS

Kitchen

Reservations

Inventory

Reports

Printing

WhatsApp

AI

Collect sign-off before launch.

34. TEST DOCUMENTATION

Maintain

Test Plan

Test Cases

Test Scripts

Bug Reports

Execution Logs

Coverage Reports

Regression Suite

Release Checklist

35. QUALITY METRICS

Track

Test Coverage

Pass Rate

Bug Density

Critical Bugs

Mean Time to Detect

Mean Time to Repair

Escaped Defects

Automation Coverage

Release Success Rate

36. RELEASE CERTIFICATION

Every release requires approval from:

Engineering Lead

QA Lead

Product Owner

Security Review (for major releases)

Deployment Owner

No production deployment without certification.

37. CONTINUOUS QUALITY

Quality does not end after release.

Monitor:

Production errors

User feedback

Crash reports

Performance

Security events

Business KPIs

Feed insights back into the backlog.

38. ACCEPTANCE CRITERIA

The quality engineering framework is complete when:

- All critical user journeys are automated.
 - Test coverage targets are met.
 - Performance, accessibility, and security requirements are validated.
 - Regression testing is repeatable.
 - UAT is completed with stakeholder approval.
 - Releases follow a documented certification process.
 - Quality metrics are monitored continuously after deployment.
-

FINAL VISION

The Restaurant Digital Platform should achieve **predictable quality** rather than relying on last-minute testing.

Every feature should be designed, implemented, verified, and monitored using a disciplined quality engineering process so that restaurant staff and customers can rely on the platform during daily operations.

RESTAURANT DIGITAL PLATFORM

OPERATIONS MANUAL & BUSINESS PROCESS DOCUMENTATION

VOLUME 19

STAFF TRAINING, STANDARD OPERATING PROCEDURES (SOP), BUSINESS OPERATIONS & DIGITAL TRANSFORMATION GUIDE

1. INTRODUCTION

This document defines how the restaurant should operate using the Restaurant Digital Platform.

The software is only one part of the business.

The real objective is to improve:

- Customer Experience
- Kitchen Efficiency
- Staff Productivity
- Inventory Accuracy
- Financial Transparency
- Decision Making

Every employee should know exactly what to do.

2. BUSINESS PHILOSOPHY

The platform exists to make restaurant operations:

Simple



Fast



Reliable



Measurable



Scalable

Technology should reduce manual work, not create more.

3. USER ROLES

Restaurant Owner



Restaurant Manager



Cashier



Kitchen Supervisor



Chef

↓

Kitchen Staff

↓

Inventory Manager

↓

Marketing Executive

↓

Customer Support

Each role has clearly defined responsibilities.

4. OWNER RESPONSIBILITIES

Daily

Revenue Review

↓

Orders

↓

Expenses

↓

Inventory

↓

Customer Feedback

Weekly

Marketing

↓

Reports

↓

Growth Review

Monthly

Profit Analysis

↓

Inventory Audit

↓

Staff Performance

↓

Financial Reports

5. MANAGER RESPONSIBILITIES

Open Restaurant

↓

Verify Staff Attendance

↓

Check Inventory

↓

Monitor Orders

↓

Handle Complaints

↓

Close Daily Operations

Manager Dashboard becomes the operational control center.

6. CASHIER SOP

Morning

Login

↓

Printer Check

↓

POS Check

↓

Payment Check

↓

Cash Drawer Count

During Shift

Take Orders

↓

Accept Payment

↓

Print Receipt

↓

Send Order to Kitchen

↓

Handle Refunds

End Shift

Cash Reconciliation

↓

Sales Report

↓

Logout

7. CHEF SOP

Kitchen Display

↓

Accept Order

↓

Prepare Food

↓

Mark Ready

↓

Notify Cashier

↓

Quality Check

↓

Serve

Kitchen should never depend on paper.

8. INVENTORY MANAGER SOP

Morning

Check Low Stock

↓

Receive Deliveries

↓

Update Inventory

↓

Verify Expiry

↓

Place Purchase Orders

Evening

Waste Recording

↓

Closing Stock

↓

Generate Inventory Report

9. CUSTOMER SUPPORT SOP

Receive Complaint



Identify Order



Review Timeline



Resolve Issue



Offer Compensation (if approved)



Close Ticket

Document every customer interaction.

10. OPENING CHECKLIST

Restaurant Clean

POS Online

Printers Connected

Kitchen Display Online

Internet Working

Inventory Checked

Tables Ready

Payments Active

Music

Lighting

AC

Everything verified before opening.

11. CLOSING CHECKLIST

Last Order Complete

Payments Reconciled

Cash Count

Inventory Updated

Waste Recorded

Reports Generated

Backup Verified

Logout

Lock System

12. ORDER PROCESS

Customer

↓

Menu

↓

Cart

↓

Payment

↓

Kitchen

↓

Preparation

↓

Ready

↓

Serve

↓

Review

Standardize every step.

13. WALK-IN CUSTOMER FLOW

Customer Arrives

↓

Table Allocation

↓

Menu

↓

Order

↓

Kitchen

↓

Billing



Payment



Feedback

14. ONLINE ORDER FLOW

Customer



Website



Checkout



Payment



Kitchen



Preparation



Pickup / Delivery



Feedback

15. RESERVATION SOP

Reservation Received

↓

Availability Check

↓

Confirmation

↓

Reminder

↓

Arrival

↓

Dining

↓

Completion

↓

Feedback

16. INVENTORY SOP

Supplier

↓

Goods Received

↓

Quality Check

↓

Stock Entry

↓

Storage

↓

Consumption

↓

Waste

↓

Reorder

17. PURCHASE SOP

Low Stock Alert

↓

Manager Approval

↓

Purchase Order

↓

Supplier

↓

Delivery

↓

Verification

↓

Payment

↓

Inventory Update

18. REFUND SOP

Customer Request

↓

Manager Approval

↓

Payment Verification

↓

Refund

↓

Invoice Adjustment

↓

Audit Log

↓

Customer Notification

19. COMPLAINT HANDLING

Receive

↓

Understand

↓

Investigate

↓

Resolve

↓

Document

↓

Follow Up

↓

Improve Process

Never ignore customer feedback.

20. STAFF ONBOARDING

New Employee

↓

Account Creation

↓

Role Assignment

↓

Training

↓

System Access



Shadow Shift



Assessment



Go Live

21. STAFF OFFBOARDING

Deactivate Account



Remove Access



Archive Records



Return Equipment



Final Reports



Audit

22. DAILY REPORTS

Revenue

Orders

Reservations

Inventory

Payments

Reviews

Refunds

Kitchen Performance

Export PDF automatically.

23. WEEKLY REPORTS

Sales Trends

Popular Dishes

Inventory Consumption

Customer Growth

Marketing Results

Staff Performance

24. MONTHLY REPORTS

Profit

Loss

GST

Inventory Audit

Customer Retention

Campaign ROI

AI Insights

Business Growth

25. CUSTOMER EXPERIENCE SOP

Customer greeted.

↓

Order placed quickly.

↓

Food delivered on time.

↓

Payment smooth.

↓

Feedback requested.

↓

Follow-up for repeat visit.

Every interaction should reinforce the restaurant's premium brand.

26. MARKETING SOP

Create Campaign

↓

Schedule

↓

Launch

↓

Track Performance

↓

Analyze

↓

Optimize

Supported channels:

WhatsApp

Instagram

Email

Future integrations can be added.

27. AI ASSISTANT SOP

AI may assist with:

Sales summaries

Inventory suggestions

Menu recommendations

Report generation

Trend analysis

AI must never perform financial or operational actions without human approval.

28. DATA GOVERNANCE

Only authorized staff may access:

Financial reports

Employee data

Customer personal information

System settings

Every access is logged.

29. TRAINING PROGRAM

Training modules:

System Introduction

POS

Kitchen Display

Inventory

Reports

Customer Service

Security

Emergency Procedures

Certification before independent operation.

30. KNOWLEDGE BASE

Maintain documentation for:

FAQs

Troubleshooting

Printer setup

Payment issues

Inventory workflows

Reports

AI usage

System updates

Accessible from within the admin dashboard.

31. CHANGE MANAGEMENT

When introducing new features:

Communicate changes.

Update documentation.

Train staff.

Collect feedback.

Monitor adoption.

Avoid disrupting daily operations.

32. BUSINESS CONTINUITY

Manual procedures should exist for:

Internet outage

Printer failure

Payment gateway outage

Power outage

System maintenance

Resume digital operations as soon as possible.

33. OPERATIONAL KPIs

Track:

Average order preparation time

Average checkout time

Customer satisfaction

Repeat customer rate

Inventory waste

Order accuracy

Reservation utilization

Daily revenue

Staff productivity

34. GOVERNANCE

Weekly management meeting:

Operations review

Customer feedback

Inventory

Incidents

Improvements

Monthly executive review:

Financial performance

Growth

Technology updates

Strategic planning

35. CONTINUOUS IMPROVEMENT

Establish a feedback loop:

Customers

↓

Staff

↓

Managers

↓

Reports

↓

Product Improvements

↓

Updated SOPs

Review and refine processes regularly.

36. ACCEPTANCE CRITERIA

The operational framework is complete when:

- Every staff role has documented responsibilities.
 - Standard Operating Procedures exist for all daily workflows.
 - Opening and closing checklists are defined.
 - Training materials are available.
 - Reporting schedules are established.
 - Incident and complaint handling processes are documented.
 - Business continuity plans are prepared.
 - Operational KPIs are tracked and reviewed.
-

FINAL VISION

The Restaurant Digital Platform should become the **operating backbone** of the restaurant.

The software should not simply record transactions—it should standardize operations, reduce errors, improve customer experience, empower staff, and provide management with the information needed to make better decisions.

Technology, people, and processes should work together as one cohesive system.

RESTAURANT DIGITAL PLATFORM

ENTERPRISE PRODUCT GOVERNANCE & LONG-TERM EVOLUTION MANUAL

VOLUME 20

PRODUCT GOVERNANCE, PLATFORM EVOLUTION, SCALING STRATEGY, MULTI-BRANCH ARCHITECTURE & 5-YEAR ROADMAP

1. INTRODUCTION

The Restaurant Digital Platform is not a one-time software project.

It is a continuously evolving digital product.

This document defines:

- Product Governance
- Technical Governance
- Business Governance
- Long-term Architecture
- Product Evolution
- AI Roadmap
- Scalability Strategy
- Technical Debt Management
- Innovation Framework

The goal is to ensure that the platform remains valuable, maintainable, and adaptable for years.

2. LONG-TERM PRODUCT VISION

The platform should evolve from:

Restaurant Website



Restaurant Operations Platform



Restaurant Intelligence Platform



Restaurant Business Operating System



Restaurant AI Platform



Multi-Branch Restaurant ERP



Franchise Management Platform



Enterprise Restaurant Ecosystem

Every stage builds upon the previous one without requiring a rewrite.

3. PRODUCT GOVERNANCE

Create a Product Steering Committee.

Members:

Restaurant Owner

↓

Product Manager

↓

Engineering Lead

↓

Operations Manager

↓

Finance Representative

↓

Marketing Representative

Meet monthly.

Responsibilities:

Prioritize roadmap

Approve major features

Review customer feedback

Allocate budget

Approve architecture changes

4. ENGINEERING GOVERNANCE

Establish an Architecture Review Board.

Responsibilities:

Review architectural changes

Approve new technologies

Maintain coding standards

Prevent unnecessary complexity

Manage technical debt

Review security decisions

No major architectural changes without review.

5. ARCHITECTURE EVOLUTION

Maintain backward compatibility whenever possible.

Introduce changes using:

API versioning

↓

Feature flags

↓

Database migrations

↓

Deprecation policy

↓

Monitoring

↓

Gradual rollout

Never introduce breaking changes without a migration path.

6. FEATURE LIFECYCLE

Every feature progresses through:

Idea



Discovery



Research



Design



Prototype



Development



Testing



Beta



General Availability



Maintenance



Deprecation



Retirement

Each stage has defined entry and exit criteria.

7. ROADMAP MANAGEMENT

Maintain three horizons:

Short Term

0–6 Months

Medium Term

6–18 Months

Long Term

18–60 Months

Review quarterly.

8. VERSIONING STRATEGY

Semantic Versioning.

Major

Breaking changes

Minor

New features

Patch

Bug fixes

Examples:

v1.0.0

v1.2.0

v1.2.5

Document every release.

9. TECHNICAL DEBT MANAGEMENT

Track debt explicitly.

Categories:

Code Quality

Architecture

Infrastructure

Documentation

Testing

Security

Assign:

Owner

Priority

Estimated effort

Target release

Allocate time in every sprint for debt reduction.

10. CUSTOMER FEEDBACK LOOP

Collect feedback from:

Website



Google Reviews



WhatsApp



Staff



Support



Analytics



Surveys

Convert validated insights into backlog items.

11. PRODUCT ANALYTICS

Monitor:

Popular menu items

Order completion rate

Reservation conversion

Checkout abandonment

Average order value

Customer retention

Repeat purchases

Campaign performance

Use data to guide product decisions.

12. AI EVOLUTION ROADMAP

Phase 1

AI Reporting

↓

Phase 2

AI Recommendations

↓

Phase 3

AI Customer Assistant

↓

Phase 4

AI Operations Assistant

↓

Phase 5

AI Forecasting

↓

Phase 6

AI Autonomous Optimization

Human approval remains mandatory for financial or operational changes.

13. MULTI-BRANCH READINESS

Design for future expansion.

Entities gain branch awareness.

Examples:

Orders

Inventory

Employees

Reservations

Reports

Settings

Support centralized and branch-specific management.

14. FRANCHISE SUPPORT

Future capabilities:

Franchise onboarding

Branch templates

Shared menu

Localized pricing

Central reporting

Franchise analytics

Compliance monitoring

Role delegation

15. INTEGRATION STRATEGY

Future integrations:

Accounting software

Delivery aggregators

CRM platforms

Marketing automation

Payroll

ERP

Business intelligence tools

Expose stable APIs for integrations.

16. SECURITY GOVERNANCE

Annual:

Security review

Penetration testing

Dependency audit

Access review

Disaster recovery drill

Quarterly:

Secret rotation

Permission review

Policy updates

17. COMPLIANCE GOVERNANCE

Maintain:

Privacy policy

Terms of service

Data retention

Consent records

Tax documentation

Audit logs

Review whenever regulations change.

18. KNOWLEDGE GOVERNANCE

Maintain a centralized knowledge repository.

Sections:

Architecture

Product

Design

Engineering

Operations

Runbooks

Training

Release notes

Ensure documentation evolves with the product.

19. CHANGE MANAGEMENT

Every significant change requires:

Business justification

Impact analysis

Risk assessment

Rollback plan

Training plan

Communication plan

Success metrics

20. PRODUCT KPIs

Business KPIs:

Revenue growth

Order volume

Customer retention

Average order value

Reservation utilization

Operational KPIs:

Order preparation time

Inventory waste

Payment success rate

Printer uptime

System KPIs:

Availability

Latency

Error rate

Deployment frequency

Mean Time to Recovery

21. SCALABILITY STRATEGY

Growth stages:

Stage 1

Single restaurant

↓

Stage 2

Multiple branches

↓

Stage 3

City-wide operations

↓

Stage 4

Regional expansion



Stage 5

National franchise network

Architecture should support each stage with minimal redesign.

22. RISK GOVERNANCE

Maintain a risk register.

Examples:

Vendor lock-in

Third-party outages

Security threats

Data loss

Scope creep

Staff turnover

Technology obsolescence

Review quarterly.

23. BUSINESS CONTINUITY GOVERNANCE

Prepare plans for:

Cloud provider outage

Payment gateway disruption

Key employee absence

Major security incident

Natural disaster

Extended internet outage

Conduct annual simulation exercises.

24. INNOVATION FRAMEWORK

Evaluate new technologies through:

Research

Proof of concept

Architecture review

Security review

Performance testing

Pilot

Adoption

Avoid adopting technology solely because it is new.

25. SUNSETTING POLICY

When retiring a feature:

Announce deprecation

Provide migration path

Support transition period

Archive documentation

Remove safely

Maintain data integrity.

26. PRODUCT INVESTMENT MODEL

Prioritize work using a balanced portfolio:

40% Core improvements

30% Customer-requested features

20% Technical debt

10% Innovation and experimentation

Adjust annually based on business needs.

27. TEAM EVOLUTION

As the platform grows:

Stage 1

Full-stack developer

↓

Stage 2

Frontend + Backend

↓

Stage 3

Dedicated QA



Stage 4

DevOps



Stage 5

Product Manager



Stage 6

Security & Data specialists

Scale the organization alongside the product.

28. ARCHITECTURE REVIEW SCHEDULE

Monthly:

Code quality review

Quarterly:

Architecture review

Biannually:

Technology roadmap review

Annually:

Major platform health assessment

Document all outcomes.

29. FIVE-YEAR PRODUCT ROADMAP

Year 1

- Launch production platform.
- Stabilize operations.
- Optimize customer experience.

Year 2

- Advanced analytics.
- Loyalty program.
- AI reporting.
- Improved marketing automation.

Year 3

- Multi-branch support.
- Mobile applications.
- Delivery partner integrations.
- Advanced inventory forecasting.

Year 4

- Franchise management.
- Enterprise reporting.
- AI operational assistant.
- Predictive staffing.

Year 5

- AI-powered restaurant operating system.
 - Autonomous forecasting.
 - Dynamic pricing recommendations.
 - Cross-branch optimization.
 - Business intelligence suite.
-

30. SUCCESS CRITERIA

The platform is considered successful when:

- It supports daily restaurant operations reliably.
 - Customers prefer digital ordering and reservations.
 - Staff productivity improves measurably.
 - Operational errors decrease.
 - Management gains actionable business insights.
 - The architecture supports expansion without major rewrites.
 - Documentation remains current and trusted.
 - The platform continues to evolve through disciplined governance.
-

31. PLATFORM MANIFESTO

The Restaurant Digital Platform exists to help restaurants deliver exceptional customer experiences while simplifying operations.

Every feature should:

- Solve a real business problem.
- Improve the customer journey.
- Reduce manual effort.
- Increase operational visibility.
- Be maintainable and secure.
- Be measurable through clear outcomes.

Complexity should never be introduced without a clear benefit.

32. FINAL ACCEPTANCE CRITERIA

This documentation suite is complete when:

- Business vision is documented.
- Product requirements are defined.
- UX and UI are fully specified.
- Frontend and backend architectures are documented.

- Database, APIs, and infrastructure are designed.
 - Quality engineering and operations are established.
 - Governance and long-term evolution are planned.
 - The platform can be built, deployed, operated, maintained, and expanded using these documents as the primary reference.
-

FINAL VISION

The Restaurant Digital Platform should become far more than a website or POS.

It should serve as the restaurant's **digital backbone**—connecting customer experience, operations, finance, analytics, marketing, and future AI capabilities within a single, cohesive ecosystem.

Every architectural decision, design choice, operational process, and product enhancement should contribute to a platform that remains valuable, reliable, and adaptable for years.